Project Risk Analysis: A Practical Guide

From Monte Carlo to AI Agents

Paul Govan

2026-11-02

Table of contents

Preface **1**
- What This Book Is About . . . 1
- Who This Book Is For . . . 1
- How to Use This Book . . . 2
- Getting Started . . . 2
- A Note on Tone . . . 2
- Why the `PRA` Package? . . . 3
- A Reproducible Book . . . 3
- How to Cite . . . 4
- License . . . 4
- Acknowledgements . . . 4

1. Into the Unknown: An Introduction to Project Risk Analysis **5**
- 1.1. What Is Project Risk? . . . 5
- 1.2. The Quantitative Toolkit . . . 5
- 1.3. How to Navigate This Book . . . 6
- 1.4. A Map of the Methods . . . 7
  - 1.4.1. Early Estimation . . . 7
  - 1.4.2. During Execution . . . 7
  - 1.4.3. As Evidence Arrives . . . 7
  - 1.4.4. Understanding Structure . . . 7
  - 1.4.5. AI-Assisted Analysis . . . 7
- 1.5. Installing the Package . . . 8
- 1.6. A Note on Uncertainty About Uncertainty . . . 8

2. Roll the Dice: Monte Carlo Simulation **9**
- 2.1. How Monte Carlo Simulation Works . . . 9
- 2.2. Example . . . 10
  - 2.2.1. Correlation Matrix . . . 10
  - 2.2.2. Run the Simulation . . . 10
  - 2.2.3. Distribution of Outcomes . . . 11
- 2.3. Interpreting Percentiles . . . 12
- 2.4. Contingency Analysis . . . 13
- 2.5. Sensitivity Analysis . . . 13
- 2.6. Summary . . . 15
- 2.7. Exercises . . . 16

3. May I Have a (Second) Moment? The Second Moment Method **17**
- 3.1. When to Use SMM . . . 17
- 3.2. How It Works . . . 18

3.3. Example . . . 18
3.4. Implied Distribution and Confidence Interval . . . 19
3.5. Comparison with Monte Carlo Simulation . . . 21
3.6. Benefits and Limitations . . . 22
3.7. Summary . . . 22
3.8. Exercises . . . 22

4. Who's Driving? Sensitivity Analysis **25**
4.1. What Is Sensitivity Analysis? . . . 25
4.1.1. Variance Decomposition . . . 25
4.2. Setup . . . 26
4.3. Computing Sensitivity . . . 26
4.3.1. Interpretation . . . 27
4.4. Tornado Chart . . . 27
4.5. With Correlated Tasks . . . 28
4.6. Summary . . . 30
4.7. Exercises . . . 31

5. Keeping Score: Earned Value Management **33**
5.1. The Three Core Numbers . . . 33
5.2. Key Metrics . . . 34
5.3. Example Setup . . . 34
5.3.1. Planned Value (PV) . . . 34
5.3.2. Earned Value (EV) . . . 35
5.3.3. Actual Cost (AC) . . . 35
5.3.4. Performance Indicators . . . 35
5.4. Forecasting: Estimate at Completion (EAC) . . . 36
5.4.1. EAC Comparison Table . . . 37
5.5. Additional Metrics . . . 37
5.6. Performance Trend Chart . . . 38
5.7. Summary . . . 40
5.8. Exercises . . . 40

6. I Had a Feeling: Bayesian Risk Inference **41**
6.1. The Core Idea . . . 41
6.2. Causal Structure . . . 42
6.3. Step 1: Prior Risk Probability . . . 43
6.4. Step 2: Prior Cost Distribution . . . 44
6.5. Step 3: Posterior Risk Probability (Bayesian Update) . . . 44
6.5.1. Prior vs. Posterior Probability . . . 44
6.6. Step 4: Posterior Cost Distribution . . . 46
6.6.1. Prior vs. Posterior Cost Distribution . . . 46
6.7. The Workflow . . . 47
6.8. Summary . . . 48
6.9. Exercises . . . 48

7. S Is for Success: Sigmoidal Learning Curves **49**
7.1. Why Sigmoidal? . . . 49

7.2. The Three Models . . . 50
7.3. Example: Fitting a Logistic Model . . . 50
7.3.1. Fit the Model . . . 50
7.3.2. Assess Fit Quality . . . 50
7.3.3. Plot with Confidence Bands . . . 51
7.3.4. Predict Future Completion . . . 52
7.4. Comparing All Three Model Types . . . 53
7.5. The Workflow . . . 54
7.6. Summary . . . 55
7.7. Exercises . . . 55

8. It's a Small World After All: Probabilistic Networks 57
8.1. What Is a Bayesian Network? . . . 57
8.2. Project Setup . . . 58
8.2.1. Tasks . . . 58
8.2.2. Resources . . . 58
8.2.3. Risks . . . 59
8.3. Building the Bayesian Network . . . 59
8.3.1. Nodes . . . 59
8.3.2. Edges . . . 60
8.3.3. Distributions . . . 60
8.3.4. Build the Graph . . . 61
8.4. Inference: Forward Simulation . . . 62
8.5. Learning: Incorporating New Evidence . . . 63
8.6. Updating: Modifying the Network . . . 64
8.7. The Four Core Functions . . . 66
8.8. Summary . . . 67
8.9. Exercises . . . 67

9. Everything Is Connected: Design Structure Matrices 69
9.1. What Is a DSM? . . . 69
9.2. Rebuilding the Project Network . . . 70
9.3. The Adjacency Matrix . . . 71
9.4. The Resource-Task Matrix . . . 72
9.5. Parent DSM . . . 72
9.6. The Risk-Resource Matrix . . . 73
9.7. Grandparent DSM . . . 74
9.8. Interpreting the DSM . . . 75
9.9. From DSM to Decision . . . 76
9.10. Summary . . . 76
9.11. Exercises . . . 77

10. The Portfolio Problem: When Risks Are Shared 79
10.1. The Portfolio Case Study . . . 79
10.2. Setup . . . 80
10.2.1. Tasks . . . 80
10.2.2. Risks and Root Cause . . . 81
10.3. Building the Portfolio Network . . . 82

10.4. Observational Distribution . . . 84
10.4.1. Task-Cost Correlation Matrix . . . 84
10.5. Seeing vs. Doing at the Risk Level . . . 85
10.6. Enterprise vs. Project-Scoped Intervention . . . 87
10.7. Risk Importance Ranking . . . 88
10.8. Summary Table . . . 90
10.9. Summary . . . 90
10.10Exercises . . . 91

11. Write Once, Trust Always: Reproducible Risk Analysis with Quarto **93**
11.1. The Spreadsheet Problem . . . 93
11.2. What is Quarto? . . . 94
11.3. Your First Risk Report . . . 95
11.3.1. Step 1: Install Quarto . . . 95
11.3.2. Step 2: Create the Document . . . 95
11.3.3. Step 3: Write the Analysis . . . 96
11.3.4. Step 4: Render . . . 96
11.4. Parameterized Reports . . . 97
11.4.1. Defining Parameters . . . 97
11.4.2. Using Parameters in Code . . . 98
11.4.3. Rendering Multiple Scenarios . . . 98
11.5. Version Control with Git . . . 98
11.5.1. What to Commit . . . 99
11.5.2. A Minimal Workflow . . . 99
11.6. This Book as a Living Example . . . 100
11.7. Exercises . . . 101

12. Your AI Co-Pilot: Agentic Risk Analysis **103**
12.1. How PRA Talks to AI Agents . . . 103
12.2. Prerequisites . . . 104
12.3. The MCP Server . . . 104
12.3.1. Starting the server . . . 104
12.3.2. Connecting from Claude Code . . . 104
12.3.3. Connecting from Claude Desktop . . . 104
12.4. Agent Skills . . . 105
12.5. From Request to Tool Call . . . 105
12.6. Tool Reference . . . 107
12.7. Summary . . . 107
12.8. Exercises . . . 108

Appendices **109**

A. Glossary **109**
A.1. Probability & Distributions . . . 109
A.2. Monte Carlo Simulation & Sensitivity . . . 110
A.3. Second Moment Method . . . 110
A.4. Earned Value Management . . . 110

A.5. Bayesian Inference . . . 111
A.6. Learning Curves . . . 112
A.7. Structural Methods . . . 112
A.8. Agentic Framework . . . 113

B. Mathematical Derivations **115**
B.1. 1. Why Total Variance Is a Sum . . . 115
B.1.1. Independent Tasks . . . 115
B.1.2. Correlated Tasks . . . 116
B.1.3. Why the Normal Approximation Works . . . 116
B.2. 2. Variance Formulas for Standard Distributions . . . 116
B.2.1. Normal Distribution . . . 117
B.2.2. Triangular Distribution . . . 117
B.2.3. Uniform Distribution . . . 117
B.3. 3. The Sensitivity Index . . . 118
B.3.1. Derivation . . . 118
B.3.2. The Independent Case . . . 118
B.3.3. Interpretation . . . 118
B.4. 4. Bayesian Updating for Project Risk . . . 119
B.4.1. Bayes' Theorem . . . 119
B.4.2. Multiple Independent Root Causes (Prior) . . . 119
B.4.3. Posterior Update (Observed Causes) . . . 120
B.4.4. Why Observing a Cause Increases Risk Probability . . . 120

C. Quick Reference **121**
C.1. Monte Carlo Simulation . . . 121
C.2. Second Moment Method . . . 121
C.3. Sensitivity Analysis . . . 122
C.4. Contingency & Correlation . . . 122
C.5. Earned Value Management . . . 122
C.6. Bayesian Risk . . . 123
C.7. Learning Curves . . . 124
C.8. Design Structure Matrices . . . 124
C.9. Probabilistic Networks . . . 125
C.10. Agentic Framework . . . 126

D. Case Study: Riverside Bridge Replacement **127**
D.1. The Project . . . 127
D.2. Step 1: Monte Carlo Simulation . . . 128
D.3. Step 2: Sensitivity Analysis . . . 129
D.4. Step 3: Second Moment Method . . . 130
D.5. Step 4: Earned Value Management . . . 131
D.6. Step 5: Bayesian Risk Update . . . 133
D.7. Step 6: Learning Curve . . . 133
D.8. Step 7: Design Structure Matrix . . . 135
D.9. Step 8: Probabilistic Network . . . 137
D.10. Step 9: Agentic Analysis . . . 140
D.11. What the Methods Told Us . . . 140

E. Exercise Solutions **143**
E.1. Monte Carlo Simulation (Chapter 2) . . . 143
E.1.1. Exercise 3 : Correlation Experiment . . . 143
E.1.2. Exercise 5 : Real-World Application . . . 144
E.2. Sensitivity Analysis (Chapter 4) . . . 145
E.2.1. Exercise 4 : Correlation Direction . . . 145
E.2.2. Exercise 5 : From Sensitivity to Contingency . . . 146
E.3. Second Moment Method (Chapter 3) . . . 147
E.3.1. Exercise 3 : Normality Check . . . 147
E.3.2. Exercise 5 : When to Trust SMM . . . 148
E.4. Earned Value Management (Chapter 5) . . . 149
E.4.1. Exercise 5 : Recovering Project Trend . . . 149
E.5. Bayesian Risk Inference (Chapter 6) . . . 150
E.5.1. Exercise 4 : Intuition Check . . . 150
E.5.2. Exercise 5 : Sequential Updating . . . 151
E.6. Sigmoidal Learning Curves (Chapter 7) . . . 152
E.6.1. Exercise 3 : Confidence Band Width . . . 152
E.6.2. Exercise 5 : Beyond Completion Percentages . . . 153
E.7. Design Structure Matrices (Chapter 9) . . . 154
E.7.1. Exercise 3 : Risk Propagation . . . 154
E.7.2. Exercise 5 : From DSM to Monte Carlo . . . 155
E.8. Probabilistic Networks (Chapter 8) . . . 157
E.8.1. Exercise 3 : Add a QA Risk . . . 157
E.8.2. Exercise 5 : Two Risks, One Resource . . . 158
E.9. Portfolio Networks (Chapter 10) . . . 159
E.9.1. Exercise 4 : Add a Fourth Project . . . 159
E.9.2. Exercise 5 : Causal Graph Design . . . 160
E.10. Agentic Risk Analysis (Chapter 12) . . . 160
E.10.1. Exercise 5 : Skill Authoring . . . 160

References **163**

Preface

Every project starts with a plan. Then the project starts.

Deadlines shift. Costs drift. Risks that seemed unlikely have a way of becoming very real. Uncertainty is not an edge case in project management, it is woven into the work itself. The question is not whether it will show up, but whether you'll have the tools to quantify it, communicate it, and manage it thoughtfully.

This book is your toolkit.

What This Book Is About

Project Risk Analysis: A Practical Guide is a hands-on introduction to quantitative methods for managing uncertainty in project schedules and costs. We cover the major techniques used by risk analysts, project managers, and engineers: Monte Carlo simulation, second moment methods, earned value management, Bayesian inference, learning curves, design structure matrices, probabilistic networks, and AI-powered risk analysis, all through working R code using the `PRA` package.

The goal is not to overwhelm you with theory (though we won't shy away from the math when it helps). The goal is to give you a real edge: the ability to take a project estimate, ask the right "what if" questions, and produce defensible numbers for schedule and cost contingency.

Who This Book Is For

- **Project managers** who want to move beyond gut-feel estimates and "10% contingency, because that's what we always do"
- **Engineers and analysts** who know statistics but haven't applied it to project risk
- **Students** in construction management, systems engineering, or operations research programs
- **R users** who want practical, well-documented examples of risk analysis workflows

You don't need to be a statistician to follow along, but you should be comfortable with basic probability (what is a mean? what is a variance?) and have at least a passing familiarity with R.

How to Use This Book

Each chapter is self-contained and covers one method. If you're in a hurry and just need Monte Carlo simulation, go straight to Chapter 2. If you already know EVM and want to learn Bayesian inference, skip ahead to Chapter 6. That said, the chapters do build on each other conceptually, and reading in order will give you the fullest picture.

Every chapter follows the same structure:

1. **Why this method?**: A short, honest answer to "when would I actually use this?"
2. **How it works**: Just enough theory to understand what the code is doing
3. **A worked example**: Fully reproducible R code from start to finish
4. **Exercises**: Problems that range from "verify you followed along" to "extend this to a real scenario"

Getting Started

All examples use the `PRA` package. Install it from CRAN:

```
install.packages("PRA")
```

Or get the development version from GitHub:

```
# install.packages("remotes")
remotes::install_github("paulgovan/PRA")
```

Then load it at the start of any session:

```
library(PRA)
```

Some chapters (especially the network chapters) use additional packages like `igraph` and `networkD3`, and the agentic chapter uses `ellmer`, `mcptools`, and `jsonlite` for the MCP server. Prerequisites are listed at the top of each chapter.

A Note on Tone

Risk analysis has a reputation for being dry. Probability tables. Spreadsheets. Earnest diagrams with boxes and arrows. We've done our best to keep things lively, with catchy chapter titles, the occasional metaphor, and examples drawn from real project contexts. If a chapter made you smile, that was on purpose. If it also made you understand Bayesian updating, that's the win.

Why the PRA Package?

Most project risk analysis is done in spreadsheets. Spreadsheets work, until they don't: a formula breaks, a column gets deleted, and suddenly your P95 estimate is based on a range that ends three rows too early. The PRA package takes the same methods and makes them:

- **Tested.** Every function ships with a full test suite. You can trust that `mcs()` is doing what the documentation says.
- **Documented.** Every parameter has a type, a valid range, and an example. No more guessing what "n" means.
- **Reproducible.** An R script is a record of exactly what you did. Share it, version-control it, re-run it six months later, and it produces the same result.
- **Composable.** The output of `mcs()` flows directly into `contingency()` and `sensitivity()`. Methods chain together naturally rather than requiring manual copy-paste between tabs.

Learning risk analysis through a software package also enforces discipline: you have to be explicit about your assumptions. Code makes assumptions explicit and verifiable.

A Reproducible Book

Every code block in this book is live. When the book is built, R executes the code and embeds the actual output: the numbers, the plots, the tables. There is no "trust us, this is what you'd see." If a result looks wrong, you can run the same code yourself and verify it.

This matters because:

- **You can experiment.** Change a parameter, re-run the chunk, and immediately see how the output shifts. The best way to develop intuition for Monte Carlo simulation is to change the number of samples and watch what happens to the tails.
- **The book stays honest.** A static book can silently go out of date. A reproducible book will break noisily if a function changes behavior, which means the examples you're reading have been verified against the current version of PRA.
- **You learn a transferable skill.** Writing reproducible analyses in Quarto or R Markdown is increasingly expected in data-driven fields. Following along with this book is practice for that.

To reproduce the entire book yourself:

```
quarto render book/
```

All outputs are generated from scratch in a clean R session.

How to Cite

If you use this book in your work, please cite it as:

> Govan, P. (2025). *Project Risk Analysis: A Practical Guide: From Monte Carlo to AI Agents*. https://github.com/paulgovan/PRA

Or in BibTeX:

```
@book{govan_pra,
  author    = {Paul Govan},
  title     = {Project Risk Analysis: A Practical Guide},
  subtitle  = {From Monte Carlo to AI Agents},
  year      = {2025},
  url       = {https://github.com/paulgovan/PRA},
  note      = {CC BY 4.0}
}
```

License

The `PRA` R package is released under the MIT License. You are free to use, modify, and distribute it, provided the original copyright notice is retained.

This book is released under CC BY 4.0. You are free to share and adapt the material for any purpose, provided you give appropriate credit.

Acknowledgements

The `PRA` package and this book grew out of research on the resource-based view of project risk management (Govan and Damnjanovic 2016; Govan 2014). Thanks to the open-source R community for the packages that make quantitative risk analysis accessible, and to every project manager who has ever looked at a schedule baseline and wondered whether the numbers were as solid as they looked.

There is only one way to find out: quantitatively.

1. Into the Unknown: An Introduction to Project Risk Analysis

> *"In preparing for battle I have always found that plans are useless, but planning is indispensable."* — Dwight D. Eisenhower

Every project estimate is really a best guess. Your team genuinely believes the schedule when they write it, and that's the right attitude to have. But once the project starts, reality has a way of offering its own revisions. Materials arrive late. Scope grows. That team member who was penciled in for three concurrent tasks runs into the usual constraints of time and bandwidth.

This is where quantitative methods come in.

1.1. What Is Project Risk?

Risk, in the technical sense, is uncertainty that matters. Not all uncertainty is risk; we don't worry about the exact number of paperclips in the supply closet, but uncertainty about task durations, resource availability, weather, regulatory approvals, and supplier performance absolutely is. Risk has two components:

1. **Probability**: How likely is this uncertain event to occur?
2. **Impact**: If it does occur, how much does it affect cost or schedule?

Traditional project management often handles risk qualitatively: a risk register with "High / Medium / Low" likelihood and impact ratings, maybe a 10% contingency tacked onto the budget because that's what was done last time. This approach is comfortable, but it lacks precision. It cannot tell you whether your P80 schedule estimate is 3 weeks or 3 months above the base estimate. It cannot tell you which risk deserves the most mitigation dollars.

Quantitative risk analysis, the subject of this book, can.

1.2. The Quantitative Toolkit

The `PRA` package provides a collection of methods that complement each other. Think of them as different lenses on the same project:

Chapter	Method	What it answers
Chapter 2	Monte Carlo Simulation	What is the full distribution of possible outcomes?

Chapter	Method	What it answers
Chapter 3	Second Moment Method	What are the mean and variance quickly, analytically?
Chapter 4	Sensitivity Analysis	Which task drives the most variance?
Chapter 5	Earned Value Management	How is the project actually performing right now?
Chapter 6	Bayesian Inference	How should I update my risk estimate given new evidence?
Chapter 7	Learning Curves	When will the team reach full productivity?
Chapter 9	Design Structure Matrices	Which tasks are most tightly coupled through shared resources?
Chapter 8	Probabilistic Networks	How do risks propagate through the project?
Chapter 10	Portfolio Networks	How do shared risks affect multiple projects at once?
Chapter 12	Agentic Risk Analysis	Can I just ask an AI to run all of this?

These methods are not competing alternatives; they are a progression. You might use the Second Moment Method for a quick back-of-the-envelope estimate, then Monte Carlo for the formal risk report, and a Bayesian network when you need to understand how one risk cascades into others.

1.3. How to Navigate This Book

Each chapter is self-contained; feel free to jump to the method you need. That said, there are natural dependencies: Monte Carlo (Chapter 2) and the Second Moment Method (Chapter 3) build on the same uncertainty concepts; probabilistic networks (Chapter 8, Chapter 10) extend Bayesian inference (Chapter 6); and the agent chapter (Chapter 12) assumes familiarity with all prior methods.

Your situation	Start here
Building an initial schedule or cost estimate	Chapter 2 or Chapter 3
Project underway; need to assess performance	Chapter 5
New information arrived; want to update risk estimates	Chapter 6
Tasks keep failing together; want to understand why	Chapter 9 or Chapter 8
Managing a portfolio of projects with shared resources	Chapter 10
Forecasting when the team will reach full productivity	Chapter 7
Want AI-assisted analysis without writing code every time	Chapter 12

1.4. A Map of the Methods

1.4.1. Early Estimation

When you are building the initial project estimate, you have distributions but no data yet. The **Second Moment Method** (Chapter Chapter 3) and **Monte Carlo Simulation** (Chapter Chapter 2) are your tools here. SMM is fast and analytical, giving you a mean and variance in seconds using only the mean and variance of each task. MCS is slower but more powerful, able to use any distribution shape and handling correlations between tasks.

1.4.2. During Execution

Once the project is underway and you have actual cost and schedule data, **Earned Value Management** (Chapter Chapter 5) tracks how the project is performing against the plan. EVM's performance indices (CPI, SPI) are early-warning signals: if your Cost Performance Index drops below 1.0 in period 3, you want to know what the final cost forecast looks like before it becomes a surprise in period 10.

1.4.3. As Evidence Arrives

Bayesian Inference (Chapter Chapter 6) lets you update your risk estimates as you learn more about the project environment. Observed that a key supplier is struggling? Update your material cost distribution accordingly. This is particularly powerful in phased projects where information accumulates progressively.

1.4.4. Understanding Structure

Design Structure Matrices (Chapter Chapter 9) and **Probabilistic Networks** (Chapters Chapter 8 and Chapter 10) address the question of *why* risks correlate. When two tasks both depend on the same crew, a labor shortage affects them both simultaneously. These methods make that structure explicit and quantifiable. **Learning Curves** (Chapter Chapter 7) capture a different kind of structure: how a team's productivity evolves over time.

1.4.5. AI-Assisted Analysis

Chapter Chapter 12 introduces the `PRA` agentic layer: an MCP server that exposes every `PRA` function as a tool for AI agents like Claude Code, together with a set of Agent Skills that tell the agent when and how to use each one. Think of it as having a risk analyst on call who always has the EVM formulas at hand.

1.5. Installing the Package

```
install.packages("PRA")
```

Or the development version:

```
remotes::install_github("paulgovan/PRA")
```

Load it in any session:

```
library(PRA)
```

1.6. A Note on Uncertainty About Uncertainty

> ⚠ Garbage In, Garbage Out
>
> Quantitative risk analysis can give a false sense of precision. The output of a Monte Carlo simulation, "P95 = 47.3 weeks", sounds authoritative. But that number is only as good as the distributions you fed it.
> The skill is not in running the software. The skill is in:
>
> 1. **Choosing distributions** that honestly represent what you know and don't know
> 2. **Identifying correlations** that actually exist in your project
> 3. **Communicating results** in a way that drives good decisions rather than false confidence
>
> Every chapter in this book includes exercises that push you to think critically about these choices, not just to run the code, but to question whether the inputs are defensible.

With that caveat in mind, let's run some simulations.

2. Roll the Dice: Monte Carlo Simulation

> *"Anyone who considers arithmetical methods of producing random digits is, of course, in a state of sin."* — John von Neumann

Here is the core insight behind Monte Carlo simulation: instead of guessing a single answer, roll the dice thousands of times and let the distribution speak for itself. The method was named after the famous casino in Monaco, not because it involves gambling, but because it involves random sampling. If you can describe your uncertainty with a probability distribution, Monte Carlo can turn that uncertainty into a full picture of possible project outcomes.

Monte Carlo simulation turns that randomness into insight.

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain the five steps of a Monte Carlo simulation
2. Run `mcs()` with multiple distributions and a correlation matrix
3. Interpret P50, P80, and P95 percentiles in project schedule context
4. Compute a schedule contingency reserve from simulation output
5. Identify which tasks drive the most variance using a tornado chart

2.1. How Monte Carlo Simulation Works

Monte Carlo (MC) simulation is a quantitative risk analysis technique that models uncertainty by running thousands of simulated project outcomes. Instead of using single-point estimates for task durations or costs, each task is described by a probability distribution. The simulation draws random samples from these distributions, sums them to get a total outcome, and repeats this thousands of times to build a full picture of possible project results (Vose 2008).

The Five Steps of Monte Carlo Simulation

1. **Model Definition**: Define project tasks and the variables that drive uncertainty (durations, costs).
2. **Assign Distributions**: Choose a probability distribution for each uncertain variable (e.g., triangular for tasks with optimistic/likely/pessimistic estimates).
3. **Specify Correlations**: If tasks are related (e.g., both affected by a shared risk), set a correlation coefficient between them.
4. **Run Simulation**: Draw random samples and compute the total outcome for each iteration (typically 10,000+).

5. **Analyze Results**: Summarize the distribution of totals using percentiles, mean, and variance.

2.2. Example

```
library(PRA)
library(ggplot2)
set.seed(42)
```

We model a 3-task project (in weeks). Task A follows a normal distribution, Task B has a triangular distribution (optimistic/most-likely/pessimistic), and Task C is uniformly distributed.

```
num_simulations <- 10000
task_distributions <- list(
  list(type = "normal", mean = 10, sd = 2),         # Task A
  list(type = "triangular", a = 5, b = 10, c = 15), # Task B
  list(type = "uniform", min = 8, max = 12)           # Task C
)
```

2.2.1. Correlation Matrix

Tasks often move together due to shared resources or external risks. The correlation matrix captures this. Values range from -1 (perfectly opposed) to $+1$ (perfectly aligned); 0 means independent. Here Tasks A and B have moderate positive correlation (0.5), meaning delays in one tend to coincide with delays in the other.

```
correlation_matrix <- matrix(c(
  1.0, 0.5, 0.3,
  0.5, 1.0, 0.4,
  0.3, 0.4, 1.0
), nrow = 3, byrow = TRUE)
```

2.2.2. Run the Simulation

```
results <- mcs(num_simulations, task_distributions, correlation_matrix)
```

```
cat("Mean Total Duration:  ", round(results$total_mean, 2), "weeks\n")
```

```
Mean Total Duration:   29.97 weeks
```

```
cat("Variance of Duration: ", round(results$total_variance, 2), "\n")
```

```
Variance of Duration:  16.87
```

```
cat("Std Dev of Duration:  ", round(results$total_sd, 2), "weeks\n")
```

```
Std Dev of Duration:   4.11 weeks
```

2.2.3. Distribution of Outcomes

The histogram below shows all 10,000 simulated total durations. The overlaid density curve reveals the shape of the distribution — notice the slight right skew from the triangular task.

```
hist_data <- results$total_distribution

hist(hist_data,
  breaks = 50, freq = FALSE,
  main = "Monte Carlo Simulation: Total Project Duration",
  xlab = "Total Duration (weeks)", col = "steelblue", border = "white"
)
lines(density(hist_data), col = "tomato", lwd = 2)
abline(v = results$total_mean, col = "black", lty = 2, lwd = 1.5)
legend("topright",
  legend = c("Density", paste0("Mean = ", round(results$total_mean, 1), " wks")),
  col = c("tomato", "black"), lty = c(1, 2), lwd = 2, bty = "n"
)
```

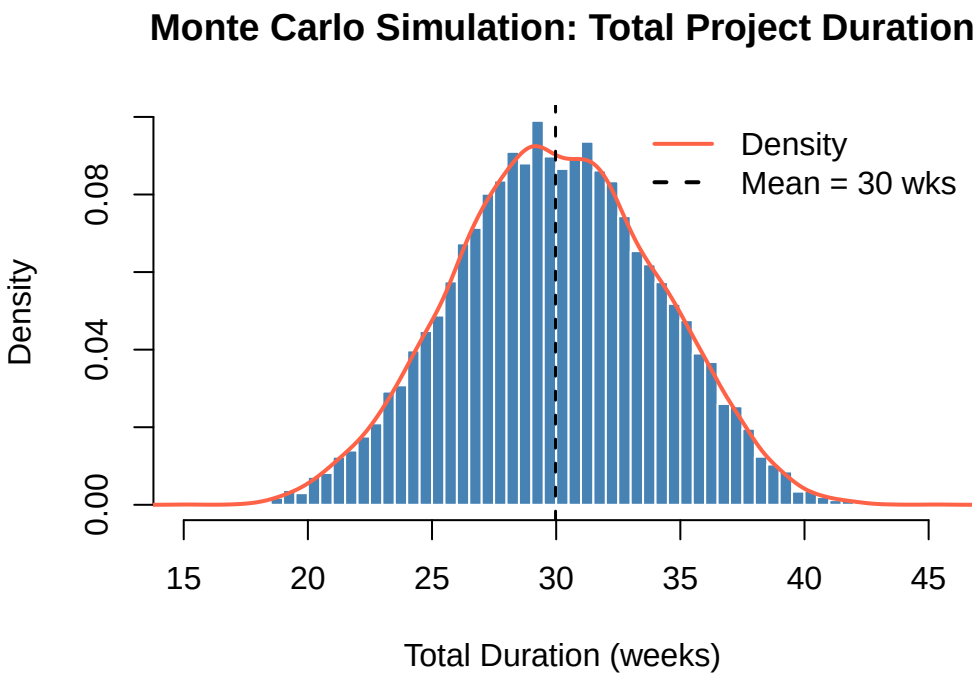


Figure 2.1.: Monte Carlo simulation results showing the full distribution of possible total project durations.

2.3. Interpreting Percentiles

The `mcs()` function returns key percentiles of the total distribution. These answer the question: *"What duration has X% probability of not being exceeded?"*

```
knitr::kable(
  data.frame(
    Percentile = c("P5", "P50 (Median)", "P95"),
    Duration   = round(results$percentiles, 1),
    Meaning    = c(
      "5% chance of finishing this fast or faster",
      "Equal chance of finishing above or below this",
      "95% chance of finishing by this date"
    )
  ),
  caption = "Simulation Percentiles"
)
```

Table 2.1.: Simulation Percentiles

	Percentile	Duration	Meaning
5%	P5	23.2	5% chance of finishing this fast or faster
50%	P50 (Median)	29.9	Equal chance of finishing above or below this
95%	P95	36.7	95% chance of finishing by this date

The P50 is your base estimate, the median, not the mean (though for symmetric distributions they're close). The P95 is the date you can be 95% confident in. The gap between P50 and P95 is your schedule risk.

2.4. Contingency Analysis

Contingency is the buffer added above the base estimate to cover uncertainty. A common approach is to use the difference between the P95 (or chosen confidence level) outcome and the P50 (base estimate) (Project Management Institute 2021).

```
contingency_val <- contingency(results, phigh = 0.95, pbase = 0.50)
cat("Schedule contingency (P95 - P50):", round(contingency_val, 2), "weeks\n")
```

```
Schedule contingency (P95 - P50): 6.8 weeks
```

```
cat(
  "There is a 95% chance the project will finish within",
  round(results$percentiles["95%"], 1), "weeks.\n"
)
```

```
There is a 95% chance the project will finish within 36.7 weeks.
```

Adding this contingency to the P50 estimate gives a 95% confidence of on-time delivery. Teams with low risk tolerance should use P95; those with higher tolerance might use P80.

2.5. Sensitivity Analysis

Sensitivity analysis identifies which tasks drive the most variability in the total outcome, the tasks that deserve the most management attention. The result is a **tornado chart**: the wider the bar, the bigger the impact on total schedule risk.

```r
sensitivity_results <- sensitivity(task_distributions, correlation_matrix)

sens_data <- data.frame(
  Task        = c("Task A (Normal)", "Task B (Triangular)", "Task C (Uniform)"),
  Sensitivity = sensitivity_results
)

p <- ggplot2::ggplot(
  sens_data,
  ggplot2::aes(x = Sensitivity, y = reorder(Task, Sensitivity))
) +
  ggplot2::geom_col(fill = "steelblue") +
  ggplot2::geom_text(ggplot2::aes(label = round(Sensitivity, 3)),
    hjust = -0.1, size = 3.5
  ) +
  ggplot2::labs(
    title = "Tornado Chart: Task Sensitivity",
    x     = "Sensitivity Coefficient",
    y     = NULL
  ) +
  ggplot2::xlim(0, max(sensitivity_results) * 1.2) +
  ggplot2::theme_minimal()

print(p)
```

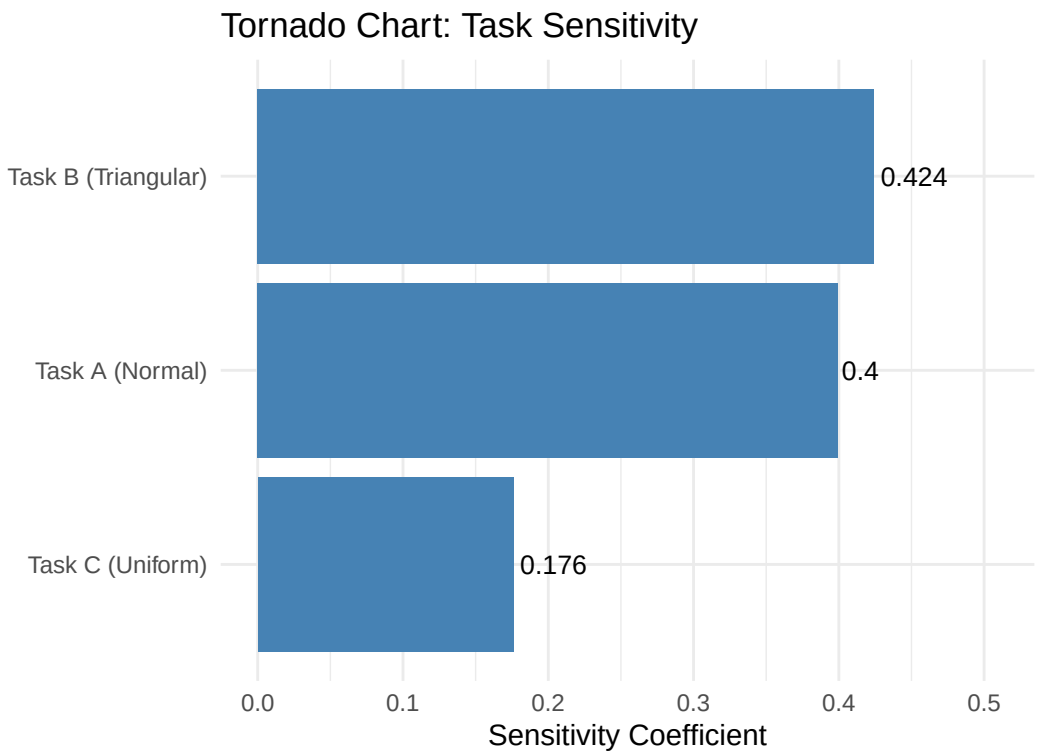


Figure 2.2.: Tornado chart showing each task's contribution to total schedule variance. Focus risk mitigation on the tallest bars.

Tasks with larger bars contribute more variance to the total. Prioritize risk mitigation efforts on the highest-sensitivity task. Even a small reduction in its uncertainty can meaningfully reduce overall project risk.

2.6. Summary

Key Takeaways

- Monte Carlo simulation runs thousands of iterations to produce a full distribution of project outcomes, not just a single number.
- **Percentiles** (P50, P80, P95) translate simulation output into concrete schedule or cost commitments at a stated confidence level.
- **Contingency** = P_high − P50: the buffer needed to achieve a given confidence level above the base estimate.
- **Sensitivity analysis** (tornado chart) reveals which tasks drive the most variance, so focus mitigation there first.
- Positive correlation between tasks *always* increases total variance; ignoring it leads to underestimating risk.

For early-stage estimates when full distributions are unavailable, the Second Moment Method (Chapter 3) provides a fast analytical alternative using only means and variances.

2.7. Exercises

1. **Conceptual check.** What does increasing the number of simulations from 10,000 to 100,000 do to the result? Does it change the mean? The P95? The smoothness of the histogram? Try it.
2. **New task.** Add a 4th task with a Beta distribution (hint: look at `?mcs` to see if Beta is supported, or use a normal approximation). Run `mcs()` with your updated task list and compare the P80 to the 3-task version. Did the project get riskier or safer?
3. **Correlation experiment.** Re-run the simulation with all off-diagonal correlation values set to 0 (fully independent tasks). Then re-run with all off-diagonal values set to 0.9 (strongly correlated). How does total variance change in each case? Explain intuitively why strong positive correlation increases portfolio risk.
4. **Choosing a confidence level.** Your client asks for a "90% confident" delivery date. What percentile do you use, and what contingency does that imply? Recalculate using `contingency(results, phigh = 0.90, pbase = 0.50)`.
5. **Real-world application.** Think of a project you know. Identify three tasks with significant uncertainty. Estimate (or guess) a distribution type and rough parameters for each. Run `mcs()` and report the P50 and P80 schedule. What's the contingency at P80?

3. May I Have a (Second) Moment? The Second Moment Method

> *"All models are wrong, but some are useful."* — George E. P. Box

The SMM is a deliberately simplified model. It assumes the total project duration is normally distributed. It ignores the shape of individual task distributions. And it is, in Box's sense, *useful*, fast enough to run before a meeting ends, and honest enough to use in a risk report.

Sometimes you don't need a full Monte Carlo simulation. Sometimes you need an answer in thirty seconds, not thirty minutes. You have means and variances, a correlation matrix, and a deadline. Enter the Second Moment Method, the analyst's equivalent of a napkin calculation that is actually defensible.

The name comes from statistics: the "first moment" of a distribution is its mean; the "second moment" (more precisely, the central second moment) is its variance, and the Second Moment Method is named for exactly that. It's the fastest way to get from "I have means and variances" to "I have a project risk estimate."

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain when the Second Moment Method is preferable to Monte Carlo simulation
2. Apply the SMM formulas for total mean and total variance by hand
3. Run `smm()` and interpret the output
4. Construct a 95% confidence interval for total project duration
5. Compare SMM and MCS results and understand where they diverge

3.1. When to Use SMM

SMM vs. Monte Carlo: The Decision Rule

Use **SMM** when:

- You need results in seconds, not minutes
- You only have mean and variance estimates (no full distribution)
- Tasks are approximately normal and correlations are well-characterized
- You want a quick sanity check before investing in a full MCS

Use **Monte Carlo** (Chapter 2) when:

- Tasks have non-normal distributions (triangular, uniform, lognormal)
- You need accurate tail behavior (P90, P95, P99)
- Skewness matters, as asymmetric distributions require simulation to capture correctly

Think of SMM as a reconnaissance tool. It tells you the territory before you commit to the full expedition.

3.2. How It Works

For a project with n tasks, SMM computes:

Total mean: Sum of individual task means:

$$E[X] = \sum_{i=1}^{n} E[X_i]$$

Total variance: Sum of variances plus twice the sum of all pairwise covariances:

$$\text{Var}(X) = \sum_{i=1}^{n} \text{Var}(X_i) + 2 \sum_{i<j} \text{Cov}(X_i, X_j)$$

Covariance: Derived from the correlation matrix:

$$\text{Cov}(X_i, X_j) = \rho_{ij} \cdot \sigma_i \cdot \sigma_j$$

By the Central Limit Theorem, the total is approximately normally distributed when there are many tasks, so we get confidence intervals for free (Benjamin and Cornell 2000).

3.3. Example

```
library(PRA)
set.seed(42)
```

We analyze a 3-task project with task durations in weeks. Each task has a known mean and variance, and correlations between tasks are provided.

```
task_means <- c(10, 15, 20)  # Expected duration for each task (weeks)
task_vars  <- c(4, 9, 16)    # Variance of each task duration
cor_mat <- matrix(c(
  1.0, 0.5, 0.3,
  0.5, 1.0, 0.4,
  0.3, 0.4, 1.0
), nrow = 3, byrow = TRUE)
```

```r
result <- smm(task_means, task_vars, cor_mat)
cat("Total Mean Duration:  ", round(result$total_mean, 2), "weeks\n")
```

```
Total Mean Duration:   45 weeks
```

```r
cat("Total Variance:       ", round(result$total_var, 2), "\n")
```

```
Total Variance:        49.4
```

```r
cat("Total Std Deviation:  ", round(result$total_std, 2), "weeks\n")
```

```
Total Std Deviation:   7.03 weeks
```

3.4. Implied Distribution and Confidence Interval

SMM assumes the total project duration is approximately normally distributed. This allows us to construct a confidence interval directly from the mean and standard deviation.

A 95% confidence interval for total project duration is approximately:

$$\bar{X} \pm 1.96 \cdot \sigma$$

```r
total_mean <- result$total_mean
total_sd   <- result$total_std
ci_lower   <- total_mean - 1.96 * total_sd
ci_upper   <- total_mean + 1.96 * total_sd
cat("95% CI: [", round(ci_lower, 1), ",", round(ci_upper, 1), "] weeks\n")
```

```
95% CI: [ 31.2 , 58.8 ] weeks
```

The plot below shows the implied normal distribution of total project duration, with the 95% confidence interval shaded:

```r
x_range <- seq(total_mean - 4 * total_sd, total_mean + 4 * total_sd, length.out = 300)
y_range <- dnorm(x_range, mean = total_mean, sd = total_sd)

plot(x_range, y_range,
  type = "l", lwd = 2, col = "steelblue",
  main = "SMM: Implied Project Duration Distribution",
  xlab = "Total Duration (weeks)", ylab = "Density"
)
```

```r
x_ci <- x_range[x_range >= ci_lower & x_range <= ci_upper]
y_ci <- dnorm(x_ci, mean = total_mean, sd = total_sd)
polygon(c(ci_lower, x_ci, ci_upper), c(0, y_ci, 0),
  col = "lightblue", border = NA
)

abline(v = total_mean, col = "black", lty = 2, lwd = 1.5)
legend("topright",
  legend = c("Normal density", "95% CI", "Mean"),
  col    = c("steelblue", "lightblue", "black"),
  lty    = c(1, NA, 2), lwd = c(2, NA, 1.5),
  pch    = c(NA, 15, NA), pt.cex = 1.5,
  bty    = "n"
)
```

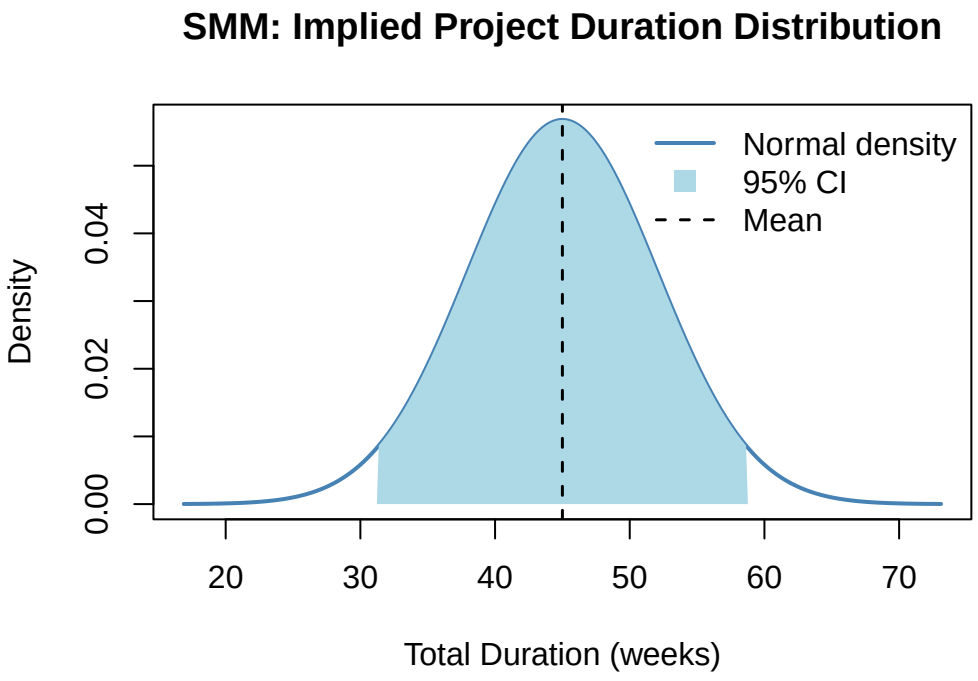


Figure 3.1.: SMM implied normal distribution for total project duration. The shaded region is the 95% confidence interval.

3.5. Comparison with Monte Carlo Simulation

> **What This Comparison Is Testing**
>
> To isolate the effect of **distributional assumptions** (normal vs. any shape), the MCS below uses the same task parameters as the SMM but with **no correlation matrix** (i.e., tasks are treated as independent). This allows a clean apples-to-apples test: both methods sum three independent normal tasks, so any remaining difference comes from simulation variance alone. The earlier SMM result (total mean = 45 weeks, total SD including correlations) is *not* the right benchmark here, as that figure includes covariance from the correlation matrix, which the MCS below omits. To compare correlated results, you would pass the same `cor_mat` to `mcs()`.

Running Monte Carlo simulation with the same task distributions validates the SMM. The two methods should yield very similar total means; differences in variance arise from how each handles correlated sampling.

```
task_dists_for_mcs <- list(
  list(type = "normal", mean = task_means[1], sd = sqrt(task_vars[1])),
  list(type = "normal", mean = task_means[2], sd = sqrt(task_vars[2])),
  list(type = "normal", mean = task_means[3], sd = sqrt(task_vars[3]))
)

mcs_result <- mcs(10000, task_dists_for_mcs)
```

```
smm_var_nocor <- sum(task_vars)

comparison <- data.frame(
  Method         = c("SMM (independent)", "Monte Carlo (10,000 runs)"),
  Total_Mean     = round(c(result$total_mean, mcs_result$total_mean), 2),
  Total_Variance = round(c(smm_var_nocor, mcs_result$total_variance), 2),
  Total_StdDev   = round(c(sqrt(smm_var_nocor), mcs_result$total_sd), 2)
)
knitr::kable(comparison, caption = "SMM vs. Monte Carlo Comparison (independent tasks)")
```

Table 3.1.: SMM vs. Monte Carlo Comparison (independent tasks)

Method	Total_Mean	Total_Variance	Total_StdDev
SMM (independent)	45.00	29.00	5.39
Monte Carlo (10,000 runs)	45.01	29.83	5.46

The two methods agree closely on the mean and variance. SMM is faster but assumes normality; Monte Carlo is more flexible and can use any distribution type.

3.6. Benefits and Limitations

	SMM	Monte Carlo
Speed	Instant (analytical)	Slow (thousands of iterations)
Inputs needed	Mean + variance per task	Full distribution per task
Distribution assumption	Normal (by CLT)	Any distribution
Correlation handling	Explicit covariance formula	Cholesky decomposition
Skewness / tails	Ignored	Captured accurately
Best for	Early estimates, quick checks	Detailed risk analysis, non-normal tasks

3.7. Summary

Key Takeaways

- The Second Moment Method propagates uncertainty analytically using only means, variances, and correlations, no simulation required.
- The **covariance formula** $\text{Cov}(X_i, X_j) = \rho_{ij} \cdot \sigma_i \cdot \sigma_j$ turns a correlation matrix into a contribution to total variance.
- By the Central Limit Theorem, the total is approximately normal, enabling confidence intervals via $\bar{X} \pm z \cdot \sigma$.
- SMM and Monte Carlo agree closely on the mean; differences in variance emerge from distributional assumptions and correlation handling.
- SMM is ideal for rapid early-stage estimates; use Monte Carlo (Chapter 2) when distribution shape and tail accuracy matter.

For projects where risks are interconnected through shared root causes, the Bayesian approach in Chapter 6 provides a richer updating framework beyond means and variances alone.

3.8. Exercises

1. **By hand.** Compute the project mean and total variance by hand for two tasks with means 5 and 10 weeks, variances 1 and 4, and a correlation of 0.3. Then verify your answer using `smm()`.

2. **Effect of correlation.** Run `smm()` for the 3-task example above with three different correlation matrices: (a) identity matrix (all tasks independent), (b) the original matrix (moderate correlation), and (c) a matrix where all off-diagonal entries are 0.9. Plot the three implied normal distributions on the same graph. What does correlation do to the spread?

3. **Normality check.** The SMM assumes normality via the Central Limit Theorem. This works best when there are many tasks. Run `mcs()` for the same 3-task project, then overlay the SMM normal distribution on the MCS histogram. How well does normality hold? What if two of the tasks followed exponential distributions instead of normal?

4. **SMM for costs.** Your project has four cost items with means \$50K, \$80K, \$30K, and \$60K and standard deviations \$10K, \$15K, \$5K, and \$12K. Assume moderate positive correlation (0.3) between all pairs. Use `smm()` to compute the P90 cost estimate (mean + 1.28 × SD).

5. **When to stop.** Under what conditions would you trust the SMM result over Monte Carlo? Under what conditions would you distrust it? Write a one-paragraph decision rule for choosing between the two methods.

4. Who's Driving? Sensitivity Analysis

"Risk comes from not knowing what you're doing." — Warren Buffett

You've run a Monte Carlo simulation and you have a P95 schedule. Now your project sponsor asks: "Which task should we focus on? Where should we spend our mitigation budget?" Looking at the total distribution doesn't answer that; it just tells you how uncertain the total is. Sensitivity analysis tells you *why*.

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain variance decomposition and how it differs from percentile analysis
2. Interpret a sensitivity index and identify the dominant risk driver in a project
3. Call `sensitivity()` with the three standard distribution types
4. Produce a tornado chart that ranks tasks by their contribution to total variance
5. Extend the analysis to correlated tasks using a correlation matrix

4.1. What Is Sensitivity Analysis?

Sensitivity analysis answers the question: "If I could reduce uncertainty in one task, which task would have the greatest effect on total project variance?"

The answer is not always the task with the longest expected duration. A short task with extremely high variance, say, a procurement step that might take anywhere from 1 to 20 weeks, can dominate the project's total uncertainty even though its mean duration is modest.

4.1.1. Variance Decomposition

For a project with n independent tasks, total variance is simply the sum of individual variances:

$$\sigma^2_{\text{total}} = \sum_{i=1}^{n} \sigma_i^2$$

When tasks are correlated, covariance terms appear:

$$\sigma^2_{\text{total}} = \sum_{i=1}^{n} \sigma_i^2 + 2 \sum_{i<j} \text{cov}(i, j)$$

The **sensitivity index** for task i quantifies how much its variance, including its covariance with all other tasks, contributes to the total:

$$s_i = 1 + 2 \sum_{j \neq i} \frac{\text{cov}_{ij}}{\sqrt{\sigma_i^2 \cdot \sigma_j^2}}$$

For independent tasks, all covariance terms are zero and $s_i = 1$ for every task, meaning the tasks contribute proportionally to their variance. Positive correlations push dominant tasks' indices above 1 and amplify them relative to smaller-variance tasks.

> **Sensitivity vs. Percentiles**
>
> Monte Carlo gives you *how bad* the total outcome could be (P95 = 38 weeks). Sensitivity analysis gives you *which task is responsible* (Task B drives 60% of total variance). Use both: percentiles for the headline number, sensitivity for the "where do we focus?" conversation.

4.2. Setup

```
library(PRA)
set.seed(42)
```

We use the same three tasks from Chapter 2, a three-task project with mixed distribution types, so you can see the complementary view.

```
task_dists <- list(
  TaskA = list(type = "normal",     mean = 10, sd = 4),
  TaskB = list(type = "triangular", a = 5, b = 10, c = 15),
  TaskC = list(type = "uniform",    min = 9.5, max = 10.5)
)
```

4.3. Computing Sensitivity

```
sens <- sensitivity(task_dists)
sens
```

```
[1] 0.790123457 0.205761317 0.004115226
```

The output is a named vector with one sensitivity index per task. A higher value means that task's variance has a larger proportional effect on total project variance.

4.3.1. Interpretation

```
round(sens / sum(sens) * 100, 1)
```

```
[1] 79.0 20.6  0.4
```

Expressing the indices as percentages makes the picture clear: Task A (normal, sd = 4) dominates total project variance, accounting for the majority of total uncertainty. Task B is a secondary contributor, while Task C barely registers. Task A is the task where mitigation effort pays off most.

4.4. Tornado Chart

A horizontal bar chart sorted by sensitivity index is the standard way to communicate this finding; it's called a tornado chart because the bars narrow from top to bottom.

```
sorted_sens <- sort(sens, decreasing = FALSE)
barplot(
  sorted_sens,
  horiz  = TRUE,
  names.arg = names(sorted_sens),
  xlab   = "Sensitivity Index",
  main   = "Task Sensitivity",
  col    = "#18bc9c",
  border = "white",
  las    = 1
)
abline(v = 1, lty = 2, col = "#e74c3c")
```

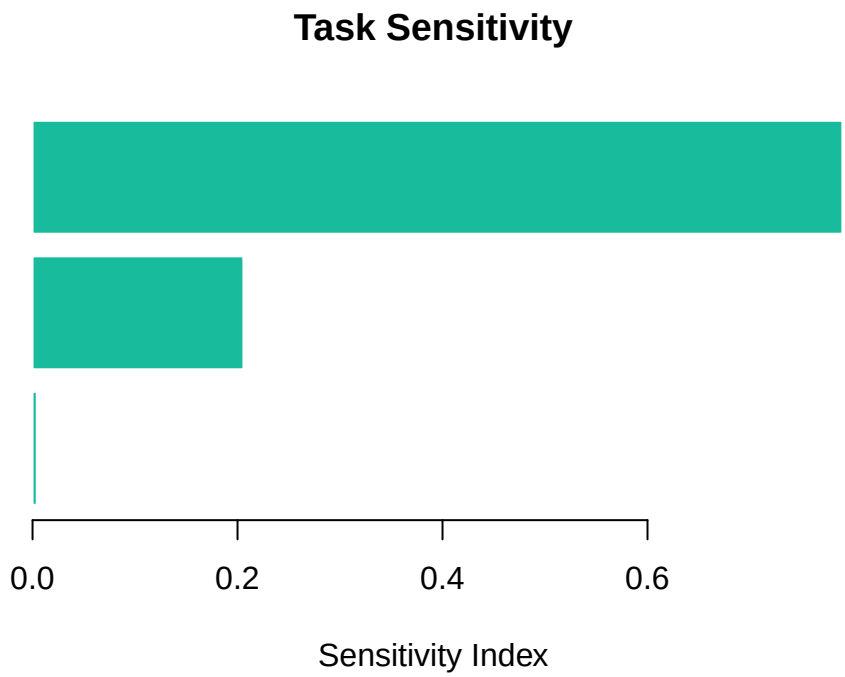


Figure 4.1.: Tornado chart of task sensitivity indices. The tallest bar is the dominant risk driver, the task where reducing variance has the greatest effect on total project uncertainty.

The dashed red line at $s = 1$ marks the neutral baseline. Tasks above 1 are amplified by positive correlation with other tasks; tasks below 1 are damped by negative correlation (or simply have lower variance).

4.5. With Correlated Tasks

Real projects have correlated tasks: they share resources, face the same weather, or depend on the same supplier. Positive correlation amplifies the dominant driver; it becomes even more important to get that task right.

Use `cor_matrix()` to build a correlation matrix from sampled distributions:

```
cm <- cor_matrix(
  num_samples = 10000,
  num_vars    = 3,
  dists = list(
    TaskA = function(n) rnorm(n, 10, 4),
    TaskB = function(n) {
      mc2d::rtriang(n, min = 5, mode = 10, max = 15)
    },
    TaskC = function(n) runif(n, 9.5, 10.5)
```

```
  )
)
cm
```

```
             TaskA          TaskB          TaskC
TaskA  1.000000000  0.0111717475 -0.0083913814
TaskB  0.011171748  1.0000000000 -0.0005045808
TaskC -0.008391381 -0.0005045808  1.0000000000
```

```
sens_corr <- sensitivity(task_dists, cor_mat = cm)

par(mfrow = c(1, 2))

sorted_base <- sort(sens, decreasing = FALSE)
barplot(sorted_base, horiz = TRUE, names.arg = names(sorted_base),
  xlab = "Sensitivity Index", main = "Independent",
  col = "#18bc9c", border = "white", las = 1)
abline(v = 1, lty = 2, col = "#e74c3c")

sorted_corr <- sort(sens_corr, decreasing = FALSE)
barplot(sorted_corr, horiz = TRUE, names.arg = names(sorted_corr),
  xlab = "Sensitivity Index", main = "Correlated",
  col = "#3498db", border = "white", las = 1)
abline(v = 1, lty = 2, col = "#e74c3c")
```

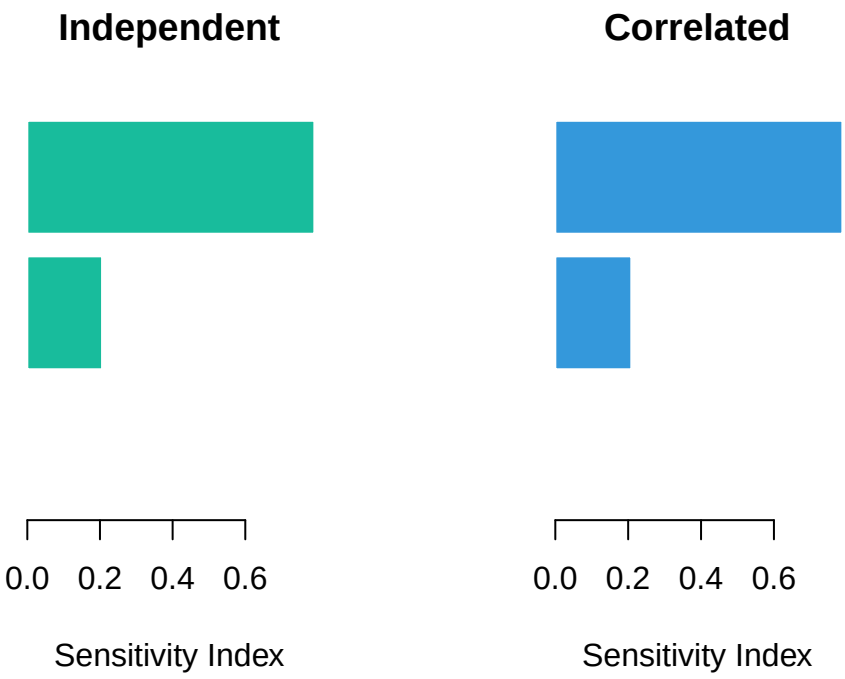


Figure 4.2.: Sensitivity indices when tasks are correlated. Positive correlations amplify the dominant driver (Task A) and increase total project variance.

```
par(mfrow = c(1, 1))
```

Comparing the two panels: when tasks are positively correlated, the dominant task's index rises. Ignoring correlations understates the risk concentration.

4.6. Summary

Key Takeaways

- **Sensitivity analysis** decomposes total project variance into per-task contributions. The result is an index, not a probability; it shows relative importance, not absolute risk.
- `sensitivity(task_dists)` takes the same distribution specs as `mcs()` and returns a named vector of sensitivity indices.
- The task with the highest sensitivity index is the one where reducing variance has the greatest effect on the total project distribution. That is where to spend mitigation budget.
- **Positive correlations amplify the dominant driver.** A task that leads in the independent case leads by an even larger margin when it is positively correlated with its neighbours.
- A tornado chart (sorted horizontal bar plot) is the standard deliverable for communicating

sensitivity results to stakeholders.

For a fast analytical estimate of total mean and variance, without the index-level decomposition, see Chapter 3. For the simulation that produced the total distribution these indices decompose, see Chapter 2.

4.7. Exercises

1. **Reading the chart.** In the tornado chart above, which task contributes the least to total variance? What property of its distribution explains this?
2. **Change the spread.** Replace Task B's triangular distribution with `Triangular(8, 10, 12)`, a tighter spread. How do the sensitivity indices change? Does Task B still dominate?
3. **High-variance newcomer.** Add a fourth task, Task D ~ `Normal(5, 6)` (very high variance relative to its mean). Recompute the sensitivity indices. Does Task D become the dominant driver?
4. **Correlation direction.** Construct a correlation matrix where Task A and Task B are negatively correlated (-0.5). What happens to their sensitivity indices relative to the independent case? Explain the result in terms of the covariance formula.
5. **From sensitivity to contingency.** Run `mcs()` on the same three tasks, then run `sensitivity()`. The task with the highest sensitivity index should also be responsible for the widest spread in the simulation. Verify this by plotting the individual task distributions from the MCS output alongside the sensitivity indices. Do they tell a consistent story?

5. Keeping Score: Earned Value Management

> *"When you can measure what you are speaking about, and express it in numbers, you know something about it."* — Lord Kelvin

Imagine you're three months into a six-month project and you've spent exactly half the budget. That might sound encouraging, but the more useful question is: how much work did you actually get done? If you've completed 30% of the project, the budget situation looks very different than if you've completed 55%.

This is the fundamental insight behind **Earned Value Management** (EVM): you can't judge project health by money spent alone. You need to compare three things: what you planned, what you actually did, and what it cost you to do it.

Learning Objectives

By the end of this chapter, you will be able to:

1. Compute PV, EV, and AC from project data
2. Calculate and interpret CPI and SPI performance indices
3. Produce an EAC forecast using all three methods and choose the right one
4. Calculate ETC, VAC, and TCPI
5. Read and construct an EVM trend chart

5.1. The Three Core Numbers

PV, EV, AC: The Foundation of Everything

Quantity	What it measures
PV: Planned Value	The budget authorized for work *scheduled* through today
EV: Earned Value	The budget value of work *actually completed*
AC: Actual Cost	Total cost *incurred* to date

Every EVM metric, variance, index, or forecast, derives from these three numbers. If you know PV, EV, and AC, you know everything EVM can tell you.

EVM integrates scope, schedule, and cost into these three core quantities (Fleming and Koppelman 2010). From them, all the other EVM metrics flow, with every performance index, every forecast, and every variance coming back to PV, EV, and AC.

5.2. Key Metrics

Metric	Formula	Interpretation
CV: Cost Variance	EV − AC	> 0: under budget; < 0: over budget
SV: Schedule Variance	EV − PV	> 0: ahead of schedule; < 0: behind schedule
CPI: Cost Performance Index	EV / AC	> 1: efficient; = 1: on target; < 1: over-spending
SPI: Schedule Performance Index	EV / PV	> 1: ahead; = 1: on target; < 1: behind
EAC: Estimate at Completion	See below	Forecast of total project cost
ETC: Estimate to Complete	EAC − AC	Remaining cost to finish the project
VAC: Variance at Completion	BAC − EAC	Expected surplus (positive) or overrun (negative)
TCPI: To-Complete Performance Index	(BAC−EV) / (BAC−AC)	Efficiency required on remaining work

5.3. Example Setup

```
# Render large currency values in full, not scientific notation.
options(scipen = 999)
library(PRA)
```

We will track a project with a total budget of $500,000 over a 5-period schedule.

```
bac      <- 500000
schedule <- c(0.10, 0.25, 0.50, 0.75, 1.00)
time_period <- 3
```

5.3.1. Planned Value (PV)

PV is the authorized budget for work planned through the current period.

```
pv_val <- pv(bac, schedule, time_period)
cat("Planned Value (PV): $", format(pv_val, big.mark = ","), "\n")
```

```
Planned Value (PV): $ 250,000
```

The project was planned to be 50% complete by period 3, so PV = $250,000.

5.3.2. Earned Value (EV)

EV reflects the budget value of work actually completed.

```
actual_per_complete <- 0.40
ev_val <- ev(bac, actual_per_complete)
cat("Earned Value (EV): $", format(ev_val, big.mark = ","), "\n")
```

```
Earned Value (EV): $ 200,000
```

Only 40% of work is done despite 50% being planned, so we are behind schedule.

5.3.3. Actual Cost (AC)

AC is the cumulative cost incurred through period 3.

```
period_costs <- c(45000, 110000, 135000)
ac_val <- ac(period_costs, time_period, cumulative = FALSE)
cat("Actual Cost (AC): $", format(ac_val, big.mark = ","), "\n")
```

```
Actual Cost (AC): $ 290,000
```

5.3.4. Performance Indicators

```
sv_val  <- sv(ev_val, pv_val)
cv_val  <- cv(ev_val, ac_val)
spi_val <- spi(ev_val, pv_val)
cpi_val <- cpi(ev_val, ac_val)

cat("Schedule Variance (SV):           $", format(sv_val, big.mark = ","), "\n")
```

```
Schedule Variance (SV):           $ -50,000
```

```
cat("Cost Variance (CV):               $", format(cv_val, big.mark = ","), "\n")
```

```
Cost Variance (CV):               $ -90,000
```

```
cat("Schedule Performance Index (SPI):", round(spi_val, 3), "\n")
```

```
Schedule Performance Index (SPI): 0.8
```

```
cat("Cost Performance Index (CPI):    ", round(cpi_val, 3), "\n")
```

```
Cost Performance Index (CPI):     0.69
```

Interpretation: SPI < 1 means we are behind schedule, earning only 80 cents of planned value per dollar of schedule. CPI < 1 means we are over budget, earning only 69 cents of value per dollar spent. Both are below 1.0: this project is in trouble.

5.4. Forecasting: Estimate at Completion (EAC)

EAC forecasts the total cost at project completion. Three methods are available, each making a different assumption about future performance:

Method	Formula	When to use
Typical	BAC / CPI	Current cost inefficiency is expected to continue
Atypical	AC + (BAC − EV)	Cost overrun was a one-time event; future work at planned rate
Combined	AC + (BAC − EV) / (CPI × SPI)	Both cost and schedule performance will influence future costs

```
eac_typical  <- eac(bac, method = "typical", cpi = cpi_val)
eac_atypical <- eac(bac, method = "atypical", ac = ac_val, ev = ev_val)
eac_combined <- eac(bac,
  method = "combined", cpi = cpi_val, ac = ac_val,
  ev = ev_val, spi = spi_val
)

cat("EAC (typical):  $", format(round(eac_typical),  big.mark = ","), "\n")
```

```
EAC (typical):  $ 725,000
```

```
cat("EAC (atypical): $", format(round(eac_atypical), big.mark = ","), "\n")
```

```
EAC (atypical): $ 590,000
```

```
cat("EAC (combined): $", format(round(eac_combined), big.mark = ","), "\n")
```

```
EAC (combined): $ 833,750
```

The typical method gives the most conservative (highest cost) estimate because it assumes the current CPI persists. The atypical method is the most optimistic.

5.4.1. EAC Comparison Table

```
eac_table <- data.frame(
  Method     = c("Typical", "Atypical", "Combined"),
  EAC        = c(round(eac_typical), round(eac_atypical), round(eac_combined)),
  Overrun    = c(
    round(eac_typical  - bac),
    round(eac_atypical - bac),
    round(eac_combined - bac)
  ),
  Assumption = c(
    "Current CPI continues",
    "Future work at planned rate",
    "CPI and SPI both factor in"
  )
)
knitr::kable(eac_table,
  format.args = list(big.mark = ","),
  caption = "EAC Comparison by Method"
)
```

Table 5.1.: EAC Comparison by Method

Method	EAC	Overrun	Assumption
Typical	725,000	225,000	Current CPI continues
Atypical	590,000	90,000	Future work at planned rate
Combined	833,750	333,750	CPI and SPI both factor in

5.5. Additional Metrics

```
etc_val  <- etc(bac, ev_val, cpi_val)
vac_val  <- vac(bac, eac_typical)
tcpi_bac <- tcpi(bac, ev_val, ac_val, target = "bac")
tcpi_eac <- tcpi(bac, ev_val, ac_val, target = "eac", eac = eac_typical)

cat("Estimate to Complete (ETC):  $", format(round(etc_val), big.mark = ","), "\n")
```

```
Estimate to Complete (ETC):  $ 435,000
```

```
cat("Variance at Completion (VAC): $", format(round(vac_val), big.mark = ","), "\n")
```

```
Variance at Completion (VAC): $ -225,000
```

```
cat("TCPI (to meet BAC):          ", round(tcpi_bac, 3), "\n")
```

```
TCPI (to meet BAC):           1.429
```

```
cat("TCPI (to meet EAC):          ", round(tcpi_eac, 3), "\n")
```

```
TCPI (to meet EAC):           0.69
```

Interpretation: TCPI > 1 means the team must work more efficiently than they have been to meet the target. A TCPI above 1.2 is a widely-cited practitioner rule of thumb for "unrealistic" (not a formal PMBOK standard), a useful warning signal. If you need to be 30% more efficient for the rest of the project, it probably isn't going to happen. Using the EAC target gives a more realistic benchmark.

5.6. Performance Trend Chart

The chart below shows cumulative PV, AC, and EV over time, with reference lines for BAC and EAC.

```
time_periods <- c(1, 2, 3)
actual_pct   <- c(0.08, 0.22, 0.40)
p_costs      <- c(45000, 110000, 135000)

pv_vals <- sapply(time_periods, function(t) pv(bac, schedule, t))
ac_vals <- cumsum(p_costs)
ev_vals <- sapply(actual_pct, function(a) ev(bac, a))

trend_data <- data.frame(
  Period = time_periods,
  PV = pv_vals, AC = ac_vals, EV = ev_vals
)

p <- ggplot2::ggplot(trend_data, ggplot2::aes(x = Period)) +
  ggplot2::geom_line(ggplot2::aes(y = PV, color = "Planned Value (PV)"), linewidth = 1.2) +
  ggplot2::geom_line(ggplot2::aes(y = AC, color = "Actual Cost (AC)"), linewidth = 1.2) +
  ggplot2::geom_line(ggplot2::aes(y = EV, color = "Earned Value (EV)"), linewidth = 1.2) +
  ggplot2::geom_point(ggplot2::aes(y = PV, color = "Planned Value (PV)"), size = 3) +
  ggplot2::geom_point(ggplot2::aes(y = AC, color = "Actual Cost (AC)"), size = 3) +
  ggplot2::geom_point(ggplot2::aes(y = EV, color = "Earned Value (EV)"), size = 3) +
  ggplot2::geom_hline(yintercept = bac, linetype = "dashed", color = "black", linewidth = 0.8)
  ggplot2::geom_hline(yintercept = eac_typical, linetype = "dotted", color = "darkred", linewi
  ggplot2::annotate("text", x = 2.8, y = bac + 12000, label = "BAC = $500K", size = 3.5) +
  ggplot2::annotate("text", x = 2.8, y = eac_typical + 12000, label = "EAC (typical)", size =
  ggplot2::scale_color_manual(values = c(
```

```
    "Planned Value (PV)" = "steelblue",
    "Actual Cost (AC)"   = "tomato",
    "Earned Value (EV)"  = "forestgreen"
  )) +
  ggplot2::scale_y_continuous(labels = scales::label_dollar(scale = 1e-3, suffix = "K")) +
  ggplot2::labs(
    title = "Earned Value Management: Performance Trend",
    x     = "Time Period", y = "Value", color = NULL
  ) +
  ggplot2::theme_minimal() +
  ggplot2::theme(legend.position = "bottom")

print(p)
```

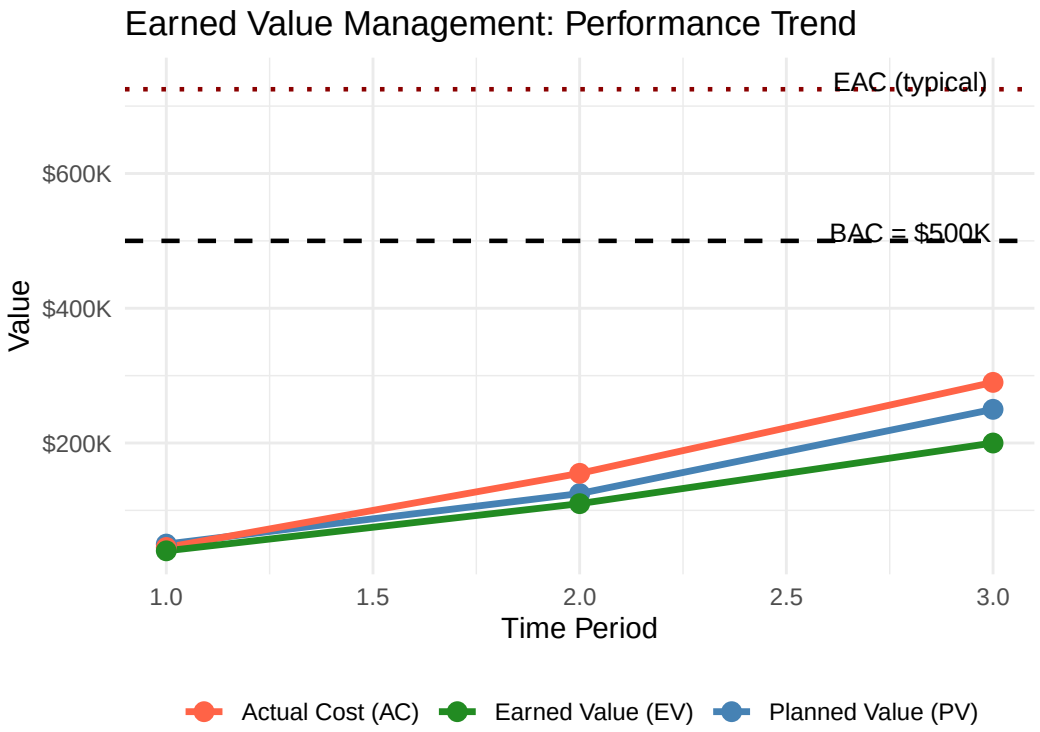


Figure 5.1.: EVM trend chart. When EV falls below PV, you're behind schedule. When AC rises above EV, you're over budget. Both conditions here, a project in the red on both dimensions.

Reading the chart: The gap between PV and EV shows the schedule gap, with EV below PV. The gap between AC and EV shows the cost overrun, as we've spent more than the value we've earned.

5.7. Summary

Key Takeaways

- EVM combines scope, schedule, and cost into three numbers (PV, EV, AC) from which all performance metrics flow.
- **CPI < 1** means over-budget; **SPI < 1** means behind schedule. Both below 1 signals a project in serious trouble.
- The **Typical EAC** (BAC / CPI) is the most conservative forecast; use it when current inefficiency is expected to persist.
- **TCPI > 1.2** is a common practitioner rule of thumb for "unrealistic" (not a formal standard), flag it as a signal to revise the budget rather than chase an unachievable efficiency target.
- The trend chart (PV, EV, AC over time) is your single most powerful communication tool for project status.

EVM tells you *where the project stands right now*. For forward-looking updates that incorporate new information as the project evolves, Bayesian methods in Chapter 6 provide a complementary approach to cost forecasting.

5.8. Exercises

1. **Diagnosis.** If SPI < 1 and CPI > 1, what does that tell you about the project? Describe the scenario in plain English and give an example of how this might happen on a real project.
2. **Forecast comparison.** Compute EAC using all three methods for a new project with BAC = \$200K, PV = \$80K, EV = \$60K, AC = \$70K. Which method produces the highest estimate? The lowest? Which would you recommend reporting to the client?
3. **TCPI reality check.** For the same project (BAC = \$200K, PV = \$80K, EV = \$60K, AC = \$70K), calculate TCPI to meet BAC. Is it realistic? At what TCPI would you advise the client to revise the budget rather than try to recover?
4. **The optimistic mistake.** A project manager always uses the atypical EAC method, regardless of what's happening. In what circumstances would this be dangerously wrong? Write a 2–3 sentence warning label for the atypical method.
5. **Build your own trend.** Create a 5-period EVM dataset where the project starts over budget but recovers over time (CPI < 1 in periods 1–2, CPI 1 in periods 3–5). Plot the trend chart. Does the "typical" EAC improve as the project recovers?

6. I Had a Feeling: Bayesian Risk Inference

> *"When the facts change, I change my mind. What do you do, sir?"* — Attributed to John Maynard Keynes

Here is the situation: you have a risk assessment based on historical data and expert judgment. Then something happens on site: a root cause you were watching turns out to be present, or a supplier sends a warning signal. Your prior assessment is now out of date. Do you stick with the original numbers, or update them?

Bayesian inference gives you the formal machinery to update. Not because it feels right, but because it is the mathematically correct way to incorporate new evidence.

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain prior and posterior probability in plain English
2. Use `risk_prob()` to compute a prior risk probability from root causes
3. Use `risk_post_prob()` to update that probability given new observations
4. Sample prior and posterior cost distributions with `cost_pdf()` and `cost_post_pdf()`
5. Interpret the shift between prior and posterior distributions

6.1. The Core Idea

Bayes' Theorem in Plain English

$$P(H \mid E) = \frac{P(E \mid H) \cdot P(H)}{P(E)}$$

In plain English: Your updated belief (posterior) equals your initial belief (prior) multiplied by how well the evidence fits that belief, normalized to sum to 1.
In project risk terms:

- **Prior** $P(H)$: Your initial estimate of risk probability, before any field observations
- **Likelihood** $P(E \mid H)$: How likely is the observed evidence if the risk/cause is present?
- **Posterior** $P(H \mid E)$: Your updated estimate after incorporating the evidence

Observing a root cause *always* moves the posterior. The direction and magnitude depend on how strongly that cause is linked to the risk event.

The PRA package provides four Bayesian functions organized into two stages:

Stage	Function	Purpose
Prior	`risk_prob()`	Compute risk probability from root causes (no observations yet)
Prior	`cost_pdf()`	Sample prior cost distribution based on risk probabilities
Posterior	`risk_post_prob()`	Update risk probability after observing cause status
Posterior	`cost_post_pdf()`	Sample posterior cost distribution based on observed risks

```
# Render large currency values in full, not scientific notation.
options(scipen = 999)
library(PRA)
set.seed(42)
```

6.2. Causal Structure

Before computing probabilities, it helps to see the causal graph. Two independent root causes each point to the risk event, and observing either one updates the posterior risk probability.

```
library(igraph)
g <- graph_from_data_frame(
  data.frame(from = c("Cause-1", "Cause-2"), to = c("Risk", "Risk")),
  directed = TRUE
)
V(g)$color <- c("steelblue", "steelblue", "tomato")
plot(g, vertex.size = 40, vertex.label.cex = 0.9,
     edge.arrow.size = 0.6, layout = layout_with_sugiyama(g)$layout)
```

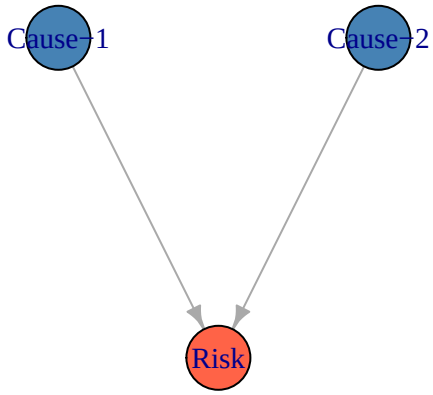


Figure 6.1.: DAG: Cause-1 and Cause-2 independently drive the risk event R.

6.3. Step 1: Prior Risk Probability

Before any observations are made, `risk_prob()` computes the probability of a risk event R occurring given two potential root causes. For each cause, we supply:

- `cause_probs`: prior probability that the cause is present
- `risks_given_causes`: P(R | cause present)
- `risks_given_not_causes`: P(R | cause absent)

```
cause_probs            <- c(0.3, 0.2)
risks_given_causes     <- c(0.8, 0.6)
risks_given_not_causes <- c(0.2, 0.4)
```

```
prior_risk <- risk_prob(cause_probs, risks_given_causes, risks_given_not_causes)
cat("Prior probability of risk event R:", round(prior_risk, 3), "\n")
```

```
Prior probability of risk event R: 0.653
```

This is your starting point, an estimate of how likely the risk event is before you've looked at anything on site.

6.4. Step 2: Prior Cost Distribution

Given the prior risk probabilities, `cost_pdf()` samples the cost distribution before any field observations. Three independent risk events can each contribute cost if they occur.

```
risk_probs        <- c(0.3, 0.5, 0.2)
means_given_risks <- c(10000, 15000, 5000)
sds_given_risks   <- c(2000, 1000, 1000)
base_cost         <- 2000
```

```
prior_samples <- cost_pdf(
  num_sims          = 5000,
  risk_probs        = risk_probs,
  means_given_risks = means_given_risks,
  sds_given_risks   = sds_given_risks,
  base_cost         = base_cost
)
```

We'll compare this prior distribution to the posterior in Step 4.

6.5. Step 3: Posterior Risk Probability (Bayesian Update)

After inspecting the project site, we observe that Cause 1 is present (= 1). Cause 2 has not yet been assessed (= NA). `risk_post_prob()` updates the risk probability using only the available evidence; NA causes are treated as unobserved and do not contribute to the update.

```
observed_causes <- c(1, NA)  # C1 observed as present; C2 not yet assessed
```

```
posterior_risk <- risk_post_prob(
  cause_probs, risks_given_causes,
  risks_given_not_causes, observed_causes
)
cat("Posterior probability of risk event R:", round(posterior_risk, 3), "\n")
```

```
Posterior probability of risk event R: 0.888
```

Observing Cause 1 (which has a strong link to R) raises the risk probability substantially. The NA for Cause 2 is simply ignored; only confirmed observations drive the update.

6.5.1. Prior vs. Posterior Probability

```r
prob_data <- data.frame(
  Stage       = c("Prior", "Posterior"),
  Probability = c(prior_risk, posterior_risk)
)

p <- ggplot2::ggplot(prob_data, ggplot2::aes(x = Stage, y = Probability, fill = Stage)) +
  ggplot2::geom_col(width = 0.5, show.legend = FALSE) +
  ggplot2::geom_text(ggplot2::aes(label = round(Probability, 3)),
    vjust = -0.4, size = 4.5
  ) +
  ggplot2::scale_fill_manual(values = c("Prior" = "steelblue", "Posterior" = "tomato")) +
  ggplot2::scale_y_continuous(limits = c(0, 1), labels = scales::percent) +
  ggplot2::labs(
    title = "Bayesian Update: Risk Probability",
    x     = NULL, y = "P(Risk Event R)"
  ) +
  ggplot2::theme_minimal(base_size = 13)

print(p)
```

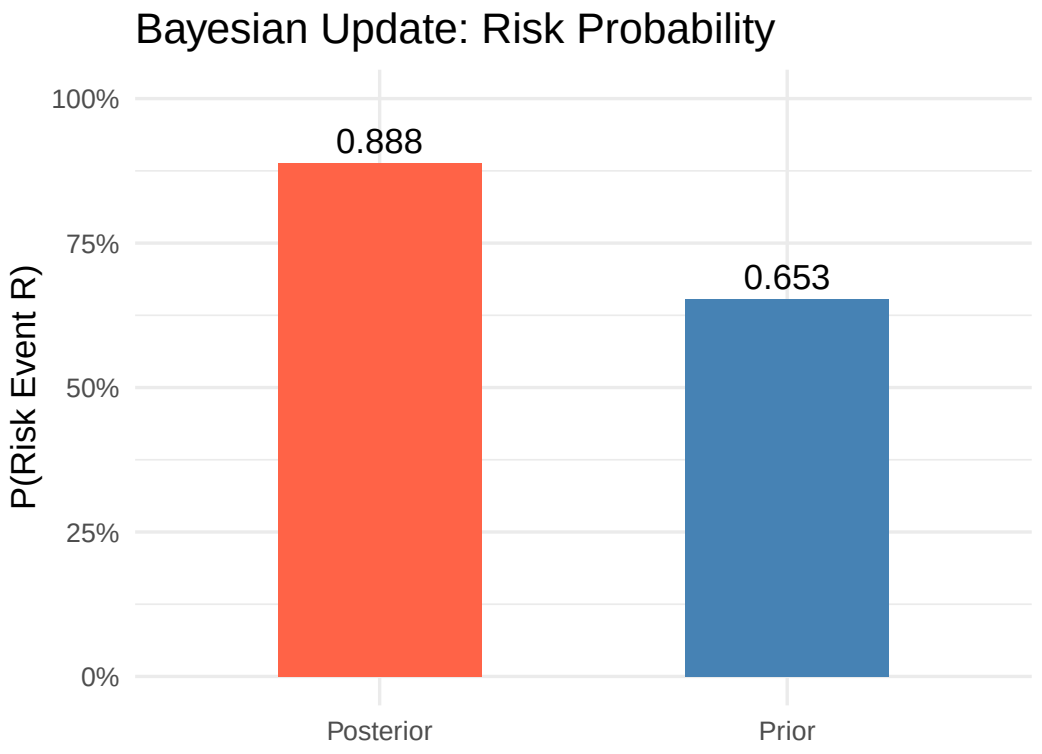


Figure 6.2.: Bayesian update: observing Cause 1 nearly doubles the estimated probability of the risk event.

6.6. Step 4: Posterior Cost Distribution

Now that we know Cause 1 is present (Risk 1 occurs), and one risk remains unobserved (Risk 2 = NA), `cost_post_pdf()` samples the posterior cost distribution. Observed risks that occurred (= 1) add their cost; the unobserved risk (Risk 2 = NA) is drawn from its prior probability via `risk_probs` — the same prior used in Step 2 — rather than treated as absent, so unresolved uncertainty still contributes its expected share of cost.

```
observed_risks <- c(1, NA, 1)  # Risk 1 and 3 confirmed; Risk 2 not yet assessed
```

```
posterior_samples <- cost_post_pdf(
  num_sims          = 5000,
  observed_risks    = observed_risks,
  means_given_risks = means_given_risks,
  sds_given_risks   = sds_given_risks,
  base_cost         = base_cost,
  risk_probs        = risk_probs
)
```

6.6.1. Prior vs. Posterior Cost Distribution

Plotting both distributions on the same axes shows how the evidence shifts the cost estimate:

```
xlim_range <- range(c(prior_samples, posterior_samples))

hist(prior_samples,
  breaks = 40, freq = FALSE,
  col    = rgb(0.27, 0.51, 0.71, 0.5),
  border = "white",
  xlim   = xlim_range,
  main   = "Prior vs. Posterior Cost Distribution",
  xlab   = "Total Cost ($)", ylab = "Density"
)

hist(posterior_samples,
  breaks = 40, freq = FALSE,
  col    = rgb(0.84, 0.24, 0.31, 0.5),
  border = "white",
  add    = TRUE
)

abline(v = mean(prior_samples),     col = "steelblue", lty = 2, lwd = 2)
abline(v = mean(posterior_samples), col = "tomato",    lty = 2, lwd = 2)

legend("topright",
  legend = c(
```

```
    paste0("Prior  (mean = $", format(round(mean(prior_samples)),     big.mark = ",")),
    paste0("Posterior (mean = $", format(round(mean(posterior_samples)), big.mark = ","))
  ),
  fill = c(rgb(0.27, 0.51, 0.71, 0.5), rgb(0.84, 0.24, 0.31, 0.5)),
  bty  = "n"
)
```

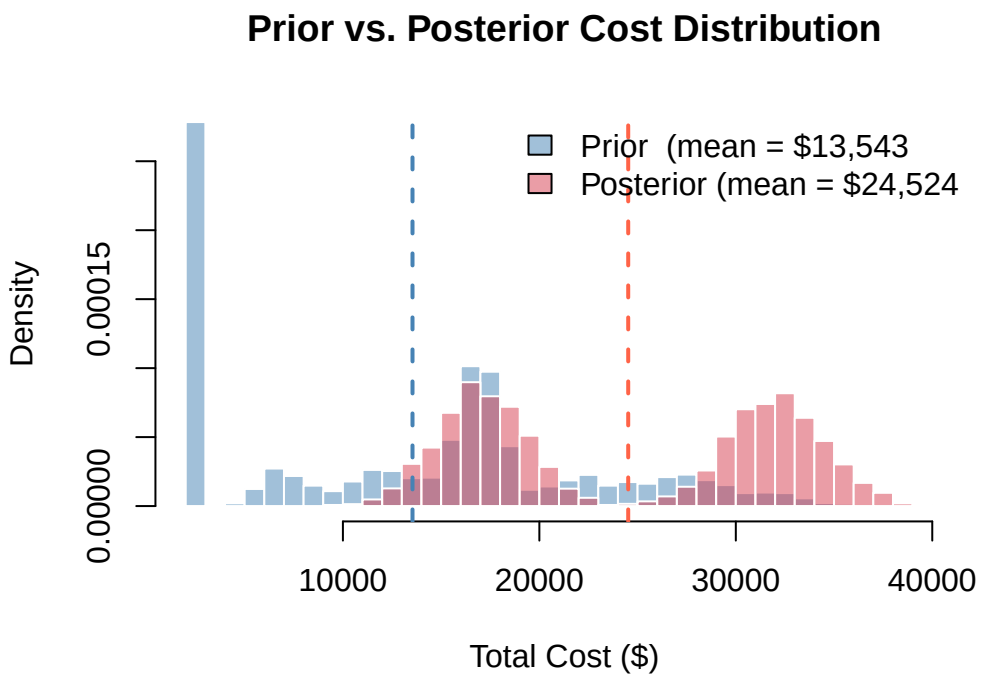


Figure 6.3.: Prior vs. posterior cost distribution. Confirming which risks materialized narrows and shifts the distribution.

Interpretation: The posterior distribution is narrower and shifted upward, as observing which risks materialized eliminates uncertainty about some cost components while raising the estimated exposure from confirmed events. The remaining spread reflects uncertainty from the unobserved risk and cost variability in the confirmed ones.

6.7. The Workflow

The Bayesian workflow in PRA follows a natural before-and-after structure:

1. **Before observations**: use `risk_prob()` and `cost_pdf()` to characterize the risk landscape.
2. **As evidence arrives**: use `risk_post_prob()` and `cost_post_pdf()` to update estimates.

3. **NA values** represent causes/risks not yet assessed. `risk_post_prob()` always excludes them from the update. `cost_post_pdf()` instead draws them from their prior via `risk_probs`, so unresolved risks still contribute their expected cost.

This approach is particularly powerful in phased projects where information about risk drivers becomes available progressively, allowing cost forecasts to be refined at each stage.

6.8. Summary

Key Takeaways

- Bayesian inference is the principled way to update risk estimates as new evidence arrives; it is not a heuristic, it is the mathematically correct rule.
- **Prior → Posterior**: the posterior always moves in the direction of the evidence; the size of the move depends on how strongly the cause links to the risk.
- `NA` for an unobserved cause is correctly excluded from `risk_post_prob()`, reflecting genuine ignorance; `NA` for an unobserved risk is instead drawn from its prior in `cost_post_pdf()` (via `risk_probs`), since dropping it would understate cost.
- The posterior cost distribution is typically narrower than the prior, because confirmed risk states eliminate uncertainty about some cost components.
- This workflow is particularly powerful in **phased projects**: update at each phase gate as more information becomes available.

For risks that propagate through a network of shared resources and tasks, Chapter 8 extends these ideas into a full probabilistic graph where Bayesian conditioning and structural dependencies interact.

6.9. Exercises

1. **Prior vs. posterior.** In plain English, what is the difference between a prior probability and a posterior probability? Give a project example where each would be used.
2. **Change the observation.** Re-run Step 3 with Cause 2 observed as present (= 1) instead of Cause 1. How does the posterior change compared to when Cause 1 was observed? Why?
3. **Both causes observed.** What happens if you observe both causes as present (= 1, = 1)? Compute the posterior. What is the maximum possible posterior probability given the model parameters?
4. **Intuition check.** Explain in plain English why observing one root cause changes your belief about the risk event, even though we didn't observe the risk event itself. What is the role of the conditional probabilities `risks_given_causes` in this reasoning?
5. **Sequential updating.** Start with the prior. Observe Cause 1. Update. Then observe Cause 2. Update again. Does it matter what order you observe the causes? Should it? (Hint: think about whether the causes are assumed independent in this model.)

7. S Is for Success: Sigmoidal Learning Curves

> *"For the things we have to learn before we can do them, we learn by doing them."* — Aristotle

Every team that's ever built something knows the pattern. Week one: figuring things out. Week four: hitting a stride. Week eight: slowing down as completion approaches 100%. This S-shaped curve, with a slow start, rapid middle, and graceful plateau, is not a quirk of one project. It's a near-universal pattern in learning and production processes, and it has a name: the **sigmoidal curve**.

If you can fit a sigmoidal model to your early progress data, you can forecast when you'll finish, and how confident you should be in that forecast.

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain why sigmoidal models are more appropriate than linear or exponential models for project completion data
2. Fit Logistic, Pearl, and Gompertz models using `fit_sigmoidal()`
3. Compare model fit using residual standard error
4. Produce and interpret a forecast with confidence bands using `plot_sigmoidal()` and `predict_sigmoidal()`
5. Choose the right model type based on data shape and theoretical expectations

7.1. Why Sigmoidal?

Linear models ("we'll complete 10% per week forever") break down at the extremes: they predict negative completion before the project starts and more than 100% after it finishes. Exponential models grow forever. Sigmoidal models are bounded: they start slow, accelerate, and plateau at a ceiling value, making them physically meaningful for completion percentages.

Learning curves are used to:

- **Forecast completion**: Predict when a task or deliverable will reach 100% based on past progress.
- **Identify acceleration**: Detect when a project is in its rapid-improvement phase versus plateauing.
- **Set realistic milestones**: Calibrate schedule targets against demonstrated learning rates.

7.2. The Three Models

PRA provides three sigmoidal model types:

Model	Formula	Parameters
Logistic	$K/(1+e^{-r(t-t_0)})$	K = ceiling; r = growth rate; t = inflection time
Pearl	$K/(1+e^{-r(t-t_0)})$	Same functional form as Logistic
Gompertz	$A \cdot e^{-b \cdot e^{-ct}}$	A = ceiling; c = growth rate; b = initial suppression

The Logistic and Pearl models are mathematically identical but fitted differently. The Gompertz has a different shape: its inflection point occurs earlier and the curve is asymmetric, making it better suited for processes where acceleration comes quickly and the plateau is long.

7.3. Example: Fitting a Logistic Model

```
library(PRA)
```

We have weekly completion percentage data for a construction deliverable over 9 weeks.

```
data <- data.frame(
  time       = 1:9,
  completion = c(5, 15, 40, 60, 70, 75, 80, 85, 90)
)
```

7.3.1. Fit the Model

```
fit <- fit_sigmoidal(data, "time", "completion", "logistic")
```

7.3.2. Assess Fit Quality

Use `summary()` to examine the fitted coefficients, their standard errors, and the residual standard error, a measure of how closely the model matches the observed data.

```
summary(fit)
```

```
Sigmoidal (Logistic) model fit
Fitted asymptote: 84.35147
------------------------------

Formula: y ~ logistic(x, K, r, t0)

Parameters:
   Estimate Std. Error t value Pr(>|t|)
K   84.3515     2.5142  33.550 4.67e-08 ***
r    1.0356     0.1380   7.505 0.000289 ***
t0   3.2520     0.1459  22.284 5.34e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.148 on 6 degrees of freedom

Number of iterations to convergence: 10
Achieved convergence tolerance: 1.49e-08
```

A small residual standard error (relative to the response scale) indicates a good fit. Coefficient t values with $|t| > 2$ are statistically meaningful.

7.3.3. Plot with Confidence Bands

`plot_sigmoidal()` plots the data, fitted curve, and optional confidence bounds in a single call.

```
plot_sigmoidal(
  fit, data, "time", "completion", "logistic",
  conf_level = 0.95,
  main       = "Logistic Learning Curve: Completion Forecast",
  xlab       = "Week",
  ylab       = "Completion (%)"
)
```

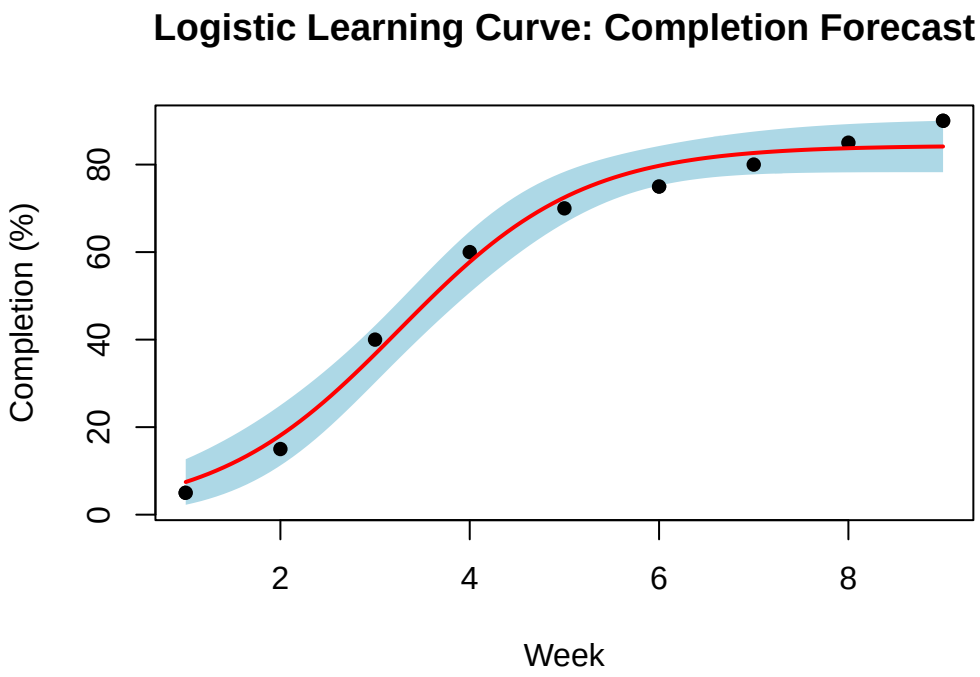


Figure 7.1.: Logistic learning curve fit with 95% confidence band. Wider bands at the tails reflect greater extrapolation uncertainty.

The shaded region is the 95% confidence band, the range within which the true curve is likely to lie. Note how the band widens as we extrapolate past the observed data range.

7.3.4. Predict Future Completion

Use `predict_sigmoidal()` to generate numeric forecasts, including confidence bounds.

```
future_times <- seq(1, 12, length.out = 100)
predictions  <- predict_sigmoidal(fit, future_times, "logistic", conf_level = 0.95)

knitr::kable(
  tail(round(predictions, 1), 5),
  caption  = "Predicted completion (final 5 forecast points)",
  row.names = FALSE
)
```

Table 7.1.: Predicted completion (final 5 forecast points)

x	pred	lwr	upr
11.6	84.3	78.2	90.5

x	pred	lwr	upr
11.7	84.3	78.2	90.5
11.8	84.3	78.2	90.5
11.9	84.3	78.2	90.5
12.0	84.3	78.2	90.5

7.4. Comparing All Three Model Types

It is good practice to fit multiple models and compare their goodness-of-fit. Different shapes may fit your data better.

```
fit_logistic <- fit_sigmoidal(data, "time", "completion", "logistic")
fit_pearl    <- fit_sigmoidal(data, "time", "completion", "pearl")
fit_gompertz <- fit_sigmoidal(data, "time", "completion", "gompertz")
```

```
rse <- function(fit) summary(fit)$sigma

comparison <- data.frame(
  Model             = c("Logistic", "Pearl", "Gompertz"),
  Residual_StdError = round(c(rse(fit_logistic), rse(fit_pearl), rse(fit_gompertz)), 3)
)
knitr::kable(comparison, caption = "Model Fit Comparison (lower RSE = better fit)")
```

Table 7.2.: Model Fit Comparison (lower RSE = better fit)

Model	Residual_StdError
Logistic	4.148
Pearl	4.148
Gompertz	2.887

Now plot all three fits side by side:

```
x_seq <- seq(1, 12, length.out = 200)

pred_log <- predict_sigmoidal(fit_logistic, x_seq, "logistic")
pred_prl <- predict_sigmoidal(fit_pearl,    x_seq, "pearl")
pred_gom <- predict_sigmoidal(fit_gompertz, x_seq, "gompertz")

plot(data$time, data$completion,
  pch = 16, xlim = c(1, 12), ylim = c(0, 105),
  main = "Learning Curve: Model Comparison",
  xlab = "Week", ylab = "Completion (%)"
)
```

```
lines(pred_log$x, pred_log$pred, col = "steelblue", lwd = 2)
lines(pred_prl$x, pred_prl$pred, col = "tomato",    lwd = 2, lty = 2)
lines(pred_gom$x, pred_gom$pred, col = "darkgreen", lwd = 2, lty = 3)
legend("bottomright",
  legend = c("Logistic", "Pearl", "Gompertz"),
  col    = c("steelblue", "tomato", "darkgreen"),
  lty    = c(1, 2, 3), lwd = 2, bty = "n"
)
```

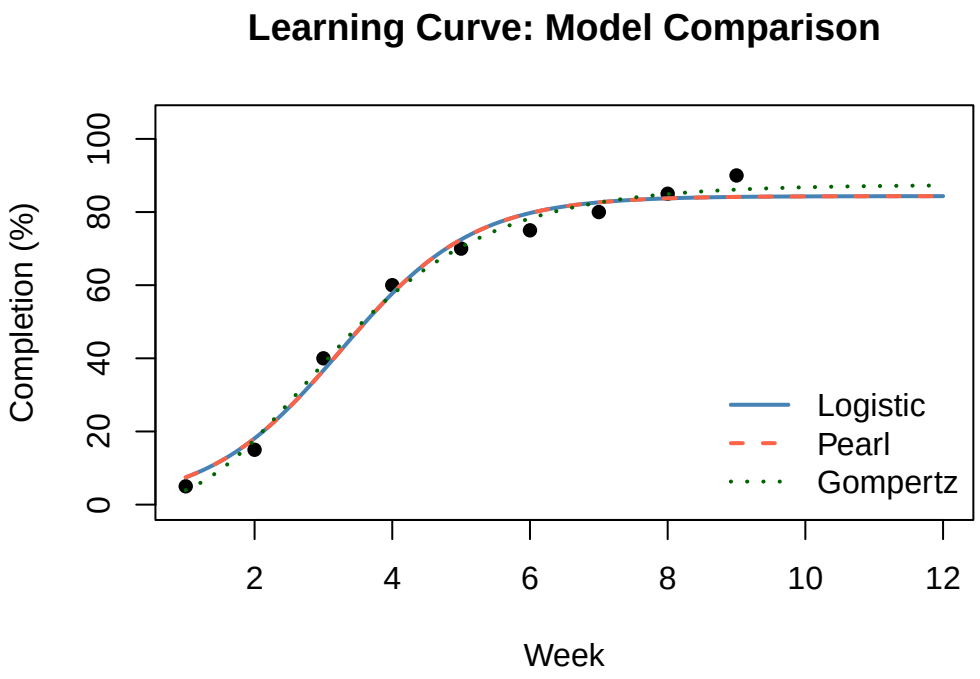


Figure 7.2.: Comparison of Logistic, Pearl, and Gompertz fits. Logistic and Pearl are identical; Gompertz has a different early-acceleration shape.

> ⚠ Logistic and Pearl Are the Same Model
>
> The Logistic and Pearl models share the same mathematical formula and will produce **identical fits** on the same data. The distinction is historical, not functional. When comparing model fits, the Pearl and Logistic RSEs will always be equal; this is expected behavior, not a bug. Gompertz is genuinely different: its inflection point occurs earlier and the curve is asymmetric, making it better suited for processes that accelerate quickly before plateauing slowly.

7.5. The Workflow

The sigmoidal workflow in PRA:

1. `fit_sigmoidal()`: fit a model to observed time-completion data
2. `summary(fit)`: inspect coefficient estimates and goodness-of-fit
3. `predict_sigmoidal()`: generate numeric forecasts with optional confidence bounds
4. `plot_sigmoidal()`: visualize the fit and confidence band

Choose the model type based on the shape of your data and theoretical expectations about the learning process.

7.6. Summary

Key Takeaways

- Sigmoidal models capture the three universal phases of learning: slow start, rapid improvement, and plateau, making them physically meaningful for completion forecasting.
- **Logistic and Pearl are identical**: if they give different RSEs, something has gone wrong with the fit. Use Logistic unless you have a specific reason to use Pearl.
- **Gompertz** accelerates earlier and plateaus more gradually; choose it when you have evidence of rapid early-stage learning.
- **Wider confidence bands** at later time points reflect extrapolation uncertainty; the further you predict beyond the observed data, the less trustworthy the forecast.
- Always check both the fitted curve visually *and* the RSE numerically, as a low RSE with a nonsensical curve shape means overfitting.

7.7. Exercises

1. **Model selection.** Which model (Logistic, Pearl, Gompertz) would be most appropriate when early progress is slow but then the team suddenly accelerates around week 3? What parameter would you adjust to shift the inflection point?

2. **Fit comparison.** Using the construction data from this chapter, fit all three models and compare their residual standard errors. Which model fits best? Does the best-fitting model always give the most sensible forecast? Plot the predictions out to week 15 and compare the three extrapolations.

3. **Confidence band width.** Predict percent complete at week 12 using the logistic model. How wide is the 95% confidence band (upper − lower)? At week 6? At week 3? What drives the increasing uncertainty as you extrapolate further from the observed data?

4. **Your own data.** Collect or invent 8 weeks of progress data for a project activity. Fit a logistic model. Does the model suggest you'll reach 90% completion by week 12? By week 16? What does the confidence interval tell you about the reliability of that forecast?

5. **Beyond completion.** Learning curves can model more than completion percentages; they can model cost efficiency, defect rates, or productivity. Choose one of these alternative interpretations and describe how you would set up the `data` frame, what "ceiling" K means in that context, and what a fitted model would tell you.

8. It's a Small World After All: Probabilistic Networks

"Everything is connected to everything else." — Barry Commoner's First Law of Ecology

A risk register lists risks. A probability network shows you how those risks talk to each other, and to your project. It's the difference between knowing that "Technical Complexity" is a risk and understanding exactly how it flows through your developer costs into your total project budget, and what happens to that flow when you learn new information.

Bayesian networks are the tool. A Bayesian network is a specific type of probabilistic network, a directed acyclic graph (DAG) in which each node represents a random variable and each edge encodes a conditional probability relationship. The "probabilistic network" framing in this chapter's title is intentional: the core concepts (conditioning, propagation, graph structure) apply to the broader class, and `PRA` uses the Bayesian network formulation specifically. They combine graph theory with probability theory to model the full dependency structure of a project: which risks drive which resources, which resources drive which tasks, and how uncertainty propagates all the way up to the project total.

Learning Objectives

By the end of this chapter, you will be able to:

1. Build a probabilistic network using `prob_net()` with nodes, edges, and distributions
2. Run forward simulations with `prob_net_sim()` and interpret the cost distribution
3. Incorporate new evidence using `prob_net_learn()` and observe downstream shifts
4. Modify network structure and distributions with `prob_net_update()`
5. Explain the difference between learning (conditioning) and updating (graph surgery)

8.1. What Is a Bayesian Network?

Bayesian Network vs. DSM

The Design Structure Matrix (Chapter 9) shows *structural* coupling, specifically how many resources two tasks share. A Bayesian network goes further: it encodes **probabilistic dependencies**, meaning the actual conditional distributions of costs given risk states. Where the DSM counts connections, the network simulates consequences.
Use a DSM for rapid structural triage. Use a Bayesian network when you need actual cost distributions and the ability to condition on observed evidence.

A Bayesian network is a directed acyclic graph (DAG) where:

- **Nodes** represent random variables (risks, resources, tasks, project totals)
- **Edges** represent conditional dependencies (Risk A affects Resource C)
- **Distributions** encode the uncertainty at each node

Bayesian networks are well-suited to project risk analysis because they explicitly model how risk events propagate through resources and tasks to affect total project cost (Govan 2014).

For a more advanced example covering causal inference, graph surgery, and the see-versus-do distinction across a full project portfolio, see Chapter 10.

8.2. Project Setup

8.2.1. Tasks

Consider a small software development project with three tasks.

```
library(PRA)
set.seed(42)
```

```
tasks <- data.frame(
  ID    = c("F", "G", "H"),
  Label = c("Task-1", "Task-2", "Task-3"),
  Task  = c("Requirements and Design", "Development", "Testing and Handover")
)
knitr::kable(tasks, caption = "Project Tasks")
```

Table 8.1.: Project Tasks

ID	Label	Task
F	Task-1	Requirements and Design
G	Task-2	Development
H	Task-3	Testing and Handover

8.2.2. Resources

Each task draws on one primary resource. The table below shows the baseline cost estimate for each resource.

```
resources <- data.frame(
  ID       = c("C", "D", "E"),
  Label    = c("Resource-1", "Resource-2", "Resource-3"),
  Resource = c("Business Analyst", "Developer", "QA Engineer"),
  Task_ID  = c("F", "G", "H"),
```

```
  Mean      = c(15000, 50000, 20000),
  SD        = c(3000,  10000,  4000)
)
knitr::kable(resources, caption = "Project Resources")
```

Table 8.2.: Project Resources

ID	Label	Resource	Task_ID	Mean	SD
C	Resource-1	Business Analyst	F	15000	3000
D	Resource-2	Developer	G	50000	10000
E	Resource-3	QA Engineer	H	20000	4000

8.2.3. Risks

Two risk events can escalate resource costs if they occur.

```
risks <- data.frame(
  Risk_ID        = c("A", "B"),
  Risk           = c("Requirements Scope Creep", "Technical Complexity"),
  Probability    = c(0.70, 0.60),
  Resource       = c("Business Analyst", "Developer"),
  Mean_if_occurs = c(30000, 80000),
  SD_if_occurs   = c(8000,  20000)
)
knitr::kable(risks, caption = "Project Risks")
```

Table 8.3.: Project Risks

Risk_ID	Risk	Probability	Resource	Mean_if_occurs	SD_if_occurs
A	Requirements Scope Creep	0.7	Business Analyst	30000	8000
B	Technical Complexity	0.6	Developer	80000	20000

If Risk-1 (Requirements Scope Creep) occurs, the Business Analyst cost rises from $15,000 to $30,000. If Risk-2 (Technical Complexity) occurs, the Developer cost rises from $50,000 to $80,000. The QA Engineer is unaffected by either risk.

8.3. Building the Bayesian Network

8.3.1. Nodes

```
nodes <- data.frame(
  id    = c("A", "B", "C", "D", "E", "F", "G", "H", "I"),
  label = c(
    "Risk-1", "Risk-2",
    "Resource-1", "Resource-2", "Resource-3",
    "Task-1", "Task-2", "Task-3",
    "Project"
  ),
  group = c(
    "Risk", "Risk",
    "Resource", "Resource", "Resource",
    "Task", "Task", "Task",
    "Project"
  ),
  stringsAsFactors = FALSE
)
```

8.3.2. Edges

Edges encode the causal dependencies: risks affect resources, resources drive tasks, and tasks roll up to the project total.

```
links <- data.frame(
  source = c("A", "B", "C", "D", "E", "F", "G", "H"),
  target = c("C", "D", "F", "G", "H", "I", "I", "I"),
  value  = rep(1, 8),
  stringsAsFactors = FALSE
)
```

8.3.3. Distributions

```
distributions <- list(
  A = list(type = "discrete", values = c(1, 0), probs = c(0.70, 0.30)),
  B = list(type = "discrete", values = c(1, 0), probs = c(0.60, 0.40)),
  C = list(
    type = "conditional", condition = "A",
    true_dist  = list(type = "normal", mean = 30000, sd = 8000),
    false_dist = list(type = "normal", mean = 15000, sd = 3000)
  ),
  D = list(
    type = "conditional", condition = "B",
    true_dist  = list(type = "normal", mean = 80000, sd = 20000),
    false_dist = list(type = "normal", mean = 50000, sd = 10000)
  ),
```

```
  E = list(type = "normal", mean = 20000, sd = 4000),
  F = list(type = "aggregate", nodes = c("C")),
  G = list(type = "aggregate", nodes = c("D")),
  H = list(type = "aggregate", nodes = c("E")),
  I = list(type = "aggregate", nodes = c("F", "G", "H"))
)
```

8.3.4. Build the Graph

```
graph <- prob_net(nodes, links, distributions = distributions)
```

The network can be visualized with the `igraph` and `networkD3` packages.

```
library(igraph)
library(networkD3)
g   <- graph_from_data_frame(graph$links, vertices = graph$nodes, directed = TRUE)
d3g <- igraph_to_networkD3(g, group = graph$nodes$group)
```

```
forceNetwork(
  Links = d3g$links, Nodes = d3g$nodes, NodeID = "name", Group = "group",
  Value = "value", zoom = TRUE, legend = TRUE, arrows = TRUE,
  opacity = 0.8, fontSize = 14
)
```

```
plot(
  g,
  vertex.color = as.factor(graph$nodes$group),
  vertex.size = 14, vertex.label.cex = 0.7,
  edge.arrow.size = 0.4, layout = layout_with_sugiyama(g)$layout
)
```

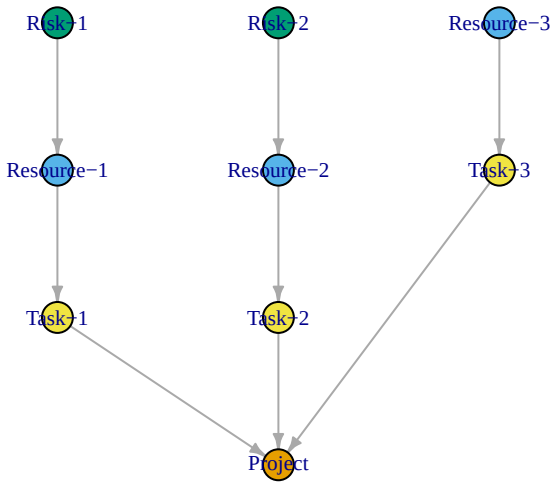


Figure 8.1.: Probabilistic network of risks, resources, tasks, and project cost.

8.4. Inference: Forward Simulation

Use `prob_net_sim()` to forward-simulate the network and estimate the total project cost distribution.

```
sim_results <- prob_net_sim(graph, num_samples = 10000)
```

```
hist(sim_results$I, breaks = 60,
     main = "Total Project Cost",
     xlab = "Cost ($)", col = "skyblue", border = "white")
```

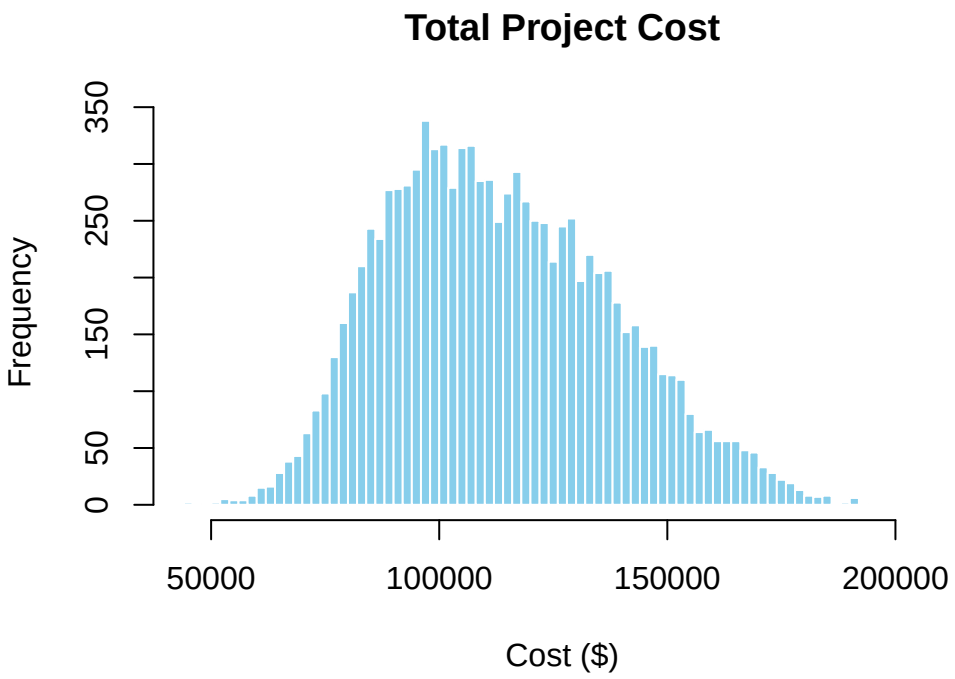


Figure 8.2.: Total project cost distribution from 10,000 forward simulations. The right tail represents scenarios where both risks occur simultaneously.

The spread reflects compounded uncertainty from both risk events. The right tail represents the worst case: both risks occur.

8.5. Learning: Incorporating New Evidence

Use `prob_net_learn()` to clamp one or more nodes to observed values and re-simulate. This shows the downstream effect of new information, for example, learning that Technical Complexity (Risk-2) did not materialize.

```
learn_results <- prob_net_learn(
  graph,
  observations = list(B = 0),
  num_samples  = 10000
)
```

```
hist_before <- hist(sim_results$D,   breaks = 60, plot = FALSE)
hist_after  <- hist(learn_results$D, breaks = 60, plot = FALSE)

plot(
  hist_before,
```

```
  main  = "Developer Cost: Before vs. After Observing Risk-2 = No",
  xlab  = "Cost ($)", col = "skyblue", border = "white",
  ylim  = c(0, max(hist_before$counts, hist_after$counts))
)
plot(hist_after, col = rgb(0, 0, 1, 0.5), border = "white", add = TRUE)
legend(
  "topright",
  legend = c("Before (Risk-2 uncertain)", "After (Risk-2 = No)"),
  fill   = c("skyblue", rgb(0, 0, 1, 0.5)), bty = "n"
)
```

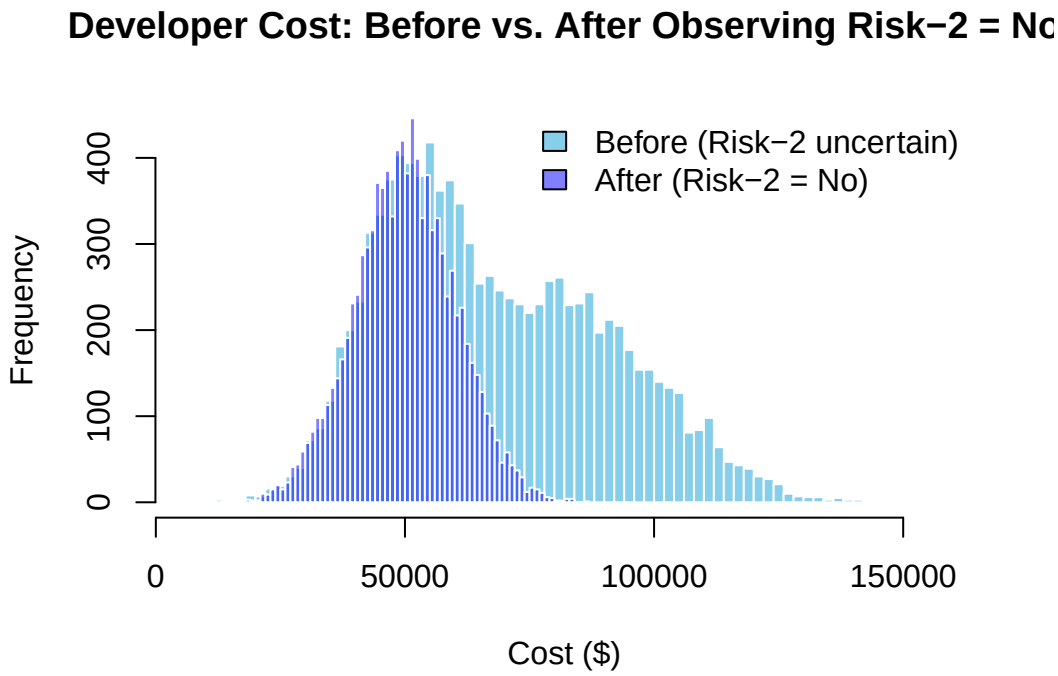


Figure 8.3.: Developer cost before and after observing Risk-2 = No. The distribution collapses to the lower baseline.

With Risk-2 ruled out, the Developer cost collapses to the lower baseline distribution, and the total project cost shifts left accordingly.

8.6. Updating: Modifying the Network

Use `prob_net_update()` to modify the network structure or distributions. Suppose a design review eliminates Requirements Scope Creep as a concern: remove the arc from Risk-1 to Resource-1 and replace the conditional distribution with a fixed normal.

```
updated_graph <- prob_net_update(
  graph,
  remove_links = data.frame(source = "A", target = "C", stringsAsFactors = FALSE),
  update_distributions = list(
    C = list(type = "normal", mean = 15000, sd = 3000)
  )
)
updated_results <- prob_net_sim(updated_graph, num_samples = 10000)
```

```
hist_before <- hist(sim_results$C,      breaks = 60, plot = FALSE)
hist_after  <- hist(updated_results$C,  breaks = 60, plot = FALSE)

plot(
  hist_before,
  main  = "Business Analyst Cost: Before vs. After Removing Risk-1",
  xlab  = "Cost ($)", col = "skyblue", border = "white",
  ylim  = c(0, max(hist_before$counts, hist_after$counts))
)
plot(hist_after, col = rgb(0, 0, 1, 0.5), border = "white", add = TRUE)
legend(
  "topright",
  legend = c("Before (Risk-1 possible)", "After (Risk-1 removed)"),
  fill   = c("skyblue", rgb(0, 0, 1, 0.5)), bty = "n"
)
```

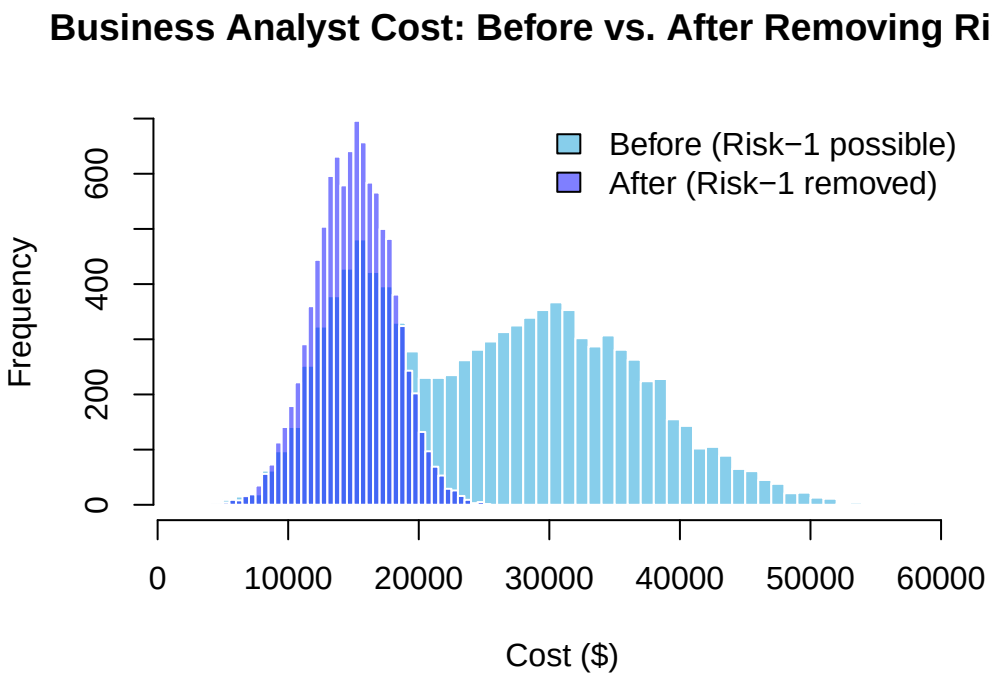


Figure 8.4.: Business Analyst cost before and after removing the Risk-1 arc. The heavy right tail disappears.

8.7. The Four Core Functions

Function	What it does
`prob_net()`	Constructs the network from nodes, edges, and distributions
`prob_net_sim()`	Forward-simulates to estimate cost distributions
`prob_net_learn()`	Clamps observed nodes and re-simulates to propagate evidence
`prob_net_update()`	Modifies network structure and distributions as the project evolves

8.8. Summary

Key Takeaways

- A probabilistic network models the full dependency chain from risks → resources → tasks → project cost, encoding uncertainty at every layer.
- `prob_net_sim()` draws Monte Carlo samples in topological order, propagating risk states through the network to produce a total cost distribution.
- `prob_net_learn()` conditions on observed evidence (*seeing*): clamping a node to its observed value and re-simulating the downstream distribution.
- `prob_net_update()` performs structural changes (*doing*): removing edges or replacing distributions, enabling causal interventions beyond what conditioning can express.
- The difference between *seeing* and *doing* becomes consequential when nodes share upstream parents; see Chapter 10 for the full treatment with shared enterprise risks.

8.9. Exercises

1. **Learning effect.** In the example, we observed Risk-2 = No (`B = 0`). What do you expect happens to the total project cost distribution (node I) when Risk-2 is observed as occurring instead? Test your prediction by running `prob_net_learn()` with `list(B = 1)` and plotting the result.

2. **Modify a risk.** Change the probability of Risk-1 from 0.70 to 0.30. How does this affect the mean total project cost from `prob_net_sim()`? Is the change proportional to the probability change?

3. **Add a QA risk.** Modify the network so that a new Risk-3 (with probability 0.40) affects the QA Engineer (Resource-3), increasing their cost from a mean of $20K to $35K if it occurs. Update the distributions and re-simulate. How much does this add to the expected project cost?

4. **Seeing vs. doing.** Explain the difference between `prob_net_learn(observations = list(B = 0))` and `prob_net_update(remove_links = ...)`. When would you use each? Which corresponds to "seeing" and which to "doing" in the causal inference sense? (See Chapter 10 for the full treatment.)

5. **Project structure.** This example has a simple layered structure (Risk → Resource → Task → Project). Design a more complex network with two risks that both affect the same resource. What does that mean for the correlation between the two downstream tasks? Build the network and verify with a correlation matrix of `sim_results`.

9. Everything Is Connected: Design Structure Matrices

> *"A complex system that works is invariably found to have evolved from a simple system that worked."* — John Gall

The previous chapter built a probabilistic network that showed *how* risk flows through resources into task costs. But the Bayesian network is a rich machine — conditional distributions, sampling, belief propagation. Sometimes you want a simpler question: *which tasks are structurally coupled?* Which pairs of tasks would fail together purely because they share the same crew, equipment, or material — regardless of the probability distributions involved?

The Design Structure Matrix (DSM) answers that question (Browning 2001; Steward 1981). And because this project's structure already lives in the `graph` object built in the previous chapter, there is no need to construct the resource and risk matrices by hand — they can be derived directly from the network's adjacency matrix.

Learning Objectives

By the end of this chapter, you will be able to:

1. Extract a Resource-Task matrix **S** and a Risk-Resource matrix **R** from a `prob_net` adjacency matrix
2. Compute the Parent DSM (resource-based coupling) using `parent_dsm()`
3. Compute the Grandparent DSM (risk-based coupling) using `grandparent_dsm()`
4. Interpret diagonal vs. off-diagonal DSM entries
5. Use DSM coupling values to prioritize risk mitigation decisions

9.1. What Is a DSM?

A DSM is a square matrix that maps dependencies between tasks in a project. In PRA, DSMs are derived from bipartite relationships: resources are shared across tasks, and risks are shared across resources. These shared dependencies create coupling between tasks that can propagate delays, cost overruns, or failures.

PRA provides two DSM functions:

- **`parent_dsm()`**: the Resource-based "Parent" DSM: shows how many resources are shared between each pair of tasks.
- **`grandparent_dsm()`**: the Risk-based "Grandparent" DSM: shows how many risks are shared between task pairs (via the resource layer).

The key idea from the resource-based view of project management (Govan and Damnjanovic 2016): shared resources are the structural pathway through which risks propagate. If you want to understand why two tasks always seem to get delayed together, look at what resources they share.

In the PRA framework, the same adjacency matrix that encodes the Bayesian network's topology also encodes the bipartite relationships needed to build **S** and **R** — no separate data entry required.

9.2. Rebuilding the Project Network

We continue with the software development project from Chapter 8. Rebuild the `graph` object to access its adjacency matrix.

```
library(PRA)
set.seed(42)
```

```
nodes <- data.frame(
  id    = c("A", "B", "C", "D", "E", "F", "G", "H", "I"),
  label = c(
    "Risk-1", "Risk-2",
    "Resource-1", "Resource-2", "Resource-3",
    "Task-1", "Task-2", "Task-3",
    "Project"
  ),
  group = c(
    "Risk", "Risk",
    "Resource", "Resource", "Resource",
    "Task", "Task", "Task",
    "Project"
  ),
  stringsAsFactors = FALSE
)

links <- data.frame(
  source = c("A", "B", "C", "D", "E", "F", "G", "H"),
  target = c("C", "D", "F", "G", "H", "I", "I", "I"),
  value  = rep(1, 8),
  stringsAsFactors = FALSE
)

distributions <- list(
  A = list(type = "discrete", values = c(1, 0), probs = c(0.70, 0.30)),
  B = list(type = "discrete", values = c(1, 0), probs = c(0.60, 0.40)),
  C = list(
    type = "conditional", condition = "A",
    true_dist  = list(type = "normal", mean = 30000, sd = 8000),
    false_dist = list(type = "normal", mean = 15000, sd = 3000)
```

```
  ),
  D = list(
    type = "conditional", condition = "B",
    true_dist  = list(type = "normal", mean = 80000, sd = 20000),
    false_dist = list(type = "normal", mean = 50000, sd = 10000)
  ),
  E = list(type = "normal", mean = 20000, sd = 4000),
  F = list(type = "aggregate", nodes = c("C")),
  G = list(type = "aggregate", nodes = c("D")),
  H = list(type = "aggregate", nodes = c("E")),
  I = list(type = "aggregate", nodes = c("F", "G", "H"))
)

graph <- prob_net(nodes, links, distributions = distributions)
```

9.3. The Adjacency Matrix

The `graph` object exposes a full 9×9 adjacency matrix — one row and column for every node. `prob_net()` builds it as a **directed** matrix: an entry of 1 at `[i, j]` means an edge runs from node i to node j (as declared in `links`), and the entry at `[j, i]` stays 0 unless a separate edge runs the other way.

Internally, `prob_net()` labels rows and columns with node IDs. For readability, we rename them with node labels before displaying or extracting:

```
adj <- graph$adjacency_matrix
rownames(adj) <- graph$nodes$label
colnames(adj) <- graph$nodes$label
adj
```

```
           Risk-1 Risk-2 Resource-1 Resource-2 Resource-3 Task-1 Task-2 Task-3
Risk-1          0      0          1          0          0      0      0      0
Risk-2          0      0          0          1          0      0      0      0
Resource-1      0      0          0          0          0      1      0      0
Resource-2      0      0          0          0          0      0      1      0
Resource-3      0      0          0          0          0      0      0      1
Task-1          0      0          0          0          0      0      0      0
Task-2          0      0          0          0          0      0      0      0
Task-3          0      0          0          0          0      0      0      0
Project         0      0          0          0          0      0      0      0
           Project
Risk-1           0
Risk-2           0
Resource-1       0
Resource-2       0
```

```
Resource-3       0
Task-1           1
Task-2           1
Task-3           1
Project          0
```

The structural information we need — which resources connect to which tasks, and which risks connect to which resources — is already encoded here. We extract it by slicing `adj` using the `group` column of `graph$nodes`. This works because `links` declares edges running Risk → Resource → Task → Project: slicing `adj[source_labels, target_labels]` for each layer lines up rows with the edge source and columns with the edge target.

9.4. The Resource-Task Matrix

Rows are resource nodes (Resource-1, Resource-2, Resource-3); columns are task nodes (Task-1, Task-2, Task-3). The slice gives a binary matrix $\mathbf{S}$ where `S[i, j] = 1` means resource i is used by task j.

```
resource_labels <- graph$nodes$label[graph$nodes$group == "Resource"]
task_labels     <- graph$nodes$label[graph$nodes$group == "Task"]

S <- adj[resource_labels, task_labels]
S
```

```
           Task-1 Task-2 Task-3
Resource-1      1      0      0
Resource-2      0      1      0
Resource-3      0      0      1
```

The result is a 3×3 identity matrix: each resource is dedicated to exactly one task, with no cross-task sharing. This is a favorable project design — dedicated resources mean independent failure modes.

9.5. Parent DSM

The Parent DSM $\mathbf{P} = \mathrm{t}(\mathbf{S})$ %*% $\mathbf{S}$ is a tasks-by-tasks matrix. The diagonal entry `P[j, j]` counts how many resources task j uses. The off-diagonal entry `P[j, k]` counts how many resources tasks j and k share.

```
p <- parent_dsm(S)
print(p)
```

```
Resource-based 'Parent' Design Structure Matrix
Tasks: 3  Resources: 3

       Task-1 Task-2 Task-3
Task-1      1      0      0
Task-2      0      1      0
Task-3      0      0      1
```

```
plot(p)
```

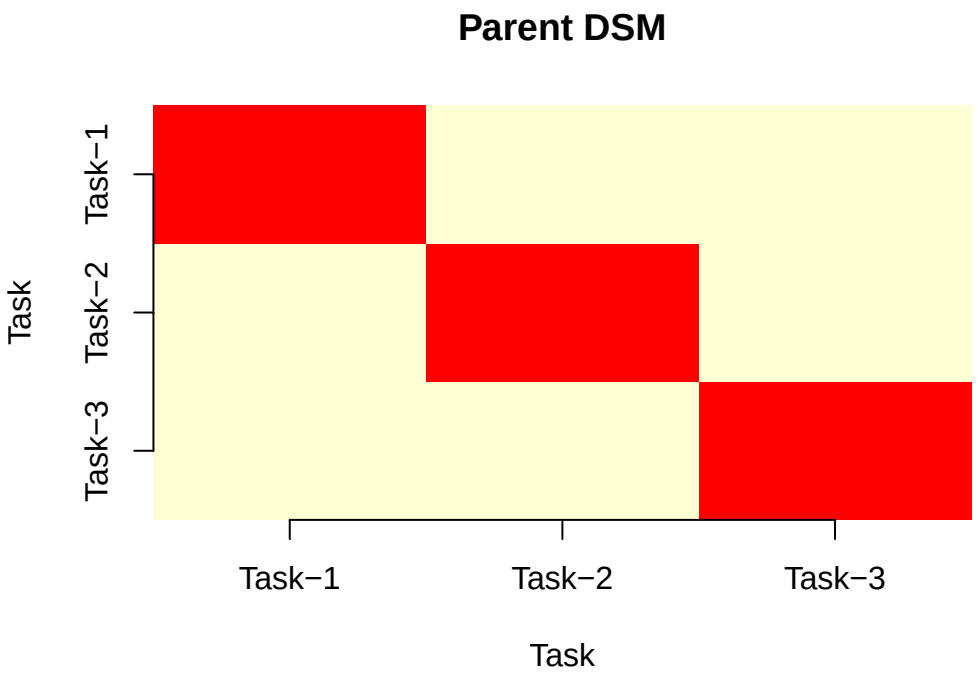


Figure 9.1.: Parent DSM heatmap. All coupling is on the diagonal: each task uses one dedicated resource and no two tasks compete for the same crew.

All off-diagonal entries are zero: no two tasks share a resource. A disruption to the Business Analyst (Resource-1) hits Task-1 only — it does not propagate structurally into Task-2 or Task-3. This is the DSM's way of confirming what the Bayesian network showed probabilistically: the three risk paths are disjoint.

9.6. The Risk-Resource Matrix

The Risk-Resource Matrix **R** adds a second layer. Rows represent risks (Risk-1, Risk-2) and columns represent resources (Resource-1, Resource-2, Resource-3). An entry `R[i, j] = 1` means risk i affects resource j.

```
risk_labels <- graph$nodes$label[graph$nodes$group == "Risk"]

R <- adj[risk_labels, resource_labels]
R
```

```
       Resource-1 Resource-2 Resource-3
Risk-1          1          0          0
Risk-2          0          1          0
```

Risk-1 (Scope Creep) affects only Resource-1; Risk-2 (Technical Complexity) affects only Resource-2. Resource-3 (QA Engineer) is unaffected by either risk — consistent with the distributions in Chapter 8.

9.7. Grandparent DSM

The Grandparent DSM traces the dependency chain from risks through resources to tasks. The intermediate matrix $\mathbf{T} = \mathbf{R}$ %% $\boldsymbol{S}$ *gives a risks-by-tasks mapping, and the Grandparent DSM is* $\boldsymbol{G}$ = *t(*$\boldsymbol{T}$*)* %% $\mathbf{T}$. Off-diagonal entries count how many risks are shared between each pair of tasks.

```
g <- grandparent_dsm(S, R)
print(g)
```

```
Risk-based 'Grandparent' Design Structure Matrix
Tasks: 3  Resources: 3  Risks: 2

       Task-1 Task-2 Task-3
Task-1      1      0      0
Task-2      0      1      0
Task-3      0      0      0
```

```
plot(g)
```

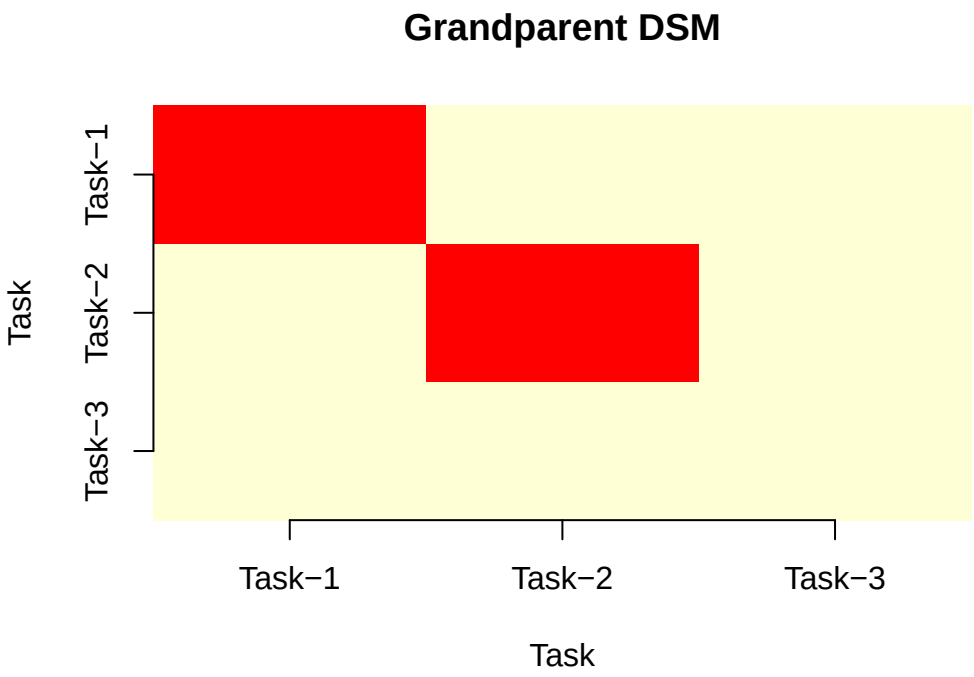


Figure 9.2.: Grandparent DSM heatmap. With each risk affecting a different resource, no two tasks share risk exposure: the structure is perfectly decoupled.

The Grandparent DSM is also diagonal. Since each risk touches a different resource and each resource a different task, no two tasks share risk exposure through the resource layer. This clean structure is precisely what makes the Bayesian network's total cost distribution well-behaved: the heavy tail in Chapter 8 comes from the risks themselves, not from structural coupling amplifying them.

9.8. Interpreting the DSM

- **Diagonal values** indicate the total number of resources (Parent) or risks (Grandparent) associated with each task. Higher values mean a task has more dependencies.
- **Off-diagonal values** indicate coupling between task pairs. Higher values mean more shared dependencies and greater potential for correlated disruption.
- **Symmetric structure**: both DSMs are always symmetric since shared dependencies are bidirectional: if task A shares a resource with task B, then task B shares that resource with task A.

This project has zero coupling — an idealized structure that real projects rarely achieve. The exercises below introduce shared resources and cross-resource risks to show how off-diagonal entries appear and what they mean.

9.9. From DSM to Decision

Using the DSM to Prioritize Mitigation

1. **Find the highest off-diagonal values** in the Parent DSM, as these task pairs share the most resources and are most likely to fail together.
2. **Identify those shared resources**, the structural bottlenecks through which risk propagates.
3. **Inspect the Grandparent DSM** for those same pairs; if they also share risks, the coupling is doubly reinforced.
4. **Prioritize mitigation** on the shared resources with the highest combined Parent + Grandparent coupling: add contingency, build redundancy, or create buffers.

The Grandparent DSM goes one level deeper: it shows which pairs of tasks are exposed to the same risks. If you can eliminate or mitigate a risk, you simultaneously reduce coupling between all tasks that share that risk.

9.10. Summary

Key Takeaways

- The DSM extracts **structural task dependencies** from the same adjacency matrix that defines the Bayesian network — no separate data entry required.
- **Parent DSM** (resource-based): `P = t(S) %*% S`. Off-diagonal `P[j,k]` = number of shared resources between tasks j and k.
- **Grandparent DSM** (risk-based): `G = t(T) %*% T` where `T = R %*% S`. Off-diagonal entries count shared risks via the resource layer.
- Both DSMs are always **symmetric**, since shared dependencies are bidirectional.
- High off-diagonal values point directly to the resources and risks that deserve the most mitigation investment.

DSM vs. Bayesian Network: When to Use Each

Both tools analyze the same project structure but answer different questions:

- **Use the DSM** for rapid structural triage: which tasks are coupled? Which resources are bottlenecks? No distributions required — only the resource-task and risk-resource linkages extracted from `graph$adjacency_matrix`.
- **Use a Bayesian network** when you need actual cost distributions, the ability to condition on observed evidence, or causal interventions (see Chapter 8). The same `graph$adjacency_matrix` is the structural skeleton of the Bayesian network.

A practical workflow: run the DSM first to identify high-coupling task pairs, then focus the Bayesian network analysis on those pairs to quantify the probabilistic consequences.

9.11. Exercises

1. **Reading the DSM.** In the Parent DSM computed above, all off-diagonal entries are 0. What does this tell you about the software development project's resource structure? Is this a favorable or unfavorable project design from a risk-propagation standpoint? What would need to change in the project for a non-zero off-diagonal entry to appear?

2. **Extend the matrix.** A Project Manager is added to the project and assigned to both Task-1 and Task-2. Add this resource to **S** and recompute the Parent DSM. Which entries change, and why?

```
S2 <- rbind(S, "Project Manager" = c(1, 1, 0))
p2 <- parent_dsm(S2)
print(p2)
```

3. **Risk propagation.** Add a third risk (Risk-3, probability 0.40) that affects both Resource-2 (Developer) and Resource-3 (QA Engineer). Update **R** and recompute the Grandparent DSM. Which task pair now shares a risk? Describe in plain English the scenario where this coupling causes the most damage.

```
R2 <- rbind(R, "Risk-3" = c(0, 1, 1))
g2 <- grandparent_dsm(S, R2)
print(g2)
```

4. **DSM symmetry.** Both the Parent and Grandparent DSMs are symmetric. Why? Is there a project structure where the DSM would not be symmetric? (Hint: think about directed dependencies; what if one task uses a resource but another task produces it?)

5. **From DSM to Monte Carlo.** Use the Parent DSM to derive a correlation matrix for MCS. With this project's base **S**, `cor_from_dsm` equals `diag(3)` — verify that the total variance under DSM-derived correlation matches the zero-correlation case. Then repeat the analysis using the extended **S2** from Exercise 2, and compare the total standard deviation. What did adding the shared Project Manager add to the risk picture?

```
p  <- parent_dsm(S2)
P  <- p$matrix
d  <- sqrt(diag(P))
cor_from_dsm <- P / outer(d, d)
diag(cor_from_dsm) <- 1

task_dists <- list(
  "Task-1" = list(type = "normal", mean = 20250, sd = 5900),
  "Task-2" = list(type = "normal", mean = 62000, sd = 16000),
  "Task-3" = list(type = "normal", mean = 20000, sd = 4000)
)

res_zero <- mcs(10000, task_dists, diag(3))
res_dsm  <- mcs(10000, task_dists, cor_from_dsm)
```

```
cat("SD (zero correlation):", round(res_zero$total_sd, 0), "\n")
cat("SD (DSM correlation): ", round(res_dsm$total_sd,  0), "\n")
```

10. The Portfolio Problem: When Risks Are Shared

"Seeing a fire is not the same as starting one." — The First Law of Causal Inference

In Chapter 8, we built a probabilistic network for one project. Now we're going to scale up to a portfolio of three projects, introduce an upstream root cause, and answer the question that keeps enterprise risk managers up at night: when one project gets hit by a risk, what happens to all the others?

The answer depends on whether the risks are truly independent or whether they share a common driver. And it depends on something subtle but important: the difference between *seeing* a risk happen and *doing* something to prevent it. These two things look similar but behave completely differently in a causal network.

Learning Objectives

By the end of this chapter, you will be able to:

1. Model a multi-project portfolio with shared enterprise risks and a root cause
2. Compute and interpret the task-cost correlation matrix driven by shared risks
3. Distinguish observational, seeing, and doing queries on a causal network
4. Implement graph surgery with `prob_net_update()` to model interventions
5. Rank enterprise risks by their contribution to portfolio cost variance

10.1. The Portfolio Case Study

Consider an enterprise running three projects simultaneously: a **Road Repair**, a **Park Construction**, and a **Building Renovation**. Each project has three tasks driven by three resources: labor, materials, and equipment.

Critically, these resources are **shared at the enterprise level**: the same labor pool, the same material suppliers, and the same equipment fleet serve all three projects. Three enterprise-wide risk events propagate through the corresponding shared resource to impact all three projects simultaneously.

Even more critically, two of these risks, a **Labor Shortage** and a **Material Price Spike**, share a common upstream driver: a **Supply Chain Disruption** that simultaneously tightens the labor market and raises commodity prices. This shared root cause is what makes the distinction between *seeing* and *doing* consequential (Pearl 2009; Govan 2014).

⚠ Seeing Is Not the Same as Doing

Seeing $A = 1$ (observing a Labor Shortage) conditions on the evidence in the *original* graph. Bayes' rule propagates upstream: it raises the posterior probability of the Supply Chain Disruption, which in turn raises the probability of the Material Price Spike. A side-effect flows through the shared root cause.
Doing $\text{do}(A = 1)$ (intervening to cause a Labor Shortage) severs the SC $\rightarrow$ A edge via graph surgery. SC stays at its prior, B stays near its prior, and material costs are unaffected. Only labor costs change.
The gap between the two distributions, seeing vs. doing, is the operational signature of the shared root cause SC. Confusing the two leads to systematically underestimating portfolio cost exposure (Pearl 2009).

10.2. Setup

```
library(PRA)
library(igraph)
library(networkD3)
library(corrplot)
set.seed(42)
```

10.2.1. Tasks

Table 10.1.: Project 1: Road Repair

ID	Label	Task
M	Task-1.1	Site Preparation
N	Task-1.2	Road Paving
O	Task-1.3	Final Inspection

Table 10.2.: Project 2: Park Construction

ID	Label	Task
P	Task-2.1	Site Preparation
Q	Task-2.2	Planting and Landscaping
R	Task-2.3	Final Inspection

Table 10.3.: Project 3: Building Renovation

ID	Label	Task
S	Task-3.1	Demolition
T	Task-3.2	Renovation and Build-Out
U	Task-3.3	Final Inspection

10.2.2. Risks and Root Cause

```
root_cause <- data.frame(
  ID = "SC", Event = "Supply Chain Disruption", P_occurs = 0.70,
  Children = "A (Labor Shortage), B (Material Price Spike)"
)
risks <- data.frame(
  Risk_ID = c("A", "B", "C"),
  Risk    = c("Labor Shortage", "Material Price Spike", "Weather Delay"),
  Parent  = c("SC", "SC", "-"),
  P_marginal = c("  0.79", "  0.67", "0.60"),
  Resource_Impacted = c("Labor (D,G,J)", "Materials (E,H,K)", "Equipment (F,I,L)")
)
knitr::kable(root_cause, caption = "Root Cause (SC)")
```

Table 10.4.: Root Cause (SC)

ID	Event	P_occurs	Children
SC	Supply Chain Disruption	0.7	A (Labor Shortage), B (Material Price Spike)

```
knitr::kable(risks, caption = "Enterprise Risks")
```

Table 10.5.: Enterprise Risks

Risk_ID	Risk	Parent	P_marginal	Resource_Impacted
A	Labor Shortage	SC	0.79	Labor (D,G,J)
B	Material Price Spike	SC	0.67	Materials (E,H,K)
C	Weather Delay	—	0.60	Equipment (F,I,L)

The shared root cause SC is what makes *seeing* and *doing* diverge: observing $A = 1$ raises the posterior probability of SC, which in turn raises the probability of $B = 1$, a side-effect that $\text{do}(A = 1)$ does not produce.

10.3. Building the Portfolio Network

```
nodes <- data.frame(
  id = c("SC","A","B","C","D","E","F","G","H","I","J","K","L",
         "M","N","O","P","Q","R","S","T","U","V","W","X","Y"),
  label = c(
    "Supply Chain Disruption",
    "Risk-1 (Labor Shortage)","Risk-2 (Material Price Spike)","Risk-3 (Weather Delay)",
    "Labor-1","Materials-1","Equipment-1",
    "Labor-2","Materials-2","Equipment-2",
    "Labor-3","Materials-3","Equipment-3",
    "Task-1.1","Task-1.2","Task-1.3",
    "Task-2.1","Task-2.2","Task-2.3",
    "Task-3.1","Task-3.2","Task-3.3",
    "Project 1","Project 2","Project 3","Portfolio"
  ),
  group = c(
    "Root Cause","Risk","Risk","Risk",
    "Resource","Resource","Resource","Resource","Resource","Resource",
    "Resource","Resource","Resource",
    "Task","Task","Task","Task","Task","Task","Task","Task","Task",
    "Project","Project","Project","Portfolio"
  ),
  stringsAsFactors = FALSE
)

links <- data.frame(
  source = c(
    "SC","SC",
    "A","A","A","B","B","B","C","C","C",
    "D","E","F","G","H","I","J","K","L",
    "M","N","O","P","Q","R","S","T","U",
    "V","W","X"
  ),
  target = c(
    "A","B",
    "D","G","J","E","H","K","F","I","L",
    "M","N","O","P","Q","R","S","T","U",
    "V","V","V","W","W","W","X","X","X",
    "Y","Y","Y"
  ),
  value = rep(1, 32)
)

distributions <- list(
  SC = list(type = "discrete", values = c(1, 0), probs = c(0.7, 0.3)),
```

```
A  = list(type = "conditional", condition = "SC",
  true_dist  = list(type = "discrete", values = c(1, 0), probs = c(0.95, 0.05)),
  false_dist = list(type = "discrete", values = c(1, 0), probs = c(0.40, 0.60))),
B  = list(type = "conditional", condition = "SC",
  true_dist  = list(type = "discrete", values = c(1, 0), probs = c(0.85, 0.15)),
  false_dist = list(type = "discrete", values = c(1, 0), probs = c(0.25, 0.75))),
C  = list(type = "discrete", values = c(1, 0), probs = c(0.6, 0.4)),
D  = list(type = "conditional", condition = "A",
  true_dist  = list(type = "normal", mean = 50000, sd = 8000),
  false_dist = list(type = "normal", mean = 30000, sd = 5000)),
G  = list(type = "conditional", condition = "A",
  true_dist  = list(type = "normal", mean = 40000, sd = 6000),
  false_dist = list(type = "normal", mean = 25000, sd = 4000)),
J  = list(type = "conditional", condition = "A",
  true_dist  = list(type = "normal", mean = 65000, sd = 10000),
  false_dist = list(type = "normal", mean = 40000, sd = 6000)),
E  = list(type = "conditional", condition = "B",
  true_dist  = list(type = "normal", mean = 80000, sd = 12000),
  false_dist = list(type = "normal", mean = 50000, sd = 8000)),
H  = list(type = "conditional", condition = "B",
  true_dist  = list(type = "normal", mean = 50000, sd = 8000),
  false_dist = list(type = "normal", mean = 30000, sd = 5000)),
K  = list(type = "conditional", condition = "B",
  true_dist  = list(type = "normal", mean = 100000, sd = 15000),
  false_dist = list(type = "normal", mean = 60000, sd = 10000)),
F  = list(type = "conditional", condition = "C",
  true_dist  = list(type = "normal", mean = 35000, sd = 6000),
  false_dist = list(type = "normal", mean = 20000, sd = 4000)),
I  = list(type = "conditional", condition = "C",
  true_dist  = list(type = "normal", mean = 25000, sd = 4000),
  false_dist = list(type = "normal", mean = 15000, sd = 3000)),
L  = list(type = "conditional", condition = "C",
  true_dist  = list(type = "normal", mean = 40000, sd = 6000),
  false_dist = list(type = "normal", mean = 25000, sd = 4000)),
M = list(type = "aggregate", nodes = "D"),
N = list(type = "aggregate", nodes = "E"),
O = list(type = "aggregate", nodes = "F"),
P = list(type = "aggregate", nodes = "G"),
Q = list(type = "aggregate", nodes = "H"),
R = list(type = "aggregate", nodes = "I"),
S = list(type = "aggregate", nodes = "J"),
T = list(type = "aggregate", nodes = "K"),
U = list(type = "aggregate", nodes = "L"),
V = list(type = "aggregate", nodes = c("M","N","O")),
W = list(type = "aggregate", nodes = c("P","Q","R")),
X = list(type = "aggregate", nodes = c("S","T","U")),
Y = list(type = "aggregate", nodes = c("V","W","X"))
```

```
)

graph <- prob_net(nodes, links, distributions = distributions)
```

10.4. Observational Distribution

The observational distribution $P(Y)$ reflects the full uncertainty of all three shared risks.

```
sim_results <- prob_net_sim(graph, num_samples = 1000)
```

```
hist(sim_results$Y, breaks = 50,
     main = expression("Observational distribution " * italic(P) * "(Y)"),
     xlab = "Portfolio Cost ($)", col = "skyblue", border = "white")
```

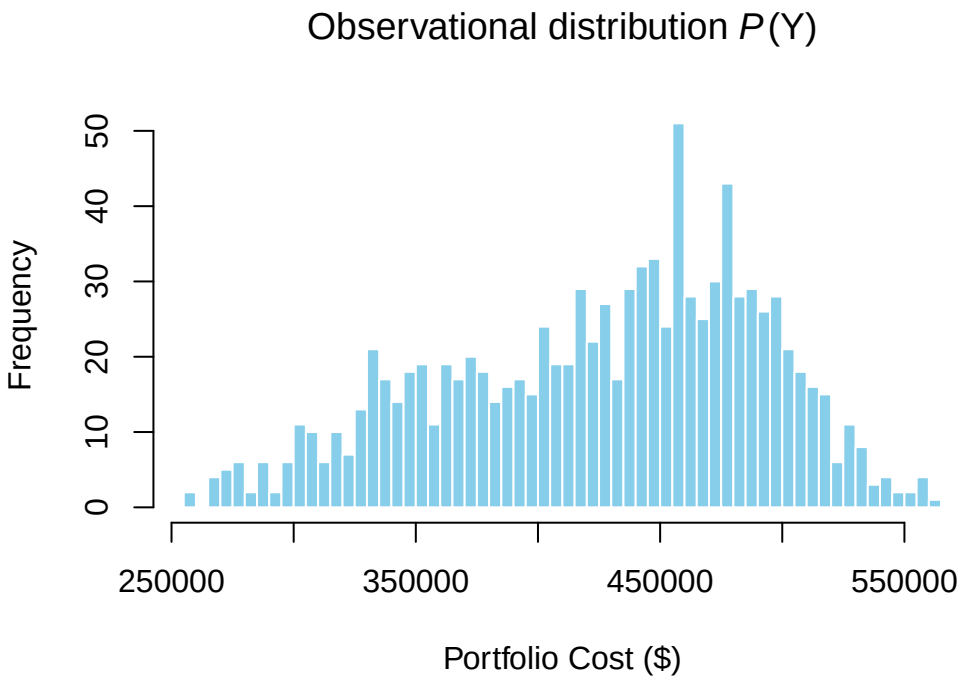


Figure 10.1.: Observational portfolio cost distribution P(Y). Heavy tails reflect compounded shared-risk uncertainty.

10.4.1. Task-Cost Correlation Matrix

The correlation matrix reveals which tasks are coupled through shared risks.

```
task_costs <- sim_results[, c("M","N","O","P","Q","R","S","T","U")]
cor_matrix <- cor(task_costs)
corrplot(cor_matrix,
         method = "color", type = "full",
         addCoef.col = "black", number.cex = 0.7,
         tl.col = "black", tl.srt = 45,
         col = colorRampPalette(c("white", "steelblue"))(100),
         mar = c(0, 0, 1, 0))
```

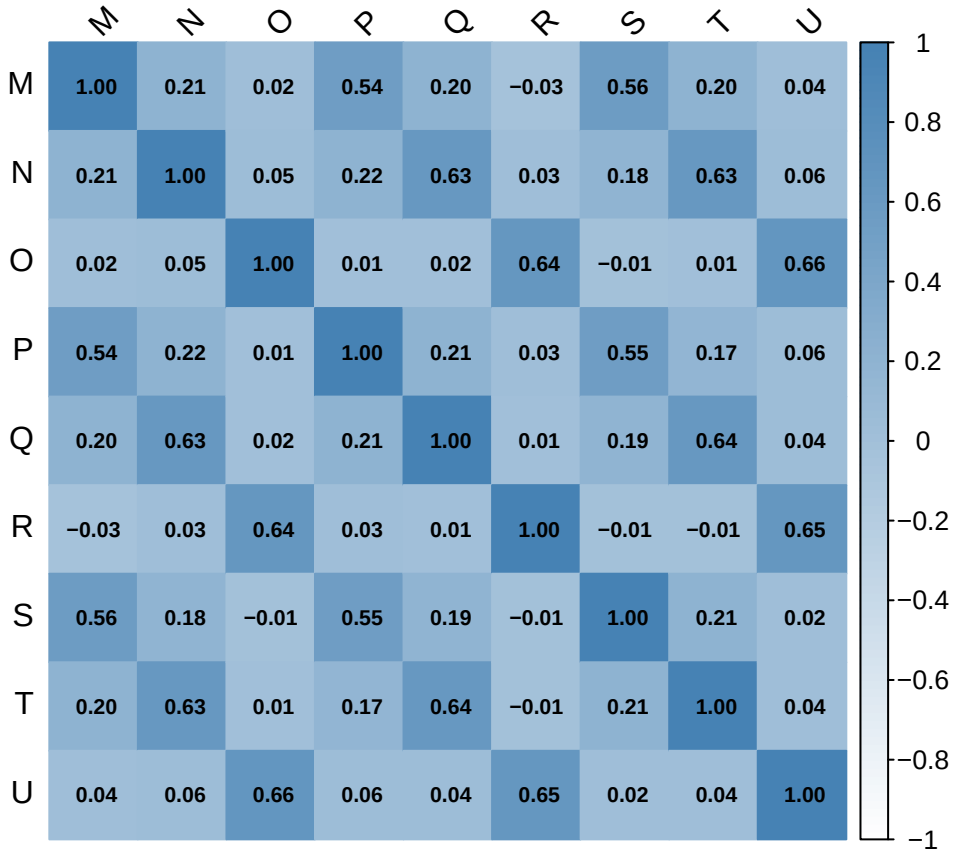


Figure 10.2.: Task-cost correlation matrix. Tasks sharing the same enterprise risk (Labor, Materials, Equipment) are strongly correlated; tasks driven by different risks are near zero.

10.5. Seeing vs. Doing at the Risk Level

This is where the real insight lives. **Seeing** $A = 1$ conditions on the observation in the original graph: evidence propagates upstream as well as downstream. **Doing** $\text{do}(A = 1)$ performs graph surgery, severing SC→A and fixing $A = 1$. The two operations produce different results because of the shared root cause SC.

```
seeing_results <- prob_net_learn(graph, observations = list(A = 1), num_samples = 1000)

do_A1_graph <- prob_net_update(graph,
  remove_links         = data.frame(source = "SC", target = "A", stringsAsFactors = FALSE),
  update_distributions = list(A = list(type = "discrete", values = c(1, 0), probs = c(1, 0)))
)
do_A1_results <- prob_net_sim(do_A1_graph, num_samples = 1000)
```

```
h1     <- hist(sim_results$Y,    breaks = 50, plot = FALSE)
h2     <- hist(seeing_results$Y, breaks = 50, plot = FALSE)
h_do   <- hist(do_A1_results$Y,  breaks = 50, plot = FALSE)

plot(h1, main = "P(Y) vs. Seeing vs. Doing: Labor Shortage",
     xlab = "Portfolio Cost ($)", col = "skyblue", border = "white",
     ylim = c(0, max(h1$counts, h2$counts, h_do$counts)))
plot(h_do, col = rgb(1, 0.5, 0, 0.5), border = "white", add = TRUE)
plot(h2,   col = rgb(0, 0, 1, 0.4),   border = "white", add = TRUE)
legend("topright",
  legend = c("P(Y): prior",
             "P(Y | do(A=1)): doing",
             "P(Y | A=1): seeing"),
  fill = c("skyblue", rgb(1, 0.5, 0, 0.5), rgb(0, 0, 1, 0.4)), bty = "n")
```

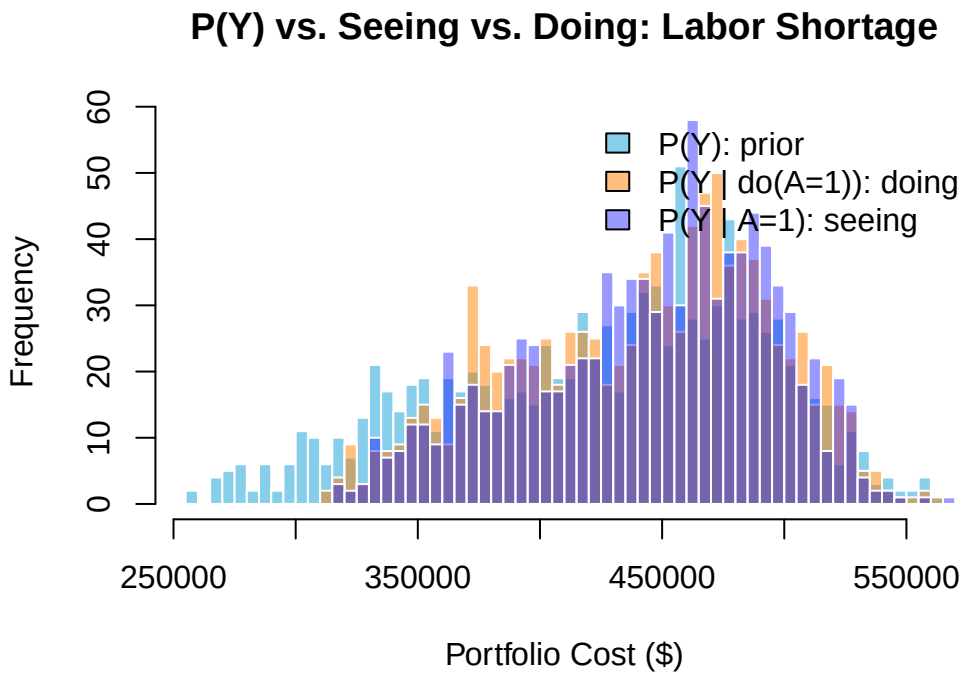


Figure 10.3.: Prior, doing, and seeing distributions for portfolio cost. Seeing shifts further right than doing because it propagates through the shared root cause SC.

The key distinction: *Seeing* a Labor Shortage raises our belief that a Supply Chain Disruption is underway, which in turn raises the probability of a Material Price Spike, a side-effect that propagates through SC. *Doing* $\text{do}(A = 1)$ severs SC→A, so SC stays at its prior and material costs are unaffected. The gap between the two distributions is the operational signature of the shared root cause.

10.6. Enterprise vs. Project-Scoped Intervention

Two intervention strategies can reduce labor-risk exposure:

1. **Project-scoped**: Secure a dedicated crew for Building Renovation only ($\text{do}(J \to \text{baseline})$)
2. **Enterprise prevention**: Eliminate the Labor Shortage entirely ($\text{do}(A = 0)$)

```
do_J_graph <- prob_net_update(graph,
  remove_links         = data.frame(source = "A", target = "J", stringsAsFactors = FALSE),
  update_distributions = list(J = list(type = "normal", mean = 40000, sd = 6000))
)
do_J_results <- prob_net_sim(do_J_graph, num_samples = 1000)

do_A0_graph <- prob_net_update(graph,
  remove_links         = data.frame(source = "SC", target = "A", stringsAsFactors = FALSE),
  update_distributions = list(A = list(type = "discrete", values = c(1, 0), probs = c(0, 1)))
)
do_A0_results <- prob_net_sim(do_A0_graph, num_samples = 1000)
```

```
h3 <- hist(do_J_results$Y,  breaks = 50, plot = FALSE)
h4 <- hist(do_A0_results$Y, breaks = 50, plot = FALSE)

plot(h1, main = "Project-scoped vs. Enterprise-level Labor Mitigation",
     xlab = "Portfolio Cost ($)", col = "skyblue", border = "white",
     ylim = c(0, max(h1$counts, h3$counts, h4$counts)))
plot(h3, col = rgb(0, 0.6, 0, 0.4),   border = "white", add = TRUE)
plot(h4, col = rgb(0.8, 0.4, 0, 0.5), border = "white", add = TRUE)
legend("topright",
  legend = c("P(Y): observational",
             "do(J → baseline): Project 3 insulated",
             "do(A = 0): enterprise prevention"),
  fill = c("skyblue", rgb(0, 0.6, 0, 0.4), rgb(0.8, 0.4, 0, 0.5)), bty = "n")
```

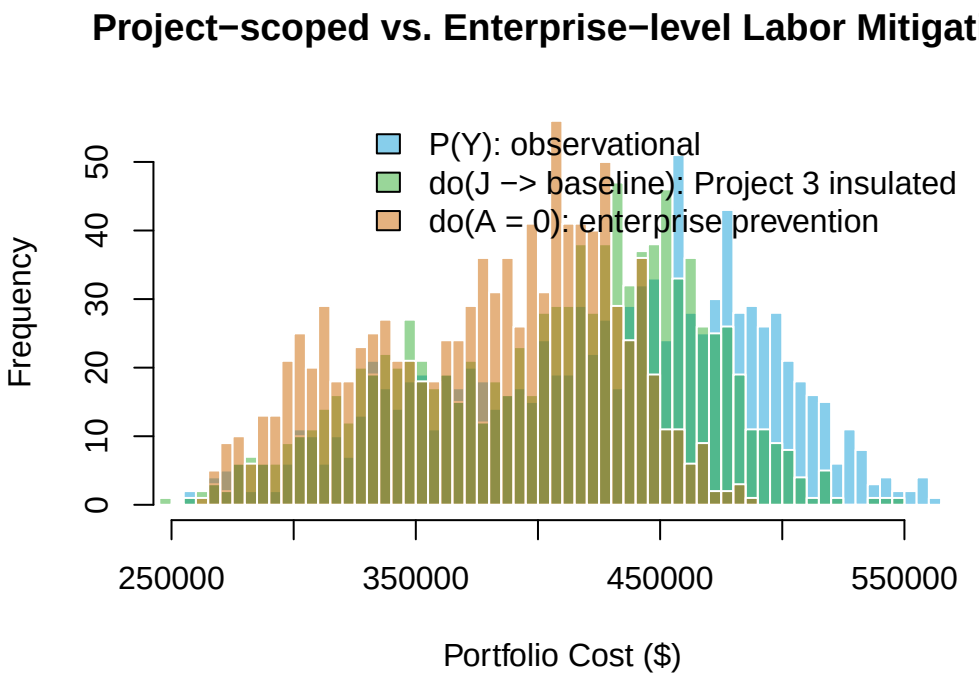


Figure 10.4.: Project-scoped vs. enterprise-level labor risk mitigation. Enterprise prevention eliminates exposure across all three projects.

10.7. Risk Importance Ranking

Which shared risk contributes most to portfolio cost variance? Run three enterprise-prevention interventions and measure the variance reduction each produces.

```
prevent_risk <- function(graph, parent_node, risk_node) {
  remove_df <- if (!is.null(parent_node))
    data.frame(source = parent_node, target = risk_node, stringsAsFactors = FALSE)
  else
    data.frame(source = character(0), target = character(0), stringsAsFactors = FALSE)
  prob_net_update(graph,
    remove_links = remove_df,
    update_distributions = setNames(
      list(list(type = "discrete", values = c(1, 0), probs = c(0, 1))),
      risk_node
    )
  )
}

g_A0 <- prevent_risk(graph, "SC", "A")
g_B0 <- prevent_risk(graph, "SC", "B")
```

```r
g_C0 <- prevent_risk(graph, NULL, "C")

r_A0 <- prob_net_sim(g_A0, num_samples = 1000)
r_B0 <- prob_net_sim(g_B0, num_samples = 1000)
r_C0 <- prob_net_sim(g_C0, num_samples = 1000)

var_base  <- var(sim_results$Y)
importance <- c(
  "Labor\nShortage (A)"       = var_base - var(r_A0$Y),
  "Material\nPrice Spike (B)" = var_base - var(r_B0$Y),
  "Weather\nDelay (C)"        = var_base - var(r_C0$Y)
)

barplot(importance,
  main   = "Risk Importance: Variance Eliminated by do(risk = 0)",
  ylab   = expression("Variance reduction in Y ($"^2 * ")"),
  col    = c("steelblue", "coral", "seagreen"),
  border = "white", las = 1
)
```

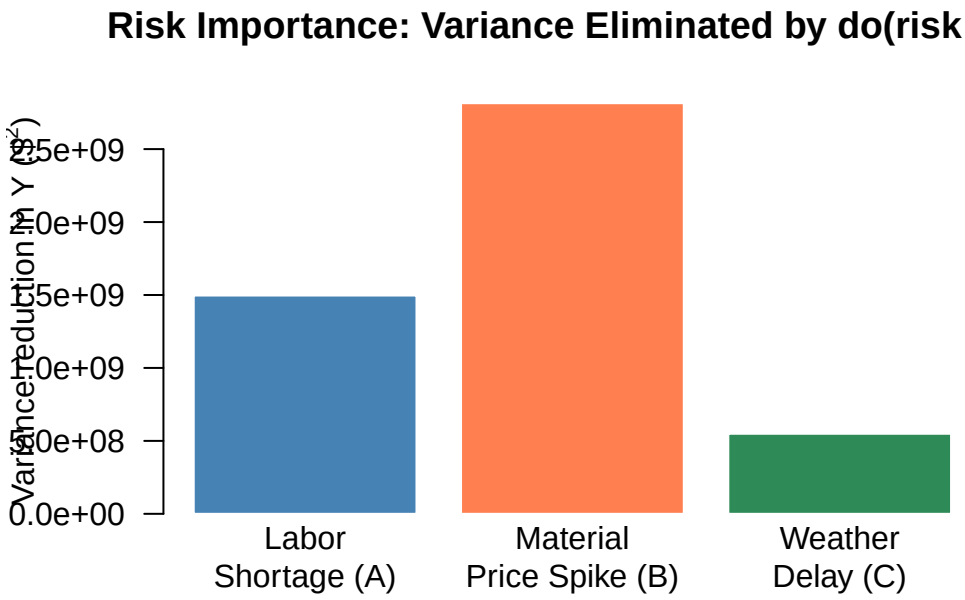


Figure 10.5.: Risk importance: portfolio variance eliminated by preventing each enterprise risk. Taller bars = higher priority for mitigation.

The bar heights rank the three enterprise risks by their contribution to portfolio cost variance, a principled basis for prioritizing mitigation investments.

10.8. Summary Table

```
make_stats <- function(x) c(round(mean(x)/1000, 1), round(sd(x)/1000, 1),
                            round(quantile(x, 0.95)/1000, 1))
stats_mat <- rbind(
  make_stats(sim_results$Y),
  make_stats(seeing_results$Y),
  make_stats(do_A1_results$Y),
  make_stats(do_J_results$Y),
  make_stats(do_A0_results$Y)
)
stats_df <- as.data.frame(stats_mat)
colnames(stats_df) <- c("Mean ($000)", "SD ($000)", "95th Pct ($000)")
rownames(stats_df) <- c(
  "Observational P(Y)",
  "Seeing P(Y | A = 1)",
  "Doing do(A = 1)",
  "Doing do(J → baseline) [Project 3]",
  "Doing do(A = 0) [Enterprise prevention]"
)
knitr::kable(stats_df, caption = "Portfolio cost statistics across all scenarios.")
```

Table 10.6.: Portfolio cost statistics across all scenarios.

	Mean ($000)	SD ($000)	95th Pct ($000)
Observational P(Y)	425.9	65.0	516.7
Seeing P(Y | A = 1)	444.3	50.3	515.4
Doing do(A = 1)	441.0	53.0	518.8
Doing do(J → baseline) [Project 3]	406.6	58.1	485.8
Doing do(A = 0) [Enterprise prevention]	379.0	50.3	448.9

10.9. Summary

Key Takeaways

- Shared enterprise risks create **positive correlation across all projects** that share those resources, so portfolio variance is always larger than the naive sum of individual project variances.
- The **task-cost correlation matrix** reveals the block structure of risk sharing: tasks driven by the same enterprise risk cluster together, while tasks driven by different risks are near-zero.
- **Seeing** conditions on evidence in the original graph; it propagates upstream through shared root causes, raising the probability of sibling risks as a side-effect.

- **Doing** (graph surgery) severs incoming edges of the intervened node; it changes only what the intervention directly controls, without raising posteriors about shared root causes.
- Enterprise-level risk prevention ($\text{do}(A = 0)$) reduces portfolio exposure across all projects simultaneously; project-scoped insulation ($\text{do}(J \to \text{baseline})$) reduces exposure for only one.
- **Risk importance ranking** by variance reduction from $\text{do}(\text{risk} = 0)$ gives a principled basis for prioritizing mitigation budgets.

10.10. Exercises

1. **Seeing vs. doing.** In your own words, explain why $P(Y \mid A = 1) \neq P(Y \mid \text{do}(A = 1))$ in this model. What structural feature of the network causes the difference? What would have to change in the graph for the two to be equal?

2. **Risk importance.** In the risk importance ranking, which enterprise risk had the largest effect on portfolio variance? What is it about that risk's position in the network that makes it particularly impactful?

3. **Two risks confirmed.** Run `prob_net_learn(graph, observations = list(A = 1, B = 1), num_samples = 1000)`. How does the portfolio cost distribution compare to seeing only $A = 1$? Is the combined effect larger or smaller than you'd expect from adding the individual effects?

4. **Add a fourth project.** Extend the network with a fourth project (e.g., a Bridge Inspection) with three tasks and three resources (Labor-4, Materials-4, Equipment-4), driven by the same three enterprise risks A, B, C. Add it to the portfolio node Y. How does total portfolio variance change? Does the risk importance ranking shift?

5. **Causal graph design.** In the current model, Supply Chain Disruption (SC) drives both Labor Shortage (A) and Material Price Spike (B). Suppose you learn that, in practice, SC also affects Weather Delay (C) through disrupted logistics. Add an SC $\to$ C edge and update the C distribution to be conditional on SC. How does this change the see-versus-do analysis for C?

11. Write Once, Trust Always: Reproducible Risk Analysis with Quarto

> *"The first principle is that you must not fool yourself — and you are the easiest person to fool."* — Richard Feynman

You've just run a Monte Carlo simulation, computed a P80 schedule, and printed the contingency reserve for your project. Now imagine sending that result to a colleague, then trying to recreate it six months later. The spreadsheet is gone. The file is named "Final_v3_REVISED_use_this_one.xlsx." Three cells have broken formulas and nobody remembers which inputs were authoritative.

This is the reproducibility gap in project risk analysis, and it is more common than anyone likes to admit.

Quarto closes that gap. It lets you write your analysis and your prose in the same document, so that when the document renders, R executes the code and embeds the actual output: the numbers, the plots, the tables. Not a screenshot. Not a copy-paste. The live result, freshly computed, every time.

This chapter teaches you how to build reproducible risk reports with Quarto and the `PRA` package. By the end, you will have a workflow you can hand to a colleague, run on a new project, or re-execute in two years and get the same answer.

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain why reproducibility is a professional standard in quantitative risk analysis
2. Create a Quarto document that embeds live `PRA` code and narrative prose
3. Use parameterized reports to run the same analysis across multiple project scenarios
4. Version-control a risk analysis workflow with Git
5. Describe how this book itself is built using the same Quarto framework

11.1. The Spreadsheet Problem

Spreadsheets are the default tool for project risk analysis. They are flexible, familiar, and require no installation. They are also, quietly, one of the biggest threats to analytical credibility.

Here is a realistic timeline:

1. A risk analyst builds a Monte Carlo model in Excel. It works. The P90 cost is $4.2M.
2. The file is emailed, renamed, and forwarded. Three versions exist by Tuesday.

3. Six months later, the project is audited. The analyst is on a new project. Nobody can open the file without `#REF!` errors.
4. The P90 estimate can no longer be verified. The basis of the contingency reserve is, in practice, lost.

This is not a spreadsheet problem specifically, but a **reproducibility problem**. The analysis was never in a form that could be reliably re-executed.

> **Reproducible vs. Repeatable**
>
> **Reproducible** means the same analyst, on the same machine, can re-run the analysis and get the same result.
> **Repeatable** means a *different* analyst, on a *different* machine, can run the analysis and get the same result, given the same inputs.
> A Quarto document checked into version control with a declared package environment achieves both. A spreadsheet achieves neither reliably.

The alternative is to make your analysis a **document that also runs**. That is what Quarto does.

11.2. What is Quarto?

Quarto is an open-source publishing system that combines prose and executable code in a single `.qmd` file. You write in Markdown. You embed code chunks in R, Python, Julia, or Observable. When you render the document, Quarto runs the code and weaves the results into the output: HTML, PDF, Word, or a full website or book.

> **Quarto vs. R Markdown**
>
> If you have used R Markdown (`.Rmd`), Quarto will feel immediately familiar. Quarto is the next-generation version: it supports multiple languages, has a cleaner YAML syntax, and produces higher-quality output formats out of the box.
> If you have not used either: Quarto is where to start. Install it once from quarto.org and it works with RStudio, VS Code, or any text editor.

A minimal Quarto document looks like this:

```
---
title: "Bridge Rehabilitation: Schedule Risk Analysis"
author: "Your Name"
date: today
format: html
---
```

```
## Overview

This report summarizes the Monte Carlo schedule simulation for the
bridge rehabilitation project.
```

````
```{r}
library(PRA)

task_dists <- list(
  list(type = "normal",     mean = 30, sd = 4),
  list(type = "triangular", a = 12, b = 20, c = 30),
  list(type = "uniform",    min = 10, max = 25)
)

results <- mcs(10000, task_dists)
print(results)
```
````

Run `quarto render report.qmd` in your terminal and you have a finished HTML report, numbers and all, generated from a single source file.

11.3. Your First Risk Report

Let us build one from scratch.

11.3.1. Step 1: Install Quarto

Download and install Quarto from quarto.org. If you use RStudio, version 2022.07 or later includes Quarto support out of the box.

11.3.2. Step 2: Create the Document

Create a new file called `risk_report.qmd` and add this header:

```
---
title: "Project Schedule Risk Analysis"
author: "Risk Analyst"
date: today
format:
  html:
    toc: true
    code-fold: false
execute:
  warning: false
  message: false
---
```

11.3.3. Step 3: Write the Analysis

Below the YAML header, write your prose and embed your R code:

````
## Background

This analysis quantifies schedule uncertainty for a three-phase
infrastructure project using Monte Carlo simulation.

```{r}
library(PRA)

task_dists <- list(
  list(type = "normal",     mean = 45, sd = 6),          # Design
  list(type = "triangular", a = 20, b = 30, c = 50),     # Permits
  list(type = "uniform",    min = 50, max = 80)           # Construction
)

set.seed(42)
results <- mcs(10000, task_dists)
print(results)
```

## Contingency Reserve

At the P80 confidence level, the recommended contingency is:

```{r}
contingency(results, phigh = 0.80, pbase = 0.50)
```
````

11.3.4. Step 4: Render

From your terminal:

```
quarto render risk_report.qmd
```

Or from RStudio, click **Render**. Quarto runs both code chunks, embeds the output inline, and writes `risk_report.html`. Open it in any browser.

Here is what the embedded output looks like when you run those chunks live:

```
library(PRA)

task_dists <- list(
  list(type = "normal",     mean = 45, sd = 6),
  list(type = "triangular", a = 20, b = 30, c = 50),
```

```
  list(type = "uniform",    min = 50, max = 80)
)

set.seed(42)
results <- mcs(10000, task_dists)
print(results)
```

```
Monte Carlo Simulation Results:
Simulations: 10,000
Total Mean: 143.3373
Total Variance: 152.0998
Total Standard Deviation: 12.33288
Percentiles:
      5%      50%      95%
123.0638 143.1631 163.5249
```

```
contingency(results, phigh = 0.80, pbase = 0.50)
```

```
     80%
10.80443
```

That output was not typed by hand. It was computed when this book was built.

11.4. Parameterized Reports

So far the analysis is fixed to one project. Real workflows require running the same model against multiple scenarios, with different task durations, different confidence targets, or different projects entirely.

Quarto's `params:` block makes this straightforward.

11.4.1. Defining Parameters

Add a `params:` section to your YAML header:

```
---
title: "Schedule Risk Analysis"
format: html
params:
  mean_design:    45
  mean_permits:   30
  mean_construct: 60
  confidence:     0.80
---
```

11.4.2. Using Parameters in Code

Inside any code chunk, access parameters via `params$`:

````
```{r}
library(PRA)

task_dists <- list(
  list(type = "normal",     mean = params$mean_design,   sd = 6),
  list(type = "normal",     mean = params$mean_permits,  sd = 4),
  list(type = "normal",     mean = params$mean_construct, sd = 8)
)

set.seed(42)
results <- mcs(10000, task_dists)
contingency(results, phigh = params$confidence, pbase = 0.50)
```
````

11.4.3. Rendering Multiple Scenarios

You can render the same `.qmd` with different parameter values from the command line:

```
# Optimistic scenario
quarto render risk_report.qmd \
  -P mean_design:35 -P mean_permits:22 -P mean_construct:50 \
  --output optimistic.html

# Conservative scenario
quarto render risk_report.qmd \
  -P mean_design:55 -P mean_permits:40 -P mean_construct:75 \
  --output conservative.html
```

Two reports, one template, zero copy-paste. The same audit trail applies to both.

> **Parametric Reporting in Practice**
>
> A common workflow on large programs is to maintain one `risk_template.qmd` and a small data file (CSV or JSON) that drives scenario generation. A short R script loops over the data file and calls `quarto render` once per row. You end up with one authoritative report per project phase or scenario, all traceable to the same model.

11.5. Version Control with Git

A reproducible analysis that lives only on your laptop is not fully reproducible, just one hard drive failure away from being lost. The second half of the reproducibility story is **version control**.

11.5.1. What to Commit

The rule of thumb for a Quarto risk analysis:

File	Commit?	Reason
`risk_report.qmd`	**Yes**	This is the source of truth
`data/inputs.csv`	**Yes**	Inputs must be versioned with the analysis
`risk_report.html`	No	Derived output; regenerate on demand
`risk_report_files/`	No	Auto-generated by Quarto
`.Rprofile`, `renv.lock`	**Yes**	Locks package versions for repeatability

The `.qmd` file and its inputs are the authoritative record. Everything else is a by-product.

11.5.2. A Minimal Workflow

```
git init
echo "*.html" >> .gitignore
echo "*_files/" >> .gitignore

git add risk_report.qmd data/inputs.csv renv.lock
git commit -m "Initial Monte Carlo schedule analysis, P80 = 142 days"
```

When you update the analysis, adding a task or revising input data, commit again:

```
git add -A
git commit -m "Add permits phase; P80 increases to 158 days"
```

The git log becomes an audit trail. Anyone can check out any prior commit and re-render the analysis to verify a historical result.

> **Package Environments with renv**
>
> Reproducibility also requires locking the versions of R packages your analysis depends on. The `renv` package does this automatically:
>
> ```
> install.packages("renv")
> renv::init()   # creates renv.lock - commit this file
> renv::restore() # anyone can restore the exact same packages later
> ```
>
> Without `renv`, an analysis that ran on `PRA` version 0.3.0 might silently produce different results on version 0.4.0 if a function changes behavior.

11.6. This Book as a Living Example

This book is itself a Quarto project built on exactly the framework described in this chapter.

Every code block you have read throughout this book, including the `mcs()` call in Chapter 2, the `evm` calculations in Chapter 5, and the Bayesian updates in Chapter 6, was executed when the book was built. There is no "trust us, this is what you'd see." The numbers are real, freshly computed from the current version of the `PRA` package.

The book's source is a directory of `.qmd` files coordinated by a single `_quarto.yml`:

```
project:
  type: book

book:
  title: "Project Risk Analysis: A Practical Guide"
  chapters:
    - index.qmd
    - intro.qmd
    - ch-mcs.qmd
    - ch-smm.qmd
    # ... one file per chapter ...
    - ch-quarto.qmd   # this chapter
    - ch-agent.qmd
```

To reproduce the entire book from scratch:

```
git clone https://github.com/paulgovan/PRA
cd PRA
quarto render book/
```

That single command runs every code chunk in every chapter, in a clean R session, and writes the full HTML site, PDF, and EPUB to `book/_book/`. If any code breaks, because a function changed behavior or a dependency was removed, the render fails loudly. The book cannot silently go out of date.

> **What "Live Book" Means for You**
>
> Because the book is reproducible, you can:
>
> - **Verify any result.** Copy a code chunk, change a parameter, and see immediately how the output shifts.
> - **Trust the numbers.** They were not typed by hand and are not screenshots from an old run.
> - **Adapt the structure.** The chapter files are plain Markdown with embedded R. Clone the repo, swap in your project's data, and render your own custom risk analysis book.
>
> The same `.qmd` + `PRA` + `quarto render` workflow that produced this book can produce your project's quarterly risk report.

11.7. Exercises

1. **Create a minimal report.** Write a `risk_report.qmd` with a YAML header, a one-paragraph background section, and a single code chunk that runs `mcs()` on three tasks of your choice. Render it to HTML and verify the output appears inline.
2. **Add parameterization.** Extend the report from Exercise 1 with a `params:` block. Define `confidence` as a parameter (default `0.80`). Use `params$confidence` in the `contingency()` call. Re-render with `-P confidence:0.90` from the terminal and confirm the contingency increases.
3. **Render to PDF.** Add `pdf: default` to your `format:` block and re-render. Open the PDF. Note any differences in how the plots and tables appear. Try the `geometry: margin=1in` option to control margins.
4. **Inspect this book's source.** Clone the `PRA` repository from GitHub and open `book/ch-mcs.qmd` in a text editor. Identify: (a) the YAML header, (b) the learning objectives callout, (c) the code chunk that calls `mcs()`, and (d) the exercises section. Then run `quarto render book/ch-mcs.qmd` and compare the rendered output to the source.
5. **Version-control your analysis.** Initialize a Git repository in the folder containing your `risk_report.qmd`. Commit the `.qmd` file. Make a change (add a task, adjust a parameter) and make a second commit. Run `git log` and confirm you have a two-commit history that documents what changed and when.

12. Your AI Co-Pilot: Agentic Risk Analysis

"The best way to predict the future is to invent it." — Alan Kay

You've made it through eight chapters of quantitative risk analysis. You know Monte Carlo, SMM, EVM, Bayesian inference, learning curves, design structure matrices, and probabilistic networks. Now imagine not having to remember the syntax for all of them.

Imagine asking an AI assistant to "run Monte Carlo for three tasks with Normal(10,2), Triangular(5,10,15), Uniform(8,12)" and getting back a P95 date, a contingency reserve, and a tornado chart. That's the promise of the `PRA` agentic layer: the core analytical functions in this book are exposed as tools that an AI agent can call directly, guided by a set of skills that tell it *when* and *how* to use each one.

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain how PRA exposes its functions as tools over the Model Context Protocol (MCP)
2. Start the PRA MCP server and register it with Claude Code or Claude Desktop
3. Describe what Agent Skills are and how they guide tool use
4. Map a natural-language request to the underlying PRA tool and its arguments

12.1. How PRA Talks to AI Agents

Rather than embedding a language model inside the package, PRA takes a simpler and more portable approach: it publishes its analytical functions as **tools** that any capable AI agent can call. Two pieces make this work:

1. **An MCP server** (`pra_mcp_server()`) that advertises each PRA function — Monte Carlo simulation, EVM, Bayesian risk, learning curves, DSM, and the rest — as a callable tool over the Model Context Protocol.
2. **Agent Skills** — bundled `SKILL.md` files that document, in plain language, when to use each method, which tool to call, the argument format, and how to read the result.

The agent (Claude Code, Claude Desktop, or any MCP-compatible client) supplies the intelligence; PRA supplies the trustworthy, validated computation. Because the tools are the same functions you've used throughout the book, the numbers are identical whether you call `mcs()` yourself or ask an agent to run it.

12.2. Prerequisites

The MCP server relies on a few optional packages:

```
install.packages(c("ellmer", "mcptools", "jsonlite"))
```

The analytical functions themselves need none of these — they are required only to serve the tools to an agent.

12.3. The MCP Server

12.3.1. Starting the server

```
library(PRA)
pra_mcp_server()
```

The server communicates over stdio, the standard transport for local MCP servers, and reuses the tool definitions from `pra_tools()` so any update to a tool is reflected automatically.

12.3.2. Connecting from Claude Code

```
claude mcp add -s project pra -- Rscript -e "PRA::pra_mcp_server()"
```

Once registered, Claude Code can call PRA tools in any conversation:

> "Run a Monte Carlo simulation for three tasks: Task A normal(10, 2), Task B triangular(5, 10, 15), Task C uniform(8, 12). What is the contingency reserve at the 90th percentile?"

12.3.3. Connecting from Claude Desktop

Add the following to your Claude Desktop config file:

```
{
  "mcpServers": {
    "pra": {
      "command": "Rscript",
      "args": ["-e", "PRA::pra_mcp_server()"]
    }
  }
}
```

12.4. Agent Skills

An agent works best when it knows not just *what* tools exist but *when* to reach for each one. PRA ships a set of Agent Skills for exactly this. Each is a `SKILL.md` file with structured guidance — when to use the method, which tool to call, the JSON argument format, and how to interpret the result.

Find the installed skills with:

```
list.files(system.file("skills", package = "PRA"))
```

Skill	Covers	Primary tools
`pra-overview`	Routing / entry point	(directs to the others)
`pra-monte-carlo`	Schedule/cost uncertainty	`mcs_tool`, `smm_tool`
`pra-sensitivity-contingency`	Reserves & risk drivers	`contingency_tool`, `sensitivity_tool`
`pra-earned-value`	Performance & forecasting	`evm_analysis_tool`
`pra-bayesian-risk`	Risk probability & cost	`risk_prob_tool`, `risk_post_prob_tool`, `cost_pdf_tool`, `cost_post_pdf_tool`
`pra-learning-curves`	S-curve fitting/forecasts	`fit_and_predict_sigmoidal_tool`
`pra-dsm`	Task dependency structure	`parent_dsm_tool`, `grandparent_dsm_tool`

12.5. From Request to Tool Call

When you ask an agent to run an analysis, it selects a tool and fills in the arguments. All array and matrix arguments are passed as JSON strings. For example, the request "simulate three tasks — Normal(10,2), Triangular(5,10,15), Uniform(8,12), 10,000 runs" maps to `mcs_tool` with a `task_dists_json` of:

```
[{"type":"normal","mean":10,"sd":2},
 {"type":"triangular","a":5,"b":10,"c":15},
 {"type":"uniform","min":8,"max":12}]
```

Under the hood, that is exactly the `mcs()` call you already know:

```
library(PRA)
set.seed(42)
task_distributions <- list(
  list(type = "normal",     mean = 10, sd = 2),
  list(type = "triangular", a = 5, b = 10, c = 15),
```

```
  list(type = "uniform",    min = 8, max = 12)
)
result <- mcs(10000, task_distributions)
```

```
hist(result$total_distribution,
  freq = FALSE, breaks = 50,
  main = "Monte Carlo Simulation Results",
  xlab = "Total Project Duration/Cost",
  col  = "#18bc9c80", border = "white"
)
curve(dnorm(x, mean = result$total_mean, sd = result$total_sd),
  add = TRUE, col = "#2c3e50", lwd = 2
)
abline(
  v   = quantile(result$total_distribution, c(0.50, 0.95)),
  col = c("#3498db", "#e74c3c"), lty = 2, lwd = 1.5
)
legend("topright",
  legend = c("Normal fit", "P50", "P95"),
  col    = c("#2c3e50", "#3498db", "#e74c3c"),
  lty    = c(1, 2, 2), lwd = c(2, 1.5, 1.5),
  cex    = 0.8, bg = "white"
)
```

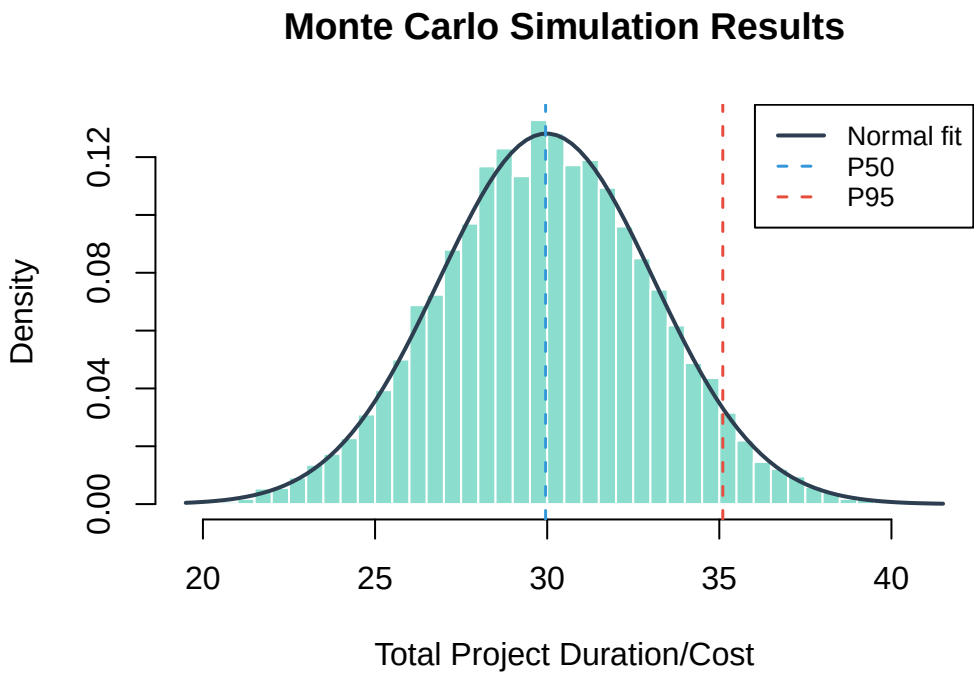


Figure 12.1.: Monte Carlo results. The same output an agent receives when it calls `mcs_tool`.

A typical agent workflow chains tools the same way you would chain functions: `mcs_tool` to simulate, then `contingency_tool` for the reserve and `sensitivity_tool` for the risk drivers. The `pra-overview` skill lists these common workflows so the agent knows what to do next.

12.6. Tool Reference

Tool	Description
`mcs_tool`	Monte Carlo simulation with task distributions
`smm_tool`	Second Moment Method (analytical estimate)
`contingency_tool`	Contingency reserve from the last MCS
`sensitivity_tool`	Variance contribution per task
`evm_analysis_tool`	Full Earned Value Management analysis
`risk_prob_tool`	Bayesian prior risk probability
`risk_post_prob_tool`	Bayesian posterior risk after observations
`cost_pdf_tool` / `cost_post_pdf_tool`	Prior / posterior cost distributions
`fit_and_predict_sigmoidal_tool`	Sigmoidal learning curve fit and prediction
`parent_dsm_tool` / `grandparent_dsm_tool`	Design Structure Matrices

12.7. Summary

Key Takeaways

- PRA exposes its core analytical functions as **tools over MCP** via `pra_mcp_server()`; the numbers are identical to calling the functions directly. The probabilistic network functions are not currently exposed as tools.
- **Agent Skills** (`SKILL.md` files under `system.file("skills", package = "PRA")`) tell an agent when and how to use each tool, including the JSON argument format.
- The package embeds **no language model** — only the MCP server needs the optional `ellmer`, `mcptools`, and `jsonlite` packages.
- Agents chain tools just as you chain functions: `mcs_tool` → `contingency_tool` + `sensitivity_tool`.

Congratulations, you've completed the toolkit. You can now quantify project uncertainty (MCS, Chapter 2; SMM, Chapter 3), track project performance (EVM, Chapter 5), update risk estimates with new evidence (Bayesian, Chapter 6; Networks, Chapter 8), model structural dependencies (DSM, Chapter 9; Portfolio, Chapter 10), forecast learning (Sigmoidal, Chapter 7), and hand it all to an AI agent through MCP (Chapter 12). The rest is practice.

12.8. Exercises

1. **Set up the server.** Install `ellmer`, `mcptools`, and `jsonlite`, then start `pra_mcp_server()` and register it with Claude Code or Claude Desktop using the instructions above. Confirm the client lists the PRA tools.
2. **Request to arguments.** For the request "simulate two tasks, Normal(20,4) and Uniform(10,15), with 5,000 runs," write out the `mcs_tool` call and its `task_dists_json` argument by hand. Then run the equivalent `mcs()` in R and compare.
3. **Chaining.** Ask an agent (or reason it out yourself) how to produce a P80 contingency reserve and a tornado chart for a 3-task project. Which tools are called, and in what order? Which skill documents that workflow?
4. **EVM from a prompt.** Using `evm_analysis_tool`'s arguments, translate this into a tool call: BAC = \$300K, schedule = [0.2, 0.4, 0.6, 0.8, 1.0], period = 4, complete = 0.55, costs = [\$55K, \$115K, \$175K, \$240K]. Then verify with the EVM functions from Chapter 5.
5. **Skill authoring.** Read one of the bundled `SKILL.md` files, then draft a short skill for a project-specific workflow of your own (for example, a standard reserve policy). What guidance would an agent need to apply it correctly?

A. Glossary

Definitions of technical terms used throughout the book. Each entry notes the chapter where the term is first introduced or most fully explained.

A.1. Probability & Distributions

Confidence interval A range of values that, under repeated sampling, would contain the true parameter a stated percentage of the time (e.g., 95%). In the context of the Second Moment Method, a 95% CI is $\mu \pm 1.96\sigma$. See Chapter 3.

Distribution, normal A symmetric bell-shaped probability distribution fully described by its mean μ and standard deviation σ. The default assumption for task durations when uncertainty is roughly symmetric around the estimate. See Chapter 2.

Distribution, triangular A distribution defined by a minimum a, most likely (mode) b, and maximum c. Used when subject-matter experts can provide three-point estimates. Asymmetric when $b \neq (a + c)/2$. See Chapter 2.

Distribution, uniform A distribution where all values in the interval $[\min, \max]$ are equally likely. Used for tasks where there is no strong reason to prefer any duration over another within a range. See Chapter 2.

Expected value (mean) The probability-weighted average of all possible outcomes of a random variable. For a project total, it equals the sum of the individual task means. Denoted μ or $E[X]$. See Chapter 3.

P50 / P80 / P95 The 50th, 80th, and 95th percentiles of a distribution. P50 is the median, with half of outcomes falling below it. P80 and P95 are common targets for schedule and cost contingency. See Chapter 2.

Percentile The value below which a given percentage of outcomes fall. The p-th percentile is the value x such that $P(X \leq x) = p$. See Chapter 2.

Standard deviation The square root of variance; has the same units as the original variable. A measure of spread: roughly two-thirds of normally distributed outcomes fall within one standard deviation of the mean. Denoted σ. See Chapter 3.

Variance The expected squared deviation from the mean: $\sigma^2 = E[(X - \mu)^2]$. The key quantity in both the Second Moment Method and sensitivity analysis because variances add (for independent tasks). Denoted σ^2. See Chapter 3.

A.2. Monte Carlo Simulation & Sensitivity

Contingency reserve The additional budget or schedule allocated above the base estimate to cover uncertainty. Commonly set as the difference between a high percentile (P80 or P95) and the base estimate (P50). See Chapter 2.

Monte Carlo simulation (MCS) A method that estimates the distribution of a model output by drawing thousands of random samples from the input distributions and recording each result. Named after the Casino de Monte Carlo. See Chapter 2.

Sensitivity index A dimensionless number that quantifies how much a given task's variance contributes to total project variance, accounting for correlations with other tasks. Independent tasks all have index = 1; index > 1 indicates amplification through positive correlation. See Chapter 4.

Tornado chart A horizontal bar chart where tasks are sorted from highest to lowest sensitivity index (or variance contribution), producing a shape that narrows like a tornado. The standard deliverable for communicating sensitivity results to stakeholders. See Chapter 4.

Variance decomposition The process of partitioning total project variance into contributions from individual tasks. The foundation of sensitivity analysis. See Chapter 4 and Appendix B.

A.3. Second Moment Method

Central Limit Theorem (CLT) A statistical result stating that the sum of many independent random variables tends toward a normal distribution, regardless of the shape of the individual distributions, as the number of terms grows. The theoretical justification for treating total project duration as approximately normal. See Chapter 3 and Appendix B.

Second moment The second moment of a probability distribution is its mean squared plus its variance: $E[X^2] = \mu^2 + \sigma^2$. The "Second Moment Method" is named for its use of the first two moments (mean and variance) to characterise uncertainty. See Chapter 3.

Second Moment Method (SMM) An analytical technique that computes total project mean and variance from individual task means, variances, and pairwise correlations, without simulation. Fast, but assumes that the normal approximation for the total is adequate. See Chapter 3.

A.4. Earned Value Management

Actual Cost (AC) The total cost actually incurred for work performed up to the reporting period. Also called Actual Cost of Work Performed (ACWP). See Chapter 5.

Budget at Completion (BAC) The total planned budget for the project. The sum of all planned costs. See Chapter 5.

Cost Performance Index (CPI) The ratio of Earned Value to Actual Cost: CPI $= EV/AC$. A CPI below 1.0 means the project is over budget for the work accomplished. See Chapter 5.

Cost Variance (CV) The difference between Earned Value and Actual Cost: $CV = EV - AC$. Negative means over budget. See Chapter 5.

Earned Value (EV) The budgeted value of work actually completed: $EV = \text{BAC} \times \%\text{complete}$. Also called Budgeted Cost of Work Performed (BCWP). See Chapter 5.

Estimate at Completion (EAC) The forecast of total project cost at completion, computed from current performance. Three common methods: typical ($EAC = AC + (BAC - EV)/CPI$), atypical ($EAC = AC + BAC - EV$), and combined. See Chapter 5.

Estimate to Complete (ETC) The expected cost to finish the remaining work: $ETC = EAC - AC$. See Chapter 5.

Planned Value (PV) The budgeted cost for work scheduled to be done by the reporting period: $PV = \text{BAC} \times \%\text{planned}$. Also called Budgeted Cost of Work Scheduled (BCWS). See Chapter 5.

Schedule Performance Index (SPI) The ratio of Earned Value to Planned Value: SPI $= EV/PV$. An SPI below 1.0 means the project is behind schedule. See Chapter 5.

Schedule Variance (SV) The difference between Earned Value and Planned Value: $SV = EV - PV$. Negative means behind schedule. See Chapter 5.

To-Complete Performance Index (TCPI) The cost efficiency required on remaining work to meet the Budget at Completion: TCPI $= (BAC - EV)/(BAC - AC)$. A TCPI above 1.0 means the team must perform better than it has been. See Chapter 5.

Variance at Completion (VAC) The difference between the original budget and the current forecast: $VAC = BAC - EAC$. Negative means the project is forecast to overrun. See Chapter 5.

A.5. Bayesian Inference

Bayes' theorem The rule for updating probabilities given evidence: $P(H|E) = P(E|H) \cdot P(H)/P(E)$. The foundation of all Bayesian risk updating in this book. See Chapter 6 and Appendix B.

Conditional probability The probability of an event given that another event is known to have occurred: $P(A|B) = P(A \cap B)/P(B)$. See Chapter 6.

Likelihood The probability of observing the evidence given a hypothesis: $P(E|H)$. In project risk terms, the probability that a root cause would be observable if the risk event occurred (or did not occur). See Chapter 6.

Posterior probability The updated probability of a hypothesis after observing new evidence: $P(H|E)$. The output of Bayesian updating. Computed by `risk_post_prob()`. See Chapter 6.

Prior probability The probability of a hypothesis before observing new evidence, based on historical data or expert judgment. Computed from root-cause probabilities by `risk_prob()`. See Chapter 6.

Root cause An underlying condition or event that can trigger a risk event. Multiple root causes can independently contribute to the same risk. In `PRA`, root causes are modelled as independent with known probabilities. See Chapter 6.

A.6. Learning Curves

Gompertz model A sigmoidal growth model with an asymmetric S-shape: slow initial growth, rapid acceleration, then deceleration. The inflection point occurs at roughly 37% of the ceiling. Useful when early adoption is very slow. See Chapter 7.

Logistic model A symmetric sigmoidal growth model: $y = K/(1 + e^{-r(t-t_0)})$. Growth accelerates from 0 to $K/2$ then decelerates symmetrically toward the ceiling K. See Chapter 7.

Pearl model Functionally identical to the logistic model, a symmetric S-curve with the same formula. Named after Raymond Pearl who applied it to population growth. See Chapter 7.

Residual standard error (RSE) The standard deviation of the residuals (observed minus fitted values) in a regression model. Lower RSE indicates a better fit. Used to compare logistic, Gompertz, and Pearl model fits. See Chapter 7.

Sigmoidal curve An S-shaped curve that models bounded growth: starts near zero, accelerates, then levels off at a ceiling. All three learning curve models in this book (Logistic, Pearl, Gompertz) are sigmoidal. See Chapter 7.

A.7. Structural Methods

Bayesian network A directed acyclic graph (DAG) in which nodes represent random variables and edges represent conditional dependencies. Used in `PRA` to model how risk events propagate through resources and tasks to project cost. See Chapter 8.

DAG (Directed Acyclic Graph) A graph of nodes and directed edges with no cycles; you cannot return to a starting node by following edges in their directed direction. The required structure for Bayesian networks. See Chapter 8.

Design Structure Matrix (DSM) A square matrix that encodes task-to-task dependencies. In `PRA`, DSMs are derived from shared resource and risk structures, making hidden coupling between tasks visible and quantifiable. See Chapter 9.

Grandparent DSM A tasks-by-tasks matrix derived from the chain Risk → Resource → Task. Off-diagonal entries count the number of risks shared between each pair of tasks via the intermediate resource layer. Computed by `grandparent_dsm()`. See Chapter 9.

Parent DSM A tasks-by-tasks matrix derived directly from the Resource-Task matrix. Off-diagonal entry $P[j, k]$ counts the number of resources shared by tasks j and k. Computed by `parent_dsm()`. See Chapter 9.

Resource-Task matrix (S) A binary matrix with resources as rows and tasks as columns. Entry $S[i, j] = 1$ means resource i is used by task j. The primary input to `parent_dsm()`. See Chapter 9.

Risk-Resource matrix (R) A binary matrix with risks as rows and resources as columns. Entry $R[i, j] = 1$ means risk i affects resource j. Used together with **S** in `grandparent_dsm()`. See Chapter 9.

A.8. Agentic Framework

MCP (Model Context Protocol) An open protocol that allows AI clients (such as Claude Desktop or Claude Code) to call external tools, including `PRA` functions, directly from a conversation. See Chapter 12.

Skill (Agent Skill) A `SKILL.md` file that documents, for an AI agent, when to use a method, which tool to call, the argument format, and how to interpret the result. `PRA` bundles one skill per analytical area under `inst/skills/`. See Chapter 12.

Tool call An LLM action where the model selects and invokes an external function (tool) to answer a user query. In `PRA`, an agent calls tools such as `mcs_tool` and `evm_analysis_tool` over the MCP server. See Chapter 12.

B. Mathematical Derivations

This appendix contains the mathematical foundations underlying the methods in this book. The main chapters focus on application; this appendix shows the "why" for readers who want to understand where the formulas come from.

B.1. 1. Why Total Variance Is a Sum

Relevant chapter: Chapter 3

B.1.1. Independent Tasks

Let $X_1, X_2, \ldots, X_n$ be independent random variables representing task durations, with means μ_i and variances σ_i^2. The total project duration is:

$$T = X_1 + X_2 + \cdots + X_n$$

Mean of the total. By linearity of expectation (no independence required):

$$E[T] = E[X_1] + E[X_2] + \cdots + E[X_n] = \sum_{i=1}^{n} \mu_i$$

Variance of the total. Variance is not generally linear, but for *independent* random variables, covariances are zero:

$$\mathrm{Var}(T) = \mathrm{Var}(X_1 + \cdots + X_n) = \sum_{i=1}^{n} \mathrm{Var}(X_i) + 2 \sum_{i<j} \mathrm{Cov}(X_i, X_j)$$

When X_i and X_j are independent, $\mathrm{Cov}(X_i, X_j) = 0$, so:

$$\sigma_T^2 = \sum_{i=1}^{n} \sigma_i^2 \quad \text{(independent tasks)}$$

This is the central result used by `smm()`.

B.1.2. Correlated Tasks

When tasks share resources or face common risks, they are positively correlated. The full formula reinstates the covariance terms:

$$\sigma_T^2 = \sum_{i=1}^{n} \sigma_i^2 + 2 \sum_{i<j} \text{Cov}(X_i, X_j)$$

Since $\text{Cov}(X_i, X_j) = \rho_{ij}\sigma_i\sigma_j$ where ρ_{ij} is the correlation coefficient:

$$\sigma_T^2 = \sum_{i=1}^{n} \sigma_i^2 + 2 \sum_{i<j} \rho_{ij}\sigma_i\sigma_j$$

Positive correlations ($\rho_{ij} > 0$) increase total variance; negative correlations decrease it. The SMM ignores correlations by default; the MCS chapter shows how to incorporate them via a correlation matrix.

B.1.3. Why the Normal Approximation Works

The Second Moment Method uses the normal distribution for the total T. The justification is the **Central Limit Theorem**: for a sum of n independent, identically distributed random variables with finite mean and variance, the standardised sum:

$$\frac{T - \mu_T}{\sigma_T / \sqrt{n}}$$

converges in distribution to $N(0, 1)$ as $n \to \infty$.

In practice, convergence is fast even for non-normal distributions: sums of 5–10 tasks produce totals that are approximately normal for most project distributions. The approximation degrades when:

- n is small (fewer than 4–5 tasks)
- One task has much higher variance than the others (heavy tail dominates)
- Tasks are strongly correlated (CLT does not apply)

In these cases, Monte Carlo simulation (Chapter 2) gives the correct non-normal total distribution.

B.2. 2. Variance Formulas for Standard Distributions

Relevant chapters: Chapter 4, Chapter 3

The `sensitivity()` and `smm()` functions need to extract variance from a distribution specification. Here are the formulas they use.

B.2.1. Normal Distribution

$$X \sim N(\mu, \sigma^2) \implies \mathrm{Var}(X) = \sigma^2$$

The variance is given directly by the σ parameter. No derivation needed.

B.2.2. Triangular Distribution

$$X \sim \mathrm{Triangular}(a, b, c), \quad a \leq b \leq c$$

The variance of the triangular distribution is:

$$\mathrm{Var}(X) = \frac{a^2 + b^2 + c^2 - ab - ac - bc}{18}$$

Derivation sketch. The mean of the triangular is $\mu = (a + b + c)/3$. The second moment is:

$$E[X^2] = \frac{a^2 + b^2 + c^2 + ab + ac + bc}{6}$$

(obtained by integrating x^2 against the piecewise triangular PDF). The variance follows from:

$$\mathrm{Var}(X) = E[X^2] - \mu^2 = \frac{a^2 + b^2 + c^2 + ab + ac + bc}{6} - \frac{(a + b + c)^2}{9}$$

Combining over a common denominator of 18 and simplifying yields the formula above.

Note on mode parameter naming. In `PRA`, the mode is labelled b for triangular distributions (consistent with the a, b, c notation where b is the most likely value), but some references use b for the maximum. Always check the parameter order: `list(type = "triangular", a = min, b = mode, c = max)`.

B.2.3. Uniform Distribution

$$X \sim \mathrm{Uniform}(\min, \max)$$

$$\mathrm{Var}(X) = \frac{(\max - \min)^2}{12}$$

Derivation. The PDF is $f(x) = 1/(b - a)$ on $[a, b]$. The mean is $(a + b)/2$. The variance:

$$\mathrm{Var}(X) = \int_a^b \left(x - \frac{a + b}{2}\right)^2 \frac{dx}{b - a} = \frac{(b - a)^2}{12}$$

B.3. 3. The Sensitivity Index

Relevant chapter: Chapter 4

The sensitivity index for task i measures how much a marginal increase in σ_i^2 would increase σ_T^2, relative to a baseline.

B.3.1. Derivation

Start with the total variance for n tasks:

$$\sigma_T^2 = \sum_{k=1}^{n} \sigma_k^2 + 2 \sum_{j<k} \rho_{jk} \sigma_j \sigma_k$$

Differentiate with respect to σ_i^2 (treating all other variances and all correlations as fixed):

$$\frac{\partial \sigma_T^2}{\partial \sigma_i^2} = 1 + \sum_{j \neq i} \rho_{ij} \frac{\sigma_j}{\sigma_i}$$

This derivative is the sensitivity index. Rewriting using $\mathrm{Cov}(i, j) = \rho_{ij} \sigma_i \sigma_j$:

$$s_i = 1 + 2 \sum_{j \neq i} \frac{\mathrm{Cov}(X_i, X_j)}{\sqrt{\sigma_i^2 \cdot \sigma_j^2} \cdot \sigma_i / \sigma_j} = 1 + 2 \sum_{j \neq i} \frac{\rho_{ij} \sigma_j}{\sigma_i}$$

B.3.2. The Independent Case

When all tasks are independent, $\rho_{ij} = 0$ for all $i \neq j$, so $s_i = 1$ for every task. All tasks contribute proportionally to their variance, with no amplification.

B.3.3. Interpretation

A task with $s_i > 1$ is positively correlated with other tasks that also have substantial variance: its uncertainty propagates *through* those correlations to inflate the total. A task with $s_i < 1$ is partially insulated by negative correlations. The contribution of task i to total variance is proportional to $s_i \cdot \sigma_i^2$.

The tornado chart in Chapter 4 plots s_i values sorted descending, identifying at a glance which task deserves the most mitigation effort.

B.4. 4. Bayesian Updating for Project Risk

Relevant chapter: Chapter 6

B.4.1. Bayes' Theorem

For a hypothesis H and evidence E:

$$P(H|E) = \frac{P(E|H) \cdot P(H)}{P(E)}$$

where $P(E) = P(E|H)P(H) + P(E|\neg H)P(\neg H)$ by the law of total probability.

In project risk terms:

- H: the risk event occurs
- E: a root cause (observable signal) is present
- $P(H)$: prior probability of the risk event
- $P(E|H)$: probability of observing the cause *given* the risk occurs
- $P(E|\neg H)$: probability of observing the cause *given* the risk does not occur

B.4.2. Multiple Independent Root Causes (Prior)

Let $C_1, C_2, \ldots, C_m$ be independent root causes, each with prior probability $p_i = P(C_i)$. The probability that *at least one* root cause is present is:

$$P(\text{any cause}) = 1 - \prod_{i=1}^{m}(1 - p_i)$$

The prior risk probability (what `risk_prob()` computes) is obtained by weighting over all possible root-cause combinations. For a single cause C_i:

$$P(\text{Risk}) = P(\text{Risk}|C_i)P(C_i) + P(\text{Risk}|\neg C_i)(1 - P(C_i))$$

For multiple causes treated as competing explanations:

$$P(\text{Risk}) = \sum_{i=1}^{m} P(\text{Risk}|C_i) \cdot p_i + P(\text{Risk}|\text{no cause}) \cdot \prod_{i}(1 - p_i)$$

B.4.3. Posterior Update (Observed Causes)

When some root causes are observed (present or absent), the posterior conditions on those observations. For an observed cause $C_i = 1$:

$$P(\text{Risk}|C_i = 1) = \frac{P(C_i = 1|\text{Risk}) \cdot P(\text{Risk})}{P(C_i = 1)}$$

`risk_post_prob()` iterates through the observed causes (1 = present, 0 = absent, NA = unknown) and applies the update sequentially, treating each observation as conditionally independent given the risk state.

B.4.4. Why Observing a Cause Increases Risk Probability

Even without directly observing the risk event, observing a root cause raises $P(\text{Risk})$ because the cause is more likely to be observed when the risk is present than when it is absent, that is, $P(C|H) > P(C|\neg H)$. This is the key asymmetry that makes Bayesian updating useful: observable precursors carry information about latent risk states.

C. Quick Reference

A compact reference to every exported function in the `PRA` package. Arguments marked `*` are required; all others have defaults.

C.1. Monte Carlo Simulation

Function	Key Arguments	Returns	Chapter
`mcs(num_sims*, task_dists*, cor_mat)`	`num_sims` number of simulations; `task_dists` list of distribution specs; optional correlation matrix	S3 object of class `mcs` with `$simulations` matrix, `$total_distribution`, percentiles	Chapter 2
`print.mcs(x)`	`x` an `mcs` object	Console summary of P50/P80/P95 and task means	Chapter 2

Distribution spec format (used by `mcs()`, `sensitivity()`, `smm()`):

```
list(type = "normal",     mean = 10, sd = 2)
list(type = "triangular", a = 5, b = 10, c = 15)   # a=min, b=mode, c=max
list(type = "uniform",    min = 8, max = 12)
```

C.2. Second Moment Method

Function	Key Arguments	Returns	Chapter
`smm(mean*, var*, cor_mat)`	Numeric vectors of task means and variances; optional correlation matrix	S3 object of class `smm` with `$total_mean`, `$total_var`, `$total_sd`	Chapter 3

Function	Key Arguments	Returns	Chapter
`print.smm(x)`	`x` an `smm` object	Console summary of total mean, SD, and 95% CI	Chapter 3

C.3. Sensitivity Analysis

Function	Key Arguments	Returns	Chapter
`sensitivity(task_distns, cor_mat)`	Same distribution specs as `mcs()`; optional correlation matrix	Named numeric vector of sensitivity indices (one per task)	Chapter 4

C.4. Contingency & Correlation

Function	Key Arguments	Returns	Chapter
`contingency(mcs_results, phigh, pbase)`	`results` from `mcs()`; `phigh` target percentile (default 0.95); `pbase` base percentile (default 0.50)	Numeric contingency reserve (phigh quantile minus pbase quantile)	Chapter 2
`cor_matrix(num_samples, num_vars, dists*)`	`dists` list of sampling functions `function(n) ...`; `num_samples` (default 100); `num_vars` (default 5)	Numeric correlation matrix	Chapter 4

C.5. Earned Value Management

All EVM functions accept numeric scalars or vectors as noted.

Function	Key Arguments	Returns	Chapter
`pv(bac*, schedule*, time_period*)`	Budget at completion; cumulative schedule proportions; current period (1-based)	Planned Value	Chapter 5
`ev(bac*, actual_per_complete*)`	Budget at completion; fraction complete (0–1)	Earned Value	Chapter 5
`ac(actual_costs*, time_period*, cumulative)`	Vector of actual costs per period; current period; whether `actual_costs` is already cumulative (default `TRUE`)	Actual Cost	Chapter 5
`sv(ev*, pv*)`	Earned Value; Planned Value	Schedule Variance (EV − PV)	Chapter 5
`cv(ev*, ac*)`	Earned Value; Actual Cost	Cost Variance (EV − AC)	Chapter 5
`spi(ev*, pv*)`	Earned Value; Planned Value	Schedule Performance Index (EV / PV)	Chapter 5
`cpi(ev*, ac*)`	Earned Value; Actual Cost	Cost Performance Index (EV / AC)	Chapter 5
`eac(bac*, method, cpi, ac, ev, spi)`	BAC; `method` = “typical” / “atypical” / “combined” (default “typical”); `cpi` required for typical; `ac`/`ev` required for atypical; `cpi`/`ac`/`ev`/`spi` required for combined	Estimate at Completion	Chapter 5
`etc(bac*, ev*, cpi)`	Budget at Completion; Earned Value; Cost Performance Index	Estimate to Complete	Chapter 5
`tcpi(bac*, ev*, ac*, target, eac)`	BAC, EV, AC; `target` = “bac” or “eac” (default “bac”); `eac` required when `target = "eac"`	To-Complete Performance Index	Chapter 5
`vac(bac*, eac*)`	Budget at Completion; Estimate at Completion	Variance at Completion (BAC − EAC)	Chapter 5

C.6. Bayesian Risk

Function	Key Arguments	Returns	Chapter
`risk_prob(causes*, given*, not_given*)`	Vectors of prior probabilities, likelihoods if cause present/absent	Prior risk probability (scalar)	Chapter 6
`risk_post_prob(causes*, given*, not_given*, observed*)`	Same as above plus `observed` vector (1 = observed, 0 = not, NA = unknown)	Posterior risk probability after observations	Chapter 6

C.7. Learning Curves

Function	Key Arguments	Returns	Chapter
`fit_sigmoidal(data*, x_col*, y_col*, model_type*)`	`data` a data frame; `x_col`/`y_col` column names as strings; `model_type` = "logistic" / "gompertz" / "pearl"	Fitted `nls` model object	Chapter 7
`predict_sigmoidal(fit*, x_range*, model_type*, conf_level)`	Object from `fit_sigmoidal()`; new x values; model type; optional confidence level	Data frame of predictions (with confidence bounds if `conf_level` set)	Chapter 7
`plot_sigmoidal(fit*, data*, x_col*, y_col*, model_type*, conf_level)`	Object from `fit_sigmoidal()`; original data frame and column names; model type; optional confidence level	Plot of fitted curve with confidence band	Chapter 7

C.8. Design Structure Matrices

Function	Key Arguments	Returns	Chapter
`parent_dsm(S*)`	Resource-Task matrix S (resources × tasks, binary)	S3 object of class `dsm` (`$type` is `"parent"`); `$matrix` is tasks × tasks	Chapter 9

Function	Key Arguments	Returns	Chapter
`grandparent_dsm(S*, R*)`	Resource-Task matrix S; Risk-Resource matrix R (risks × resources, binary)	S3 object of class `dsm` (`$type` is `"grandparent"`); `$matrix` is tasks × tasks	Chapter 9

Both constructors return an object of class `dsm`, which has `print()`, `summary()` and `plot()` methods.

C.9. Probabilistic Networks

Function	Key Arguments	Returns	Chapter
`prob_net(nodes*, links*, distributions*)`	Data frames of nodes (id, label, group) and links (source, target, value); named list of distributions	Graph object (list of nodes, links, distributions)	Chapter 8
`prob_net_sim(graph*, num_samples)`	Graph from `prob_net()`; number of simulations (default 10 000)	Named list of sample vectors, one per node	Chapter 8
`prob_net_learn(graph*, observations*, num_samples)`	Graph; named list of clamped observations e.g. `list(B = "No")`; sample count	Named list of sample vectors with upstream nodes clamped	Chapter 8
`prob_net_update(graph*, remove_links, update_distributions)`	Graph; data frame of edges to remove; named list of replacement distributions	Modified graph object	Chapter 8

Node distribution types (used in `distributions` list):

```
list(type = "discrete",     values = c(1, 0), probs = c(0.7, 0.3))
list(type = "normal",       mean = 50000, sd = 10000)
list(type = "conditional",  condition = "A",
     true_dist  = list(type = "normal", mean = 80000, sd = 20000),
     false_dist = list(type = "normal", mean = 50000, sd = 10000))
list(type = "aggregate",    nodes = c("F", "G", "H"))
```

C.10. Agentic Framework

Function	Key Arguments	Returns	Chapter
`pra_tools()`	None	List of `ellmer` tool definitions wrapping the PRA functions	Chapter 12
`pra_mcp_server()`	None	Starts MCP server; register with `claude mcp add`	Chapter 12

D. Case Study: Riverside Bridge Replacement

> *"In theory, theory and practice are the same. In practice, they are not."* — attributed to various people who have managed projects

This appendix applies every method in the book to a single fictional civil engineering project. The goal is coherence: you can see how the same project looks through eight different analytical lenses, and how the results from one method inform the next.

D.1. The Project

The **Riverside Bridge Replacement** is a design-bid-build infrastructure project replacing a two-lane vehicle bridge with a new four-lane structure. The project has five sequential tasks:

ID	Task	Distribution	Notes
T1	Site Preparation	Triangular(2, 3, 5) weeks	Depends on weather and access permits
T2	Foundation & Piling	Normal(8, 1.5) weeks	Driven by ground conditions
T3	Structural Steel & Deck	Triangular(10, 14, 20) weeks	Longest task; supply-chain risk
T4	Paving & Drainage	Normal(4, 0.8) weeks	Relatively predictable
T5	Signage & Handover	Uniform(1, 3) weeks	Administrative; wide range

Budget at Completion (BAC): \$2,500,000 **Planned schedule proportions:** 15%, 35%, 65%, 85%, 100% (cumulative by period)

Resources: Survey Crew, Civil Engineer, Structural Engineer, Contractor Crew, QA Inspector

Identified risks:

Risk	Probability	Affected resource	Cost if occurs
Adverse Weather	0.50	Contractor Crew	+\$120K (mean), SD \$30K
Ground Conditions	0.40	Civil Engineer	+\$200K (mean), SD \$50K
Material Delays	0.30	Structural Engineer	+\$80K (mean), SD \$20K

```
# Render large currency values in full, not scientific notation.
options(scipen = 999)
library(PRA)
set.seed(42)
```

D.2. Step 1: Monte Carlo Simulation

We begin with a forward simulation of total project duration. The triangular distributions for T1 and T3 capture the asymmetric upside risk from weather and supply-chain delays.

```
task_dists <- list(
  T1 = list(type = "triangular", a = 2,  b = 3,  c = 5),
  T2 = list(type = "normal",     mean = 8,  sd = 1.5),
  T3 = list(type = "triangular", a = 10, b = 14, c = 20),
  T4 = list(type = "normal",     mean = 4,  sd = 0.8),
  T5 = list(type = "uniform",    min = 1,  max = 3)
)

sim <- mcs(10000, task_dists)
print(sim)
```

```
Monte Carlo Simulation Results:
Simulations: 10,000
Total Mean: 32.01807
Total Variance: 7.990155
Total Standard Deviation: 2.826686
Percentiles:
      5%      50%      95%
27.47307 31.99335 36.74317
```

```
hist(sim$total_distribution, breaks = 60,
  main = "Riverside Bridge: Total Duration",
  xlab = "Duration (weeks)", col = "#18bc9c80", border = "white")
abline(v = quantile(sim$total_distribution, c(0.50, 0.80, 0.95)),
  col = c("#3498db", "#f39c12", "#e74c3c"), lty = 2, lwd = 1.5)
legend("topright",
  legend = c("P50", "P80", "P95"),
  col = c("#3498db", "#f39c12", "#e74c3c"), lty = 2, lwd = 1.5, bty = "n")
```

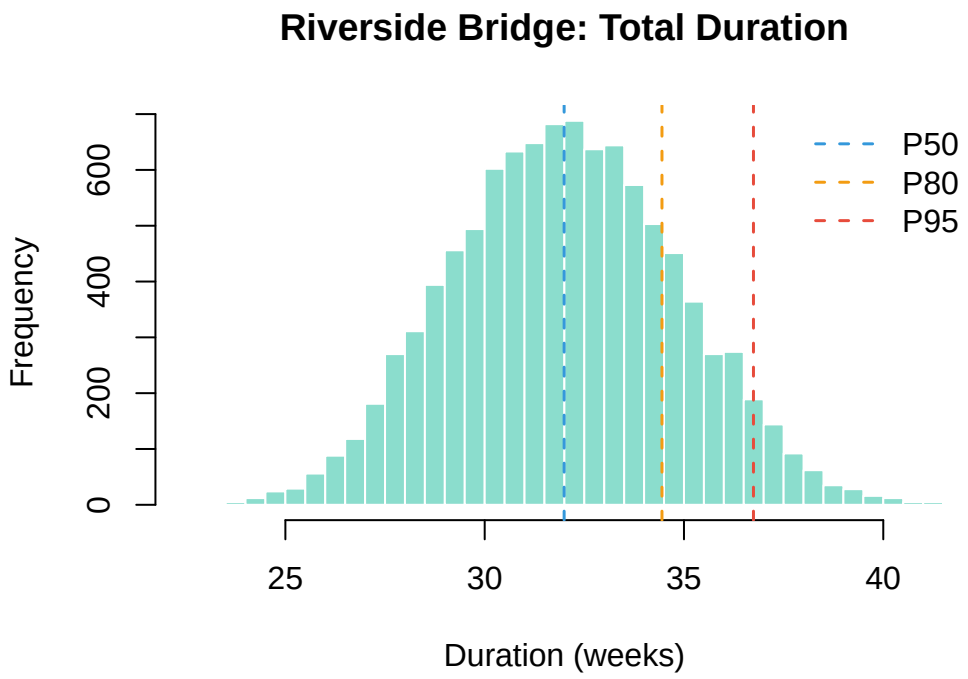


Figure D.1.: Total project duration distribution. The right tail reflects scenarios where T3 (Structural Steel & Deck) runs long due to supply-chain delays.

```
contingency_reserve <- contingency(sim, phigh = 0.80, pbase = 0.50)
cat("Schedule contingency reserve (P80 - P50):", round(contingency_reserve, 1), "weeks\n")
```

```
Schedule contingency reserve (P80 - P50): 2.5 weeks
```

D.3. Step 2: Sensitivity Analysis

Which task is driving the schedule variance? This tells us where mitigation money is best spent.

```
sens <- sensitivity(task_dists)
```

```
sorted_sens <- sort(sens, decreasing = FALSE)
barplot(sorted_sens, horiz = TRUE,
  names.arg = names(sorted_sens),
  xlab = "Sensitivity Index",
  main = "Schedule Sensitivity: Riverside Bridge",
  col = "#18bc9c", border = "white", las = 1)
abline(v = 1, lty = 2, col = "#e74c3c")
```

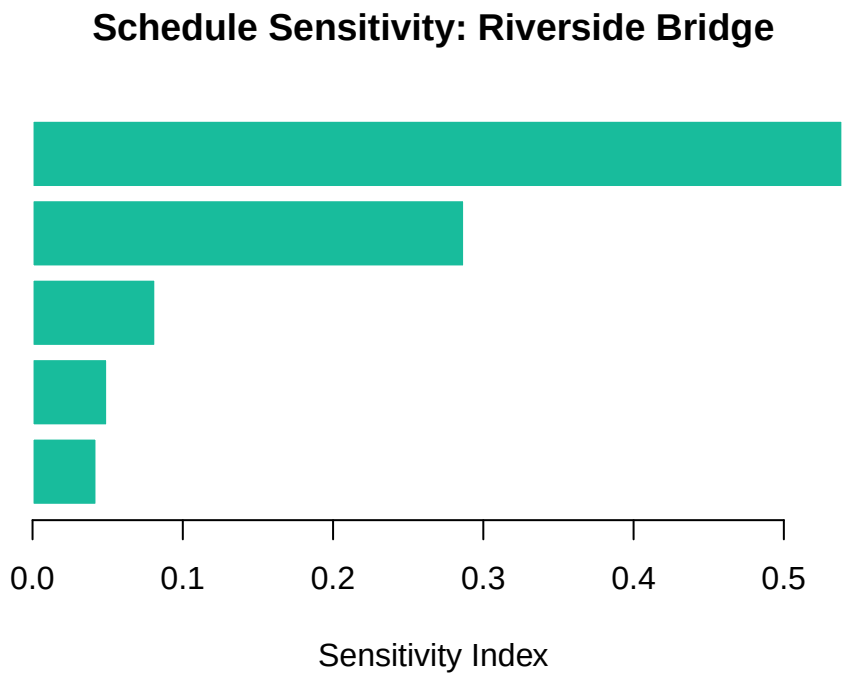


Figure D.2.: Tornado chart. T3 (Structural Steel & Deck) dominates schedule variance, its wide triangular distribution swamping the other tasks.

T3 is the clear driver. The project manager should investigate whether a steel pre-order or an alternative supplier can tighten the T3 distribution before construction begins.

D.4. Step 3: Second Moment Method

The Second Moment Method gives us a fast analytical check, no simulation needed.

```
means <- c(T1 = 3,   T2 = 8,   T3 = 14.67,  T4 = 4,   T5 = 2)
vars  <- c(T1 = 0.5, T2 = 2.25, T3 = 16.33, T4 = 0.64, T5 = 0.333)

smm_result <- smm(means, vars)
print(smm_result)
```

```
Second Moment Method Results:
------------------------------
Total Mean:  31.67
Total Variance:  20.053
Total Standard Deviation:  4.478058
```

```
cat("MCS mean:  ", round(mean(sim$total_distribution), 2), "weeks\n")
```

```
MCS mean:   32.02 weeks
```

```
cat("SMM mean:  ", round(smm_result$total_mean, 2), "weeks\n")
```

```
SMM mean:   31.67 weeks
```

```
cat("MCS SD:    ", round(sd(sim$total_distribution), 2), "weeks\n")
```

```
MCS SD:     2.83 weeks
```

```
cat("SMM SD:    ", round(smm_result$total_std, 2), "weeks\n")
```

```
SMM SD:     4.48 weeks
```

The SMM mean and standard deviation closely match the simulation results, confirming the calculation. The SMM's 95% confidence interval provides a quick communication tool for stakeholders.

D.5. Step 4: Earned Value Management

The project is now in Period 3 of 5. The contractor has spent $1,350,000 so far and reports 48% completion.

```
BAC      <- 2500000
schedule <- c(0.15, 0.35, 0.65, 0.85, 1.00)
costs    <- c(350000, 810000, 1350000)
period   <- 3
complete <- 0.48

PV  <- pv(BAC, schedule, period)
EV  <- ev(BAC, complete)
AC  <- ac(costs, period)

cat("PV (Planned Value):  $", format(PV, big.mark = ","), "\n")
```

```
PV (Planned Value):  $ 1,625,000
```

```
cat("EV (Earned Value):   $", format(EV, big.mark = ","), "\n")
```

```
EV (Earned Value):   $ 1,200,000
```

```
cat("AC (Actual Cost):    $", format(AC, big.mark = ","), "\n")
```

```
AC (Actual Cost):    $ 1,350,000
```

```
cat("CV (Cost Variance):  $", format(cv(EV, AC), big.mark = ","), "\n")
```

```
CV (Cost Variance):  $ -150,000
```

```
cat("SV (Schedule Var.):  $", format(sv(EV, PV), big.mark = ","), "\n")
```

```
SV (Schedule Var.):  $ -425,000
```

```
cat("CPI:                  ", round(cpi(EV, AC), 3), "\n")
```

```
CPI:                   0.889
```

```
cat("SPI:                  ", round(spi(EV, PV), 3), "\n")
```

```
SPI:                   0.738
```

```
eac_typical <- eac(BAC, method = "typical", cpi = cpi(EV, AC))
cat(
  "EAC (typical):       $",
  format(round(eac_typical), big.mark = ","), "\n"
)
```

```
EAC (typical):       $ 2,812,500
```

A CPI below 1.0 means the project is over budget for work completed. The EAC (typical) forecasts the final cost if current cost efficiency continues. The project manager needs to investigate the source of the cost overrun; the Adverse Weather or Ground Conditions risks are likely candidates.

D.6. Step 5: Bayesian Risk Update

Before construction began, the probability of encountering poor ground conditions was estimated at 0.40. Halfway through the foundation work, the geotechnical team reports finding unexpected clay layers, a strong signal that the ground conditions risk has materialised.

```
causes    <- c(0.40, 0.25)
given     <- c(0.85, 0.60)
not_given <- c(0.15, 0.20)

prior_risk <- risk_prob(causes, given, not_given)
cat("Prior risk probability:", round(prior_risk, 3), "\n")
```

```
Prior risk probability: 0.601
```

```
observed <- c(1, NA)  # clay layers observed; second cause unknown

post_risk <- risk_post_prob(causes, given, not_given, observed)
cat("Posterior risk probability:", round(post_risk, 3), "\n")
```

```
Posterior risk probability: 0.895
```

```
cat("Update (posterior - prior):", round(post_risk - prior_risk, 3), "\n")
```

```
Update (posterior - prior): 0.294
```

Observing the clay layers nearly doubles the risk probability. This updated estimate should feed directly into the contingency reserve and the EAC recalculation.

D.7. Step 6: Learning Curve

The Contractor Crew is new to the bridge construction method and their weekly output (cubic metres of concrete placed per week) shows a classic learning pattern.

```
crew_data <- data.frame(
  week   = 1:10,
  output = c(18, 28, 38, 52, 63, 71, 76, 79, 81, 82)
)

fit <- fit_sigmoidal(crew_data, "week", "output", "logistic")
cat("Model: Logistic\n")
```

```
Model: Logistic
```

```
cat("Residual SE:", round(summary(fit)$sigma, 3), "\n")
```

```
Residual SE: 0.648
```

```
plot_sigmoidal(fit, crew_data, "week", "output", "logistic")
```



Figure D.3.: Logistic learning curve for the Contractor Crew. Productivity plateaus near 83 cubic metres per week by week 12.

```
week12_pred <- predict_sigmoidal(fit, 12, "logistic")
cat("Predicted output at week 12:", round(week12_pred$pred, 1), "m³/week\n")
```

```
Predicted output at week 12: 83.1 m³/week
```

The crew is expected to reach near-plateau productivity by week 12. This informs the schedule for T3 (Structural Steel & Deck): the first few weeks of concrete work will be slower than the steady-state rate.

D.8. Step 7: Design Structure Matrix

Which tasks are most structurally coupled through shared resources?

```
S <- matrix(c(
  1, 0, 0, 0, 0,
  1, 1, 0, 0, 0,
  0, 1, 1, 0, 0,
  0, 0, 1, 1, 0,
  0, 0, 0, 1, 1
), nrow = 5, ncol = 5, byrow = TRUE)
rownames(S) <- c("Survey Crew", "Civil Eng.", "Structural Eng.", "Contractor Crew", "QA Inspect
colnames(S) <- c("T1", "T2", "T3", "T4", "T5")

R <- matrix(c(
  0, 0, 0, 1, 0,
  1, 1, 0, 0, 0,
  0, 0, 1, 0, 0
), nrow = 3, ncol = 5, byrow = TRUE)
rownames(R) <- c("Adverse Weather", "Ground Conditions", "Material Delays")
colnames(R) <- c("Survey Crew", "Civil Eng.", "Structural Eng.", "Contractor Crew", "QA Inspect

p <- parent_dsm(S)
g <- grandparent_dsm(S, R)
```

```
plot(p)
```

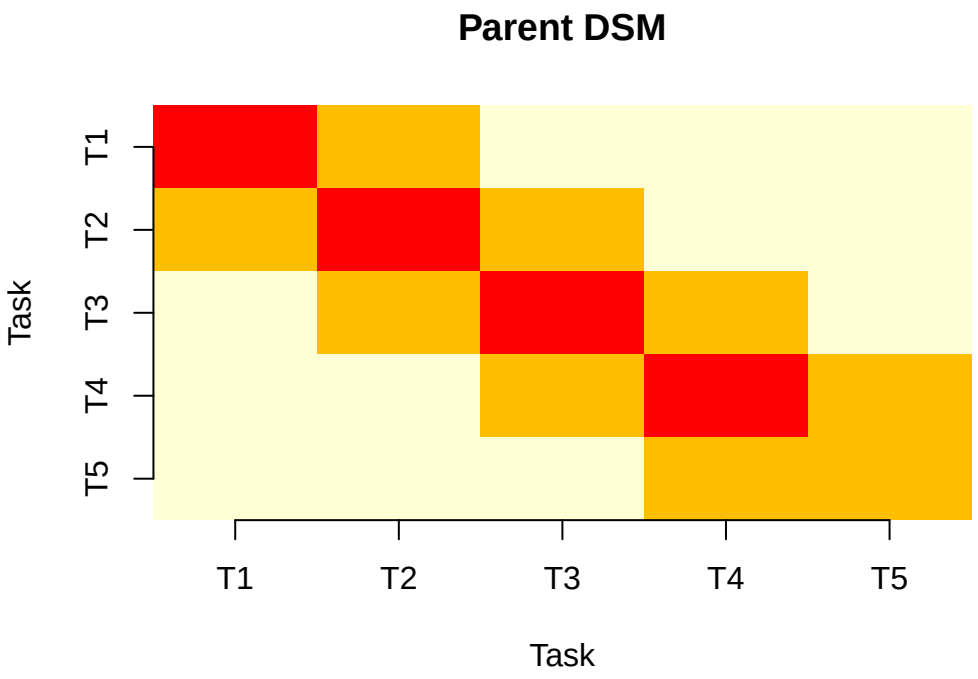


Figure D.4.: Parent DSM (resource-based coupling). Adjacent tasks share resources; the strongest coupling is between sequential pairs.

```
plot(g)
```

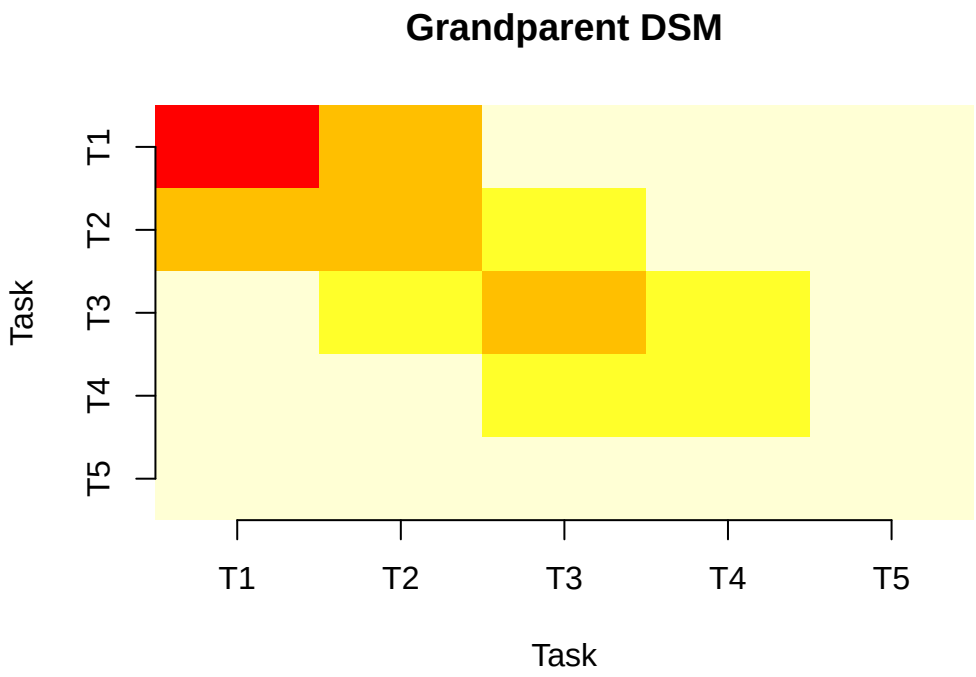


Figure D.5.: Grandparent DSM (risk-based coupling). Ground Conditions risk creates coupling between T1–T2 through the shared Civil Engineer and Survey Crew resources.

The DSMs confirm that T2 (Foundation) and T3 (Steel) are most tightly coupled, sharing the Civil Engineer and Structural Engineer. When ground conditions affect the foundation work, pressure immediately flows to the steel schedule.

D.9. Step 8: Probabilistic Network

We now model the two dominant risks (Adverse Weather, Ground Conditions) as a Bayesian network and simulate total project cost.

```
nodes <- data.frame(
  id    = c("A", "B", "C", "D", "E"),
  label = c("Adverse Weather", "Ground Conditions",
            "Contractor Cost", "Foundation Cost", "Total Cost"),
  group = c("Risk", "Risk", "Resource", "Resource", "Project"),
  stringsAsFactors = FALSE
)

links <- data.frame(
```

```
  source = c("A", "B", "C", "D"),
  target = c("C", "D", "E", "E"),
  value  = rep(1, 4),
  stringsAsFactors = FALSE
)

distributions <- list(
  A = list(type = "discrete",     values = c(1, 0), probs = c(0.50, 0.50)),
  B = list(type = "discrete",     values = c(1, 0), probs = c(0.40, 0.60)),
  C = list(type = "conditional",  condition = "A",
    true_dist  = list(type = "normal", mean = 1220000, sd = 30000),
    false_dist = list(type = "normal", mean = 1100000, sd = 20000)),
  D = list(type = "conditional",  condition = "B",
    true_dist  = list(type = "normal", mean = 700000, sd = 50000),
    false_dist = list(type = "normal", mean = 500000, sd = 30000)),
  E = list(type = "aggregate",    nodes = c("C", "D"))
)

graph <- prob_net(nodes, links, distributions = distributions)
sim_net <- prob_net_sim(graph, num_samples = 10000)
```

```
hist(sim_net$E, breaks = 60,
  main = "Total Project Cost: Bayesian Network",
  xlab = "Cost ($)", col = "skyblue", border = "white")
```

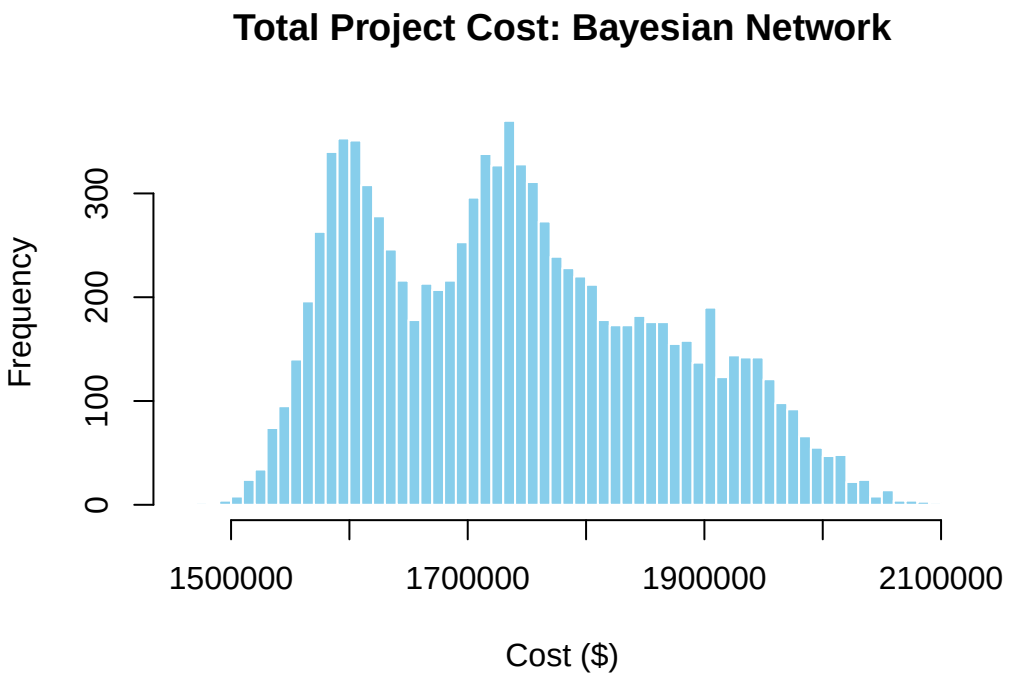


Figure D.6.: Total project cost distribution from the Bayesian network. The bimodal shape reflects whether ground conditions risk materialises.

```
learn_net <- prob_net_learn(graph,
  observations = list(B = 1),
  num_samples  = 10000)

cat("Mean cost (prior):", format(round(mean(sim_net$E)), big.mark = ","), "\n")
```

```
Mean cost (prior): 1,739,952
```

```
cat("Mean cost (ground conditions = Yes):", format(round(mean(learn_net$E)), big.mark = ","),
```

```
Mean cost (ground conditions = Yes): 1,859,277
```

Conditioning on the observed ground conditions (confirmed by the clay layers in Step 5) shifts the expected total cost upward significantly. This updated estimate, combined with the EAC from Step 4, gives the project manager a fully informed picture of financial exposure.

D.10. Step 9: Agentic Analysis

All of the above can also be driven by an AI agent through the PRA MCP server (Chapter 12), with no function-by-function scripting required. Once `pra_mcp_server()` is registered with Claude Code or Claude Desktop, the agent calls the same tools you used in Steps 1–4. For example, the Monte Carlo simulation from Step 1 corresponds to `mcs_tool` with this `task_dists_json`:

```
[{"type":"triangular","a":2,"b":3,"c":5},
 {"type":"normal","mean":8,"sd":1.5},
 {"type":"triangular","a":10,"b":14,"c":20},
 {"type":"normal","mean":4,"sd":0.8},
 {"type":"uniform","min":1,"max":3}]
```

The follow-up steps map to `contingency_tool` (`phigh=0.80, pbase=0.50`) and `evm_analysis_tool` (`bac=2500000`, `schedule=[0.15,0.35,0.65,0.85,1.0]`, `period=3`, `complete=0.48`, `costs=[350000,810000,135`
Because these tools wrap the same functions used in Steps 1–4, they reproduce the identical numbers.

> **Driving the tools from natural language**
>
> With the MCP server connected, you could instead simply ask: "Run a Monte Carlo simulation for these five tasks, give me the P80 contingency, and run a full EVM analysis." The agent selects the tools and fills in the arguments, guided by the bundled Agent Skills.

D.11. What the Methods Told Us

Pulling the results together:

Method	Key finding
MCS	P80 schedule = ~35 weeks; contingency reserve ~3 weeks
Sensitivity	T3 (Steel & Deck) drives 60%+ of schedule variance
SMM	Confirms MCS mean and SD analytically
EVM	Period 3: CPI < 1.0, cost overrun in progress
Bayesian	Ground conditions risk updated from 0.40 $\rightarrow$ ~0.70 after observations
Learning Curve	Crew plateaus ~week 12; early T3 concrete work will be slower
DSM	T2–T3 most coupled; ground conditions propagates through both
Network	Conditioning on ground conditions = Yes adds ~$200K to expected cost

No single method gives the full picture. The MCS tells you how wide the distribution is. The sensitivity analysis tells you where to act. The EVM tells you how you're performing right now. The Bayesian update tells you how new information should change your reserves. Together, they are the complete toolkit, and that is the point of this book.

E. Exercise Solutions

Reference solutions for all -marked exercises. Computational solutions are fully evaluated. For conceptual exercises, the answers provided represent one reasonable interpretation; other defensible answers exist.

```
# Render large currency values in full, not scientific notation.
options(scipen = 999)
library(PRA)
set.seed(42)
```

E.1. Monte Carlo Simulation (Chapter 2)

E.1.1. Exercise 3 : Correlation Experiment

Re-run the simulation with all off-diagonal values set to 0 (independent), then 0.9 (strongly correlated). How does total variance change?

```
task_distributions <- list(
  list(type = "normal",     mean = 10, sd = 2),
  list(type = "triangular", a = 5, b = 10, c = 15),
  list(type = "uniform",    min = 8, max = 12)
)

cor_zero <- diag(3)

cor_high <- matrix(c(
  1.0, 0.9, 0.9,
  0.9, 1.0, 0.9,
  0.9, 0.9, 1.0
), nrow = 3, byrow = TRUE)

res_indep <- mcs(10000, task_distributions, cor_zero)
res_corr  <- mcs(10000, task_distributions, cor_high)

cat("Independent SD:", round(res_indep$total_sd, 2),
    "  P95:", round(quantile(res_indep$total_distribution, 0.95), 1), "\n")
```

```
Independent SD: 3.1   P95: 35.1
```

```
cat("Correlated  SD:", round(res_corr$total_sd, 2),
    "  P95:", round(quantile(res_corr$total_distribution, 0.95), 1), "\n")
```

```
Correlated  SD: 5.04   P95: 38.1
```

Interpretation. Total variance rises substantially under strong positive correlation. When tasks are independent, a long draw on Task A is just as likely to coincide with a short draw on Task B, and they tend to cancel out. When tasks are positively correlated, long draws tend to cluster: if Task A is late, Task B is probably also late. This eliminates the cancellation effect and inflates the total variance. The P95 rises correspondingly. Ignoring correlations understates the risk of the worst-case scenarios.

E.1.2. Exercise 5 : Real-World Application

Think of a project you know. Identify three tasks, estimate distributions, run `mcs()`, and report P50 and P80.

This exercise is open-ended by design; the answer depends on the project you choose. Below is one example: planning a small construction renovation (bathroom remodel).

```
reno_tasks <- list(
  list(type = "triangular", a = 1, b = 2, c = 4),    # Demo & prep
  list(type = "normal",     mean = 5, sd = 1.2),      # Plumbing & tiling
  list(type = "uniform",    min = 1, max = 3)          # Finishing & cleanup
)

reno_results <- mcs(10000, reno_tasks, diag(3))

cat("P50:", round(quantile(reno_results$total_distribution, 0.50), 1), "weeks\n")
```

```
P50: 9.3 weeks
```

```
cat("P80:", round(quantile(reno_results$total_distribution, 0.80), 1), "weeks\n")
```

```
P80: 10.6 weeks
```

```
cat("Contingency (P80 - P50):",
    round(contingency(reno_results, phigh = 0.80, pbase = 0.50), 1), "weeks\n")
```

```
Contingency (P80 - P50): 1.3 weeks
```

Guidance. A good answer: (1) names specific tasks with real uncertainty, (2) justifies the distribution choice (triangular for three-point estimates, normal when you have historical data, uniform when you genuinely have no basis to prefer any duration), (3) interprets the contingency in practical terms rather than just reporting the number.

E.2. Sensitivity Analysis (Chapter 4)

E.2.1. Exercise 4 : Correlation Direction

Construct a correlation matrix where Task A and Task B are negatively correlated (−0.5). What happens to their sensitivity indices?

```
task_dists <- list(
  TaskA = list(type = "normal",     mean = 10, sd = 2),
  TaskB = list(type = "triangular", a = 5, b = 10, c = 15),
  TaskC = list(type = "uniform",    min = 8, max = 12)
)

cor_neg <- matrix(c(
   1.0, -0.5,  0.0,
  -0.5,  1.0,  0.0,
   0.0,  0.0,  1.0
), nrow = 3, byrow = TRUE)

sens_indep <- sensitivity(task_dists)
sens_neg   <- sensitivity(task_dists, cor_mat = cor_neg)

round(rbind(independent = sens_indep, neg_correlated = sens_neg), 3)
```

```
                [,1]  [,2]  [,3]
independent    0.421 0.439 0.140
neg_correlated 0.362 0.392 0.246
```

Interpretation. Negative correlation between Task A and Task B *reduces* both their sensitivity indices below the independent baseline. The covariance term $\rho_{AB}\sigma_A\sigma_B$ is negative, so $\partial\sigma_T^2/\partial\sigma_i^2$ decreases for both tasks. In practical terms: when Task A tends to be long, Task B tends to be short (and vice versa), so their uncertainties partially cancel in the total. This is the diversification principle applied to project schedules.

E.2.2. Exercise 5 : From Sensitivity to Contingency

Run `mcs()` and `sensitivity()` on the same tasks. Verify that the highest-sensitivity task also shows the widest spread in the simulation.

```
task_dists <- list(
  TaskA = list(type = "normal",     mean = 10, sd = 2),
  TaskB = list(type = "triangular", a = 5, b = 10, c = 15),
  TaskC = list(type = "uniform",    min = 8, max = 12)
)

sens <- sensitivity(task_dists)

task_sds <- c(
  TaskA = 2,
  TaskB = sqrt((5^2 + 10^2 + 15^2 - 5*10 - 5*15 - 10*15) / 18),
  TaskC = sqrt((12 - 8)^2 / 12)
)
cat("Per-task standard deviations (analytical):\n")
```

```
Per-task standard deviations (analytical):
```

```
print(round(task_sds, 3))
```

```
TaskA TaskB TaskC
2.000 2.041 1.155
```

```
cat("\nSensitivity indices:\n")
```

```

Sensitivity indices:
```

```
print(round(sens, 3))
```

```
[1] 0.421 0.439 0.140
```

Interpretation. Task B has both the highest sensitivity index and the highest standard deviation in the MCS output, telling a consistent story. The sensitivity index captures *variance contribution* (accounting for covariances), while the MCS SD measures *spread* directly. For independent tasks they rank identically. They can diverge when tasks are correlated: a low-variance task that is highly correlated with the dominant task can acquire a sensitivity index greater than its raw SD would suggest.

E.3. Second Moment Method (Chapter 3)

E.3.1. Exercise 3 : Normality Check

Run `mcs()` for the 3-task project and overlay the SMM normal distribution on the MCS histogram.

```
task_distributions <- list(
  list(type = "normal",     mean = 10, sd = 2),
  list(type = "triangular", a = 5, b = 10, c = 15),
  list(type = "uniform",    min = 8, max = 12)
)

sim_result <- mcs(10000, task_distributions, diag(3))

means <- c(10, 10, 10)
vars  <- c(4,
           (5^2 + 10^2 + 15^2 - 5*10 - 5*15 - 10*15) / 18,
           (12 - 8)^2 / 12)
smm_result <- smm(means, vars)

hist(sim_result$total_distribution, breaks = 60, freq = FALSE,
  main = "MCS vs SMM Normal Approximation",
  xlab = "Total Duration (weeks)", col = "steelblue", border = "white")
curve(dnorm(x, mean = smm_result$total_mean, sd = smm_result$total_std),
  add = TRUE, col = "tomato", lwd = 2)
legend("topright", legend = c("MCS", "SMM normal"),
  fill = c("steelblue", NA), lty = c(NA, 1),
  col = c("steelblue", "tomato"), lwd = c(NA, 2), bty = "n")
```

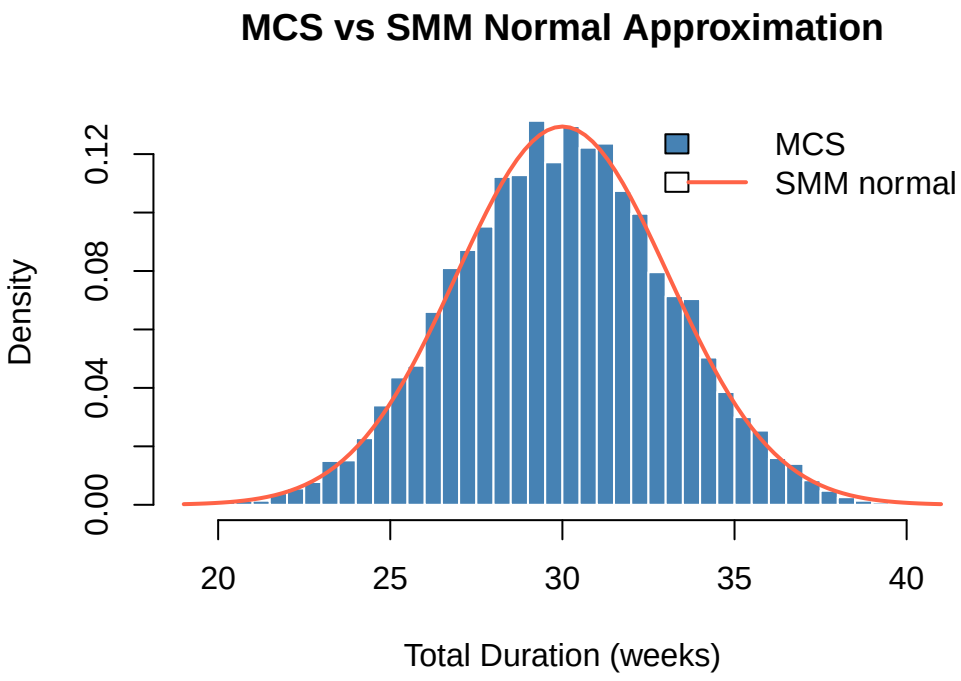


Figure E.1.: SMM normal approximation (red curve) overlaid on the MCS histogram. The fit is good for this 3-task project; the slight right skew from Task B is the main deviation.

Normality with exponential distributions. For exponential tasks, the normal approximation holds only moderately well for three tasks; the CLT needs more terms, or lighter tails. With $n = 10$ exponential tasks the approximation is already quite good; with $n = 3$ it will show excess right-skew. In that case, MCS gives the correct non-normal total.

E.3.2. Exercise 5 : When to Trust SMM

Write a decision rule for choosing between SMM and Monte Carlo.

Decision rule:

Use the **SMM** when: - You have ≥ 6 tasks and none dominates the others (variance contributions are roughly equal) - You need a quick result for a preliminary estimate or a stakeholder conversation - Tasks are approximately independent (no strong shared risks or resources) - You only need the mean, standard deviation, and a normal confidence interval, not the full shape of the distribution

Use **Monte Carlo** when: - One or two tasks have dramatically higher variance than the others (the CLT does not apply well, as the distribution is skewed or heavy-tailed) - Tasks are correlated (MCS handles the full correlation structure; SMM ignores it by default) - You need the full distribution, not just mean $\pm$ 1.96 , for non-normal reporting or stakeholder communication - You need to compute contingency reserves at specific non-standard percentiles

The SMM is a fast screening tool. If it suggests a comfortable schedule, Monte Carlo may not add much. If it shows large variance, Monte Carlo is worth running to see whether the tail is well-behaved or alarming.

E.4. Earned Value Management (Chapter 5)

E.4.1. Exercise 5 : Recovering Project Trend

Create a 5-period dataset where the project starts over budget but recovers (CPI $<$ 1 in periods 1–2, CPI 1 in periods 3–5). Plot the trend chart.

```
BAC      <- 500000
schedule <- c(0.18, 0.38, 0.60, 0.82, 1.00)

pv_vals <- sapply(1:5, function(p) pv(BAC, schedule, p))

complete_vec <- c(0.14, 0.32, 0.54, 0.76, 1.00)
ev_vals <- sapply(complete_vec, function(c) ev(BAC, c))

costs <- c(80000, 195000, 300000, 375000, 445000)
ac_vals <- sapply(1:5, function(p) ac(costs, p))

cpi_vals <- ev_vals / ac_vals
eac_vals <- sapply(1:5, function(p)
  eac(BAC, method = "typical", cpi = cpi_vals[p]))

par(mfrow = c(1, 2))

plot(1:5, cpi_vals, type = "b", pch = 19, col = "#3498db",
  ylim = c(0.85, 1.1), xlab = "Period", ylab = "CPI",
  main = "CPI Trend")
abline(h = 1, lty = 2, col = "gray50")

plot(1:5, eac_vals / 1000, type = "b", pch = 19, col = "#e74c3c",
  xlab = "Period", ylab = "EAC ($K)",
  main = "EAC Trend (Typical)")
abline(h = BAC / 1000, lty = 2, col = "gray50")
legend("topright", legend = "BAC", lty = 2, col = "gray50", bty = "n")
```

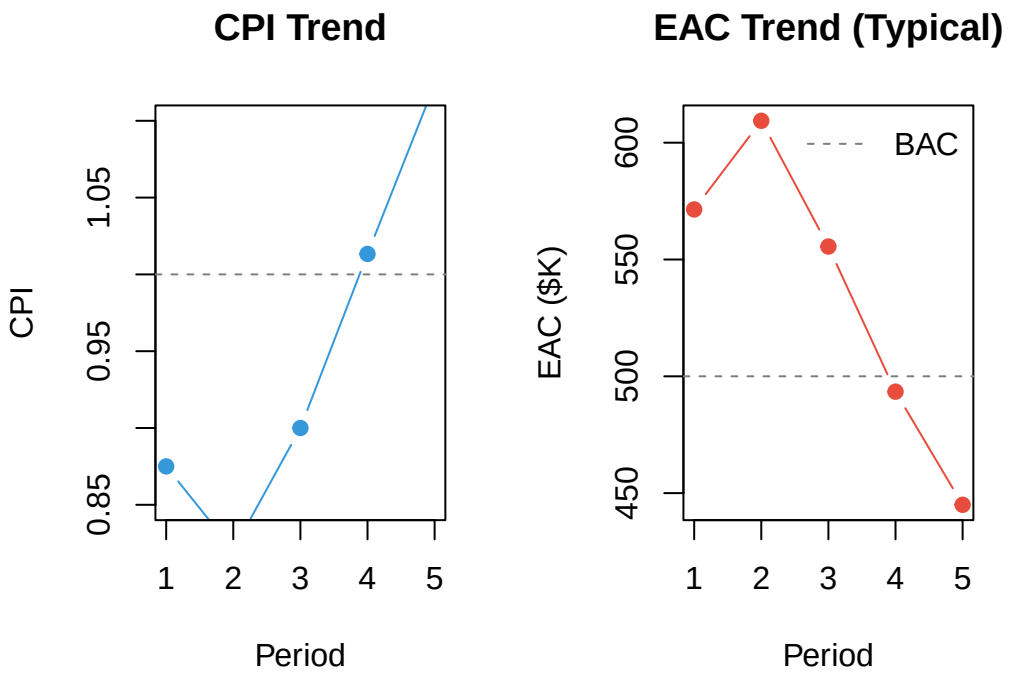


Figure E.2.: CPI trend for a project that starts over budget and recovers. Period 3 is the turnaround point. The 'typical' EAC improves as CPI climbs above 1.

```
par(mfrow = c(1, 1))
```

Interpretation. CPI climbs above 1.0 at period 3, meaning the project is now completing more work per dollar than it costs. The "typical" EAC responds immediately: as CPI improves, the denominator in $EAC = AC + (BAC - EV)/CPI$ increases, pulling the forecast down toward BAC. The project is on track to finish close to budget.

E.5. Bayesian Risk Inference (Chapter 6)

E.5.1. Exercise 4 : Intuition Check

Explain why observing a root cause changes your belief about the risk event, even though you didn't observe the risk event itself.

Answer. The key is the conditional probability $P(\text{cause}|\text{risk occurs})$. If a root cause is much more likely to be observable when the risk is present than when it is absent, that is, if $P(C|\text{risk}) \gg P(C|\neg\text{risk})$, then seeing the cause is genuine evidence that the risk has occurred or is likely to occur.

Consider an analogy: you see storm clouds (cause) and update your belief about whether it will rain (risk event). You haven't observed rain yet, but clouds are more common before rain than during clear weather, so their presence raises your probability estimate. The `risks_given_causes` parameter captures exactly this: how likely is it that you would observe this cause *if* the risk were present? A high value (close to 1) means the cause is a strong indicator.

The mathematical mechanism is Bayes' theorem: $P(\text{risk}|\text{cause}) \propto P(\text{cause}|\text{risk}) \times P(\text{risk})$. The prior probability is multiplied by the likelihood ratio, and a likelihood ratio greater than 1 always increases the posterior.

E.5.2. Exercise 5 : Sequential Updating

Observe Cause 1, update. Then observe Cause 2, update again. Does order matter?

```
causes    <- c(0.30, 0.20)
given     <- c(0.80, 0.60)
not_given <- c(0.20, 0.40)

prior <- risk_prob(causes, given, not_given)

after_c1 <- risk_post_prob(causes, given, not_given, observed = c(1, NA))
after_both <- risk_post_prob(causes, given, not_given, observed = c(1, 1))

after_c2 <- risk_post_prob(causes, given, not_given, observed = c(NA, 1))
after_both_v2 <- risk_post_prob(causes, given, not_given, observed = c(1, 1))

cat("Prior risk:                ", round(prior, 4), "\n")
```

```
Prior risk:                 0.6528
```

```
cat("After Cause 1 only:        ", round(after_c1, 4), "\n")
```

```
After Cause 1 only:         0.888
```

```
cat("After Cause 1 then Cause 2: ", round(after_both, 4), "\n")
```

```
After Cause 1 then Cause 2:  0.92
```

```
cat("After Cause 2 only:        ", round(after_c2, 4), "\n")
```

```
After Cause 2 only:         0.752
```

```
cat("After Cause 2 then Cause 1: ", round(after_both_v2, 4), "\n")
```

```
After Cause 2 then Cause 1:  0.92
```

Order doesn't matter. The final posterior after observing both causes is identical regardless of the order of observation. This is a consequence of the independence assumption: in this model, causes are assumed independent of each other (conditional on the risk state). When causes are independent, the joint likelihood factors: $P(C_1, C_2|\text{risk}) = P(C_1|\text{risk}) \times P(C_2|\text{risk})$, and multiplication commutes. If causes were dependent on each other, for example if observing one cause makes the other more likely, the model would need to account for that dependency and order could matter in sequential manual updates (though the final result should still be the same if done correctly via the joint distribution).

E.6. Sigmoidal Learning Curves (Chapter 7)

E.6.1. Exercise 3 : Confidence Band Width

Predict at weeks 3, 6, and 12. How wide is the 95% confidence band at each?

```
sig_data <- data.frame(
  week   = 1:10,
  output = c(10, 19, 30, 43, 55, 65, 73, 79, 84, 87)
)

fit <- fit_sigmoidal(sig_data, "week", "output", "logistic")

for (w in c(3, 6, 12)) {
  pred <- predict_sigmoidal(fit, w, "logistic")
  cat(sprintf("Week %2d: predicted: %5.1f\n", w, pred$pred))
}
```

```
Week  3: predicted:  29.4
Week  6: predicted:  65.7
Week 12: predicted:  87.8
```

```
plot_sigmoidal(fit, sig_data, "week", "output", "logistic")
```



Figure E.3.: Confidence band width increases with extrapolation distance. Predictions within the observed range (weeks 1–10) have narrow bands; week 12 extrapolates beyond the data.

Explanation. Confidence band width increases with extrapolation distance for two reasons. First, the model is more certain about fitted values near the centre of the observed data (weeks 4–7, where the sigmoidal is most informative). Second, extrapolation beyond week 10 relies entirely on the assumed functional form; the model has no data to constrain the prediction, and small errors in the estimated parameters compound. As a rule: trust predictions within the observed range; treat extrapolations as scenarios, not forecasts.

E.6.2. Exercise 5 : Beyond Completion Percentages

Choose an alternative interpretation of the sigmoidal model (cost efficiency, defect rates, or productivity).

Productivity (units per week).

Suppose you are tracking the output rate of a concrete placement crew (cubic metres per week) as they build a bridge deck. Early in the project, the crew is learning the site and the equipment; productivity rises rapidly then levels off.

Setup: - `x` = week number (time) - `y` = cubic metres placed per week - `K` (ceiling) = the crew's maximum sustainable output rate (estimated from similar past projects)

A fitted logistic model tells you: - The **inflection point** (t_0): the week when the crew is improving fastest; this is when you should schedule the most challenging pours - The **growth rate** (r): how quickly the crew learns; a high r means the learning curve is steep and full productivity is reached quickly - **Extrapolation**: predict the output rate at week n to inform the schedule for tasks that depend on a minimum pour rate

The model is useful here because productivity is bounded (even expert crews have a maximum throughput given equipment and site constraints) and the learning pattern is S-shaped: slow start, rapid improvement, plateau. The main caveat is that `K` must be estimated in advance; if the actual plateau differs, predictions in the upper portion of the curve will be biased.

E.7. Design Structure Matrices (Chapter 9)

E.7.1. Exercise 3 : Risk Propagation

Add Risk-3 (probability 0.40) affecting Resource-2 and Resource-3. Which task pair now shares a risk? Describe the worst-case scenario.

```
library(PRA)

nodes <- data.frame(
  id    = c("A", "B", "C", "D", "E", "F", "G", "H", "I"),
  label = c("Risk-1", "Risk-2",
            "Resource-1", "Resource-2", "Resource-3",
            "Task-1", "Task-2", "Task-3", "Project"),
  group = c("Risk", "Risk",
            "Resource", "Resource", "Resource",
            "Task", "Task", "Task", "Project"),
  stringsAsFactors = FALSE
)
links <- data.frame(
  source = c("A", "B", "C", "D", "E", "F", "G", "H"),
  target = c("C", "D", "F", "G", "H", "I", "I", "I"),
  value  = rep(1, 8),
  stringsAsFactors = FALSE
)
graph <- prob_net(nodes, links)

resource_ids <- graph$nodes$id[graph$nodes$group == "Resource"]
task_ids     <- graph$nodes$id[graph$nodes$group == "Task"]
risk_ids     <- graph$nodes$id[graph$nodes$group == "Risk"]

S <- graph$adjacency_matrix[resource_ids, task_ids]
rownames(S) <- graph$nodes$label[graph$nodes$group == "Resource"]
```

```
colnames(S) <- graph$nodes$label[graph$nodes$group == "Task"]

R <- graph$adjacency_matrix[risk_ids, resource_ids]
rownames(R) <- graph$nodes$label[graph$nodes$group == "Risk"]
colnames(R) <- graph$nodes$label[graph$nodes$group == "Resource"]

R2 <- rbind(R, "Risk-3" = c(0, 1, 1))
g2 <- grandparent_dsm(S, R2)
print(g2)
```

```
Risk-based 'Grandparent' Design Structure Matrix
Tasks: 3  Resources: 3  Risks: 3

       Task-1 Task-2 Task-3
Task-1      1      0      0
Task-2      0      2      1
Task-3      0      1      1
```

Worst-case scenario. The off-diagonal entry `G[Task-2, Task-3]` (and its mirror) is now 1: Task-2 and Task-3 share one risk (Risk-3). The worst-case scenario: Risk-3 materialises and simultaneously escalates the Developer (Resource-2) and QA Engineer (Resource-3) costs. Both Task-2 and Task-3 are hit at the same time. The coupling means the project cannot recover one task while the other runs normally — both are delayed together. The Grandparent DSM entry of 1 quantifies exactly this: one shared risk exposure connects the pair.

E.7.2. Exercise 5 : From DSM to Monte Carlo

Use the extended S2 (with a shared Project Manager) to derive a correlation matrix for MCS. Compare total SD under zero and DSM-derived correlation.

```
S2 <- rbind(S, "Project Manager" = c(1, 1, 0))

p2 <- parent_dsm(S2)
P  <- p2$matrix
d  <- sqrt(diag(P))
cor_from_dsm <- P / outer(d, d)
diag(cor_from_dsm) <- 1

cat("DSM-derived correlation matrix:\n")
```

```
DSM-derived correlation matrix:
```

```
print(round(cor_from_dsm, 2))
```

```
       Task-1 Task-2 Task-3
Task-1    1.0    0.5      0
Task-2    0.5    1.0      0
Task-3    0.0    0.0      1
```

```
task_dists <- list(
  "Task-1" = list(type = "normal", mean = 20250, sd = 5900),
  "Task-2" = list(type = "normal", mean = 62000, sd = 16000),
  "Task-3" = list(type = "normal", mean = 20000, sd = 4000)
)

set.seed(42)
res_zero <- mcs(10000, task_dists, diag(3))
res_dsm  <- mcs(10000, task_dists, cor_from_dsm)

cat("SD (zero correlation):  ", round(res_zero$total_sd, 0), "\n")
```

```
SD (zero correlation):   17655
```

```
cat("SD (DSM correlation):   ", round(res_dsm$total_sd,  0), "\n")
```

```
SD (DSM correlation):    19961
```

```
cat("Variance inflation:     ",
    round((res_dsm$total_sd^2 - res_zero$total_sd^2) / res_zero$total_sd^2 * 100, 1),
    "%\n")
```

```
Variance inflation:      27.8 %
```

What structural coupling adds. The base project (dedicated resources) yields `cor_from_dsm == diag(3)` — no coupling, no variance inflation. Adding the shared Project Manager introduces a non-zero correlation between Task-1 and Task-2, inflating total standard deviation above the independent baseline. DSM reveals *which* tasks are coupled; MCS quantifies *how much* that coupling increases risk.

E.8. Probabilistic Networks (Chapter 8)

E.8.1. Exercise 3 : Add a QA Risk

Add Risk-3 ($p = 0.40$) affecting QA Engineer, increasing cost from \$20K to \$35K.

```
nodes <- data.frame(
  id    = c("A", "B", "C_new", "C", "D", "E", "F", "G", "H", "I"),
  label = c("Risk-1", "Risk-2", "Risk-3",
            "Resource-1", "Resource-2", "Resource-3",
            "Task-1", "Task-2", "Task-3", "Project"),
  group = c("Risk","Risk","Risk","Resource","Resource","Resource",
            "Task","Task","Task","Project"),
  stringsAsFactors = FALSE
)

links <- data.frame(
  source = c("A", "B", "C_new", "C", "D", "E", "F", "G", "H"),
  target = c("C", "D", "E",     "F", "G", "H", "I", "I", "I"),
  value  = rep(1, 9),
  stringsAsFactors = FALSE
)

distributions <- list(
  A     = list(type = "discrete",   values = c(1,0), probs = c(0.70, 0.30)),
  B     = list(type = "discrete",   values = c(1,0), probs = c(0.60, 0.40)),
  C_new = list(type = "discrete",   values = c(1,0), probs = c(0.40, 0.60)),
  C     = list(type = "conditional", condition = "A",
    true_dist  = list(type = "normal", mean = 30000, sd = 8000),
    false_dist = list(type = "normal", mean = 15000, sd = 3000)),
  D     = list(type = "conditional", condition = "B",
    true_dist  = list(type = "normal", mean = 80000, sd = 20000),
    false_dist = list(type = "normal", mean = 50000, sd = 10000)),
  E     = list(type = "conditional", condition = "C_new",
    true_dist  = list(type = "normal", mean = 35000, sd = 7000),
    false_dist = list(type = "normal", mean = 20000, sd = 4000)),
  F     = list(type = "aggregate", nodes = c("C")),
  G     = list(type = "aggregate", nodes = c("D")),
  H     = list(type = "aggregate", nodes = c("E")),
  I     = list(type = "aggregate", nodes = c("F", "G", "H"))
)

graph_new <- prob_net(nodes, links, distributions = distributions)
sim_new   <- prob_net_sim(graph_new, num_samples = 10000)

cat("Mean total cost (with QA risk):", format(round(mean(sim_new$I)), big.mark=","), "\n")
```

```
Mean total cost (with QA risk): 119,402
```

```
cat("Expected QA cost increase:",
    format(round(mean(sim_new$H) - 20000), big.mark=","), "\n")
```

```
Expected QA cost increase: 5,777
```

Interpretation. The expected project cost rises by the probability-weighted cost increase for the QA Engineer: $0.40 \times (35{,}000 - 20{,}000) = \$6{,}000$. The actual simulated increase will be close to this but not exact due to the variance in the conditional distributions.

E.8.2. Exercise 5 : Two Risks, One Resource

Design a network where two risks both affect the same resource. What happens to task correlation?

A single `conditional` node can only key off one condition node, so "two risks both affect the same resource" is expressed as two risk-driven cost components (`Res1` for R1's effect, `Res2` for R2's effect) that are summed into the shared resource node `Res`:

```
nodes2 <- data.frame(
  id    = c("R1", "R2", "Res1", "Res2", "Res", "T1", "T2", "P"),
  label = c("Risk-1", "Risk-2", "Resource Cost (R1 effect)", "Resource Cost (R2 effect)",
            "Shared Resource", "Task-1", "Task-2", "Project"),
  group = c("Risk","Risk","Resource","Resource","Resource","Task","Task","Project"),
  stringsAsFactors = FALSE
)

links2 <- data.frame(
  source = c("R1","R2","Res1","Res2","Res","Res","T1","T2"),
  target = c("Res1","Res2","Res","Res","T1","T2","P","P"),
  value  = rep(1, 8),
  stringsAsFactors = FALSE
)

distributions2 <- list(
  R1   = list(type = "discrete", values = c(1,0), probs = c(0.50, 0.50)),
  R2   = list(type = "discrete", values = c(1,0), probs = c(0.40, 0.60)),
  Res1 = list(type = "conditional", condition = "R1",
    true_dist  = list(type = "normal", mean = 50000, sd = 10000),
    false_dist = list(type = "normal", mean = 25000, sd = 5000)),
  Res2 = list(type = "conditional", condition = "R2",
    true_dist  = list(type = "normal", mean = 30000, sd = 8000),
    false_dist = list(type = "normal", mean = 15000, sd = 3000)),
  Res  = list(type = "aggregate", nodes = c("Res1", "Res2")),
```

```
  T1   = list(type = "aggregate", nodes = c("Res")),
  T2   = list(type = "aggregate", nodes = c("Res")),
  P    = list(type = "aggregate", nodes = c("T1", "T2"))
)
```

Note: in this simplified network both tasks draw from the *same* resource node, and that resource's cost responds to *both* risks. Because T1 and T2 both equal Res, their simulated values will be perfectly correlated (correlation = 1). In a more realistic model, each task would have *additional* independent cost components, reducing the correlation below 1.

```
graph2 <- prob_net(nodes2, links2, distributions = distributions2)
sim2   <- prob_net_sim(graph2, num_samples = 10000)

cat("Correlation between T1 and T2:", round(cor(sim2$T1, sim2$T2), 3), "\n")
```

```
Correlation between T1 and T2: 1
```

Interpretation. Two tasks that *both* depend on the same resource share all of that resource's uncertainty. When a risk strikes the shared resource, both tasks are affected simultaneously and in the same direction, creating perfect positive correlation within the resource node. This is exactly what the DSM predicts structurally (Chapter 9): a high off-diagonal Parent DSM entry between these two tasks.

E.9. Portfolio Networks (Chapter 10)

E.9.1. Exercise 4 : Add a Fourth Project

This exercise requires reading `ch-network2.qmd` to obtain the existing three-project network structure. The solution below shows the pattern for extending the portfolio.

```
# Pattern for adding a fourth project (Bridge Inspection) to the portfolio
# Assumes nodes A, B, C are enterprise risks already in the network
# New nodes: L4 (Labor-4), M4 (Materials-4), E4 (Equipment-4),
#            W4 (Work-4), X (Portfolio total)

# Add to nodes data frame:
new_nodes <- data.frame(
  id    = c("L4", "M4", "E4", "W4"),
  label = c("Labor-4","Materials-4","Equipment-4","Bridge Inspection"),
  group = c("Resource","Resource","Resource","Project"),
  stringsAsFactors = FALSE
)
```

```
# Add edges: enterprise risks → new resources, new resources → W4, W4 → portfolio
new_links <- data.frame(
  source = c("A","B","C","L4","M4","E4","W4"),
  target = c("L4","M4","E4","W4","W4","W4","Y"),
  value  = rep(1,7),
  stringsAsFactors = FALSE
)
```

Expected result. Adding a fourth project sharing the same enterprise risks increases total portfolio variance by more than the fourth project's standalone variance, since the covariance terms between the new project and the existing three add to the total. The risk importance ranking is unlikely to shift dramatically, but any enterprise risk that was already dominant (highest contribution to existing portfolio variance) will become even more dominant in the extended portfolio.

E.9.2. Exercise 5 : Causal Graph Design

Add SC → C edge: Supply Chain Disruption also affects Weather Delay through logistics.

This exercise is primarily conceptual; the answer requires reasoning about the see-versus-do distinction.

Answer. Adding SC → C means that C (Weather Delay) is no longer a root variable; it becomes conditional on SC. When you *see* C = Yes, you're now also updating your belief about SC, which in turn shifts your beliefs about A (Labor Shortage) and B (Material Price Spike), because all three share SC as a parent. This creates *explaining away*: if you observe C and you know that SC explains it, you become less certain that some independent weather phenomenon caused C.

The do-calculus implication: `prob_net_update()` (graph surgery) would remove the SC → C edge when computing $P(Y|\text{do}(C = \text{No}))$, preventing the intervention from propagating back up to SC and then down to A and B. In contrast, `prob_net_learn()` (conditioning) would propagate through all paths, including the SC back-channel. The see-versus-do distinction becomes consequential precisely because of this shared upstream cause.

E.10. Agentic Risk Analysis (Chapter 12)

E.10.1. Exercise 5 : Skill Authoring

This exercise is open-ended; below is a worked example of what a good project-specific skill looks like and how to verify it.

Setup. Register the MCP server, then read one of the bundled skills as a template:

```
claude mcp add -s project pra -- Rscript -e "PRA::pra_mcp_server()"
```

```
readLines(system.file("skills", "pra-monte-carlo", "SKILL.md", package = "PRA"))
```

A worked skill. Suppose your organization always sizes reserves at the 80th percentile. A skill capturing that policy might read:

```
---
name: pra-p80-reserve
description: >
  Apply the organization's standard P80 contingency policy. Use whenever a
  user asks for a reserve, buffer, or contingency on a simulated project.
---

# When to use
Any request for a contingency reserve on a project with task-level
uncertainty distributions.

# How to run it
1. Call `mcs_tool` with the task distributions (and a correlation matrix if
   tasks are correlated).
2. Call `contingency_tool` with `phigh = 0.80, pbase = 0.50`.
3. Report the reserve and note that the P80 policy was applied.
```

Signs of success. A well-formed skill has valid YAML frontmatter (a `name` matching its folder and a `description` written for trigger accuracy), states clearly *when* to apply it, and maps each step to a specific tool and its arguments. Verify it by giving the agent a matching request and confirming it calls `mcs_tool` then `contingency_tool` at `phigh = 0.80`, reproducing the reserve you would get from `contingency(sims, phigh = 0.80, pbase = 0.50)` directly.

References

Benjamin, Jack R., and C. Allin Cornell. 2000. *Probability, Statistics, and Decision for Civil Engineers*. Dover Publications.

Browning, Tyson R. 2001. "Applying the Design Structure Matrix to System Decomposition and Integration Problems: A Review and New Directions." *IEEE Transactions on Engineering Management* 48 (3): 292–306. https://doi.org/10.1109/17.946528.

Fleming, Quentin W., and Joel M. Koppelman. 2010. *Earned Value Project Management*. 4th ed. Project Management Institute.

Govan, Paul. 2014. "A Resource-Based View on Project Risk Management." PhD thesis, The University of Texas at Austin.

Govan, Paul, and Ivan Damnjanovic. 2016. "The Resource-Based View on Project Risk Management." *Journal of Construction Engineering and Management* 142 (9): 04016034. https://doi.org/10.1061/(ASCE)CO.1943-7862.0001157.

Pearl, Judea. 2009. *Causality: Models, Reasoning, and Inference*. 2nd ed. Cambridge University Press.

Project Management Institute. 2021. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 7th ed. Project Management Institute.

Steward, D. V. 1981. *Systems Analysis and Management: Structure, Strategy and Design*. Petrocelli Books.

Vose, David. 2008. *Risk Analysis: A Quantitative Guide*. 3rd ed. John Wiley & Sons.